

AI Robotics

Artificial Intelligence

Jacopo Parretti

II Semester 2025–2026

Contents

I	Mobile Robotics: Introduction	11
1	Autonomous Mobile Robots	11
1.1	The Three Key Questions	11
1.2	General Control Scheme	11
2	Kinematics and Motion Control	11
2.1	Wheel Types and Constraints	11
2.2	Forward and Inverse Kinematics	12
3	Perception	12
3.1	Sensing Modalities	12
4	Localization and Mapping	12
4.1	Localization: The Core Problem	12
4.2	Markov Localization	12
4.3	Visual Odometry	13
5	Cognition	13
5.1	Obstacle Avoidance	13
5.2	Global Path Planning	13
5.3	Decision Making	13
6	Learning	14
6.1	Deep Reinforcement Learning for Robotic Navigation	14
II	Wheeled Locomotion: Kinematics	14
7	Introduction to Kinematics for Mobile Robots	14
7.1	Wheeled Locomotion: Motivation	14
7.2	Mobile Robots vs. Manipulator Arms	14
7.3	Non-Holonomic Systems	15
7.4	Forward and Differential Kinematics	15
8	Wheel Arrangement	15
8.1	Three Design Concepts	15
8.2	Basic Wheel Types	15

8.3	Wheel Configurations and Stability	16
9	Mobile Robot Kinematics	16
9.1	Representing Robot Position	16
9.2	Frame Mapping: Rotation Matrix	16
9.3	Forward Kinematic Model (Differential Drive)	16
9.4	Wheel Kinematic Constraints	17
10	Mobile Robot Maneuverability	18
10.1	Degrees of Mobility and Steerability	18
10.2	Five Basic Three-Wheel Configurations	18
11	Motion Control	18
11.1	Overview	18
11.2	Open Loop Control	18
11.3	Feedback Control	18
III	Perception: Introduction and Sensors	20
12	Introduction to Perception	20
12.1	Perception in the Control Loop	20
12.2	Perception as Information Compression	20
12.3	Challenges	20
13	Sensor Classification	21
13.1	What They Measure	21
13.2	How They Measure	21
13.3	Sensor Classification Tables	21
14	Individual Sensor Types	22
14.1	Encoders	22
14.2	Heading Sensors	22
14.3	Accelerometers	22
14.4	Inertial Measurement Units (IMU)	23
14.5	Beacons and GPS	23
14.6	Range Sensors	23
IV	Perception: Vision Basics	24
15	Introduction to Vision	24
15.1	Human Vision	24
15.2	Challenges	25
16	Camera and Image Formation	25
16.1	Digital Cameras: CCD and CMOS	25
16.2	Color Cameras	25
16.3	Pinhole Camera Model	26
16.4	Why Lenses? The Thin Lens Equation	26
16.5	Pinhole Approximation	26
17	Perspective Projection	27

17.1 From Scene Points to Image Plane	27
17.2 From Image Coordinates to Pixel Coordinates	27
17.3 The Intrinsic Camera Matrix	27
17.4 General Case: Extrinsic Parameters	27
18 Radial Distortion	28
19 Camera Calibration	29
19.1 Goal	29
19.2 Direct Linear Transform (DLT)	29
19.3 Zhang's Planar Grid Method	29
V Perception: Stereo Vision	30
20 Structure from Stereo: Motivation	30
20.1 Stereo Vision vs. Structure from Motion	30
21 Stereo Vision: Working Principle	31
21.1 Triangulation	31
21.2 Simplified Case: Aligned Cameras	31
21.3 General Case	32
22 Correspondence Search	32
22.1 Area-Based Methods	32
22.2 Feature-Based Methods	32
22.3 Epipolar Constraint	32
23 Epipolar Rectification	33
23.1 Rectification Pipeline	33
24 Disparity Maps	33
25 Structure from Motion	34
VI Perception: Line Extraction from Range Data	34
26 Introduction	34
27 Probabilistic Line Fitting	35
27.1 Line Representation in Polar Coordinates	35
27.2 Least-Squares Estimation	35
28 Line Extraction Algorithms	35
28.1 Split and Merge	35
28.2 Line Regression	36
28.3 RANSAC	36
28.4 Hough Transform	37
29 Algorithm Comparison	38
VII Localization: Recursive State Estimation	38

30 Introduction to Localization	39
30.1 The Localization Problem	39
30.2 Why Probabilistic?	39
31 Probability Background	39
31.1 Discrete and Continuous Random Variables	39
31.2 Joint, Conditional, and Marginal Probability	39
31.3 Bayes' Rule	40
31.4 Conditional Independence	40
32 The Bayes Filter	40
32.1 Setup and Belief	40
32.2 The Markov Assumption	41
32.3 Derivation of the Recursive Update	41
32.4 Prediction and Correction	41
32.5 Worked Example: Door State Estimation	42
33 Realizations of the Bayes Filter	42
VIII Localization: Kalman Filter	43
34 The Kalman Filter	43
34.1 Overview	43
34.2 The Gaussian Belief	43
34.3 Linear Motion and Observation Models	43
34.4 Closure under Gaussian Belief	44
34.5 The Kalman Filter Algorithm	44
34.6 Interpreting Each Step	44
35 Mobile Robot Localization with the KF	45
35.1 Markov Localization	45
35.2 KF Localization for Feature-Based Maps	45
35.3 Localization Taxonomy	45
36 Extended Kalman Filter	46
36.1 Motivation	46
36.2 Nonlinear Models	46
36.3 First-Order Taylor Linearization	46
36.4 EKF Algorithm	46
36.5 When Does EKF Work?	47
36.6 Extensions	47
IX Localization: Particle Filters and Monte Carlo Localization	48
37 Non-Parametric Belief Representation	48
37.1 Motivation	48
37.2 Sample-Based Distribution Representation	48
38 Importance Sampling	48
38.1 The Core Idea	48

39 The Particle Filter Algorithm	49
39.1 Predict, Weight, Resample	49
39.2 Why This Works	50
40 Resampling	50
40.1 Purpose	50
40.2 Roulette Wheel Sampling	50
40.3 Low-Variance (Systematic) Resampling	50
41 Monte Carlo Localization	51
41.1 Algorithm	51
41.2 The MCL Cycle	51
41.3 Global Localization	51
42 Practical Considerations	52
42.1 Pros and Cons of Particle Filters	52
42.2 Particle Depletion	52
42.3 Variants	52
X Simultaneous Localization and Mapping	52
43 The SLAM Problem	52
43.1 Motivation	52
43.2 Problem Formulations	53
43.3 Key Challenges	53
44 Grid-Based SLAM with Rao-Blackwellized Particle Filters	53
44.1 Factoring the Joint Distribution	53
44.2 Particle Filter over Trajectories	54
44.3 Improved Proposal via Scan Matching	54
45 Graph-Based SLAM	54
45.1 Pose Graph Representation	54
45.2 Front-End: ICP Scan Matching	55
45.3 Back-End: Gauss-Newton Optimization	55
45.4 Sparsity and Scalability	56
46 SLAM in Practice	57
46.1 Comparison of Approaches	57
46.2 Data Association	57
46.3 Loop Closure	57
XI Occupancy Grid Maps	57
47 Map Representations	57
47.1 Why Maps Matter	57
47.2 Mapping with and without Known Poses	58
48 Occupancy Grid Maps	58
48.1 Definition and Assumptions	58
48.2 Factorization of the Map Distribution	58

49 Binary Bayes Filter for Static States	59
49.1 Derivation of the Update Rule	59
49.2 Log Odds Representation	59
50 Occupancy Grid Mapping Algorithm	60
51 Inverse Sensor Models	60
51.1 Role of the Inverse Sensor Model	60
51.2 Laser Rangefinder Model	61
51.3 Sonar Model	61
52 Map Quality and Practical Considerations	61
52.1 Sensor Comparison	61
52.2 Comparison with Feature Maps	62
52.3 Summary of Key Properties	62
XII Probabilistic Observation Models	62
53 Role of the Observation Model	62
54 Range Sensors and the Scan Factorization	62
55 Beam-Based Model (Advanced Ray-Cast Model)	63
55.1 Physical Decomposition	63
55.2 Component 1: Correct Range with Gaussian Noise (p_{hit})	63
55.3 Component 2: Unexpected Obstacle (p_{short})	63
55.4 Component 3: Random Measurement (p_{rand})	64
55.5 Component 4: Maximum Range Failure (p_{max})	64
55.6 Parameter Learning	64
56 Likelihood Field Model	64
56.1 Motivation	64
56.2 Construction	64
56.3 Sensor Update	65
56.4 Comparison of Beam and Likelihood Field Models	65
57 Landmark-Based Observation Models	65
57.1 Range-Bearing Model	65
57.2 Bearing-Only Model	66
58 Summary	66
XIII Probabilistic Motion Models	66
59 Motion Uncertainty and the Motion Model	66
59.1 Why Model Uncertainty?	66
59.2 Role in the Bayes Filter	67
59.3 Two Model Families	67
60 Odometry Motion Model	67
60.1 Motion Decomposition	67
60.2 Noise Model	67

60.3 Noise Distributions	68
60.4 Computing $P(x_t \mid u_t, x_{t-1})$: Density Evaluation	68
60.5 Sampling from $P(x_t \mid u_t, x_{t-1})$: Particle Generation	69
61 Velocity-Based Motion Model	69
61.1 Kinematics	69
61.2 Noise Model	69
61.3 State Update with Noise	70
61.4 Resulting Distributions	70
62 Summary	70
 XIV Robot Motion Planning and Navigation	 71
63 Overview and Historical Context	71
63.1 What is Motion Planning?	71
63.2 Historical Development	71
64 Configuration Space	71
64.1 Definitions	71
64.2 Mobile Robot C-Space Simplifications	72
65 Combinatorial C-Space Representations	72
65.1 General Strategy	72
65.2 Visibility Graph	72
65.3 Voronoi Diagram	72
65.4 Exact Cell Decomposition	72
65.5 Approximate Cell Decomposition	73
66 Potential Field Methods	73
67 Sampling-Based Methods	73
67.1 Motivation	73
67.2 Probabilistic Road Map (PRM)	73
67.3 Rapidly Exploring Random Trees (RRT)	74
68 Graph-Search Algorithms	74
68.1 Problem Setup and Complexity Metrics	74
68.2 Uninformed Search: Algorithm Comparison	74
68.3 A* Search	74
69 Multi-Agent Path Finding (MAPF)	75
69.1 Problem Definition	75
69.2 Naive Approach and Its Limitations	75
69.3 Independent Detection (ID)	75
69.4 Conflict-Based Search (CBS)	75
70 Multi-Agent Pickup and Delivery (MAPD)	76
70.1 From One-Shot to Lifelong Planning	76
70.2 Well-Formed MAPD Instances	76
70.3 Token Passing (TP)	76
70.4 Centralized Assignment (CENTRAL)	77
70.5 Algorithm Trade-offs	77

71 Obstacle Avoidance	77
71.1 Bug Algorithms	77
71.2 Dynamic Window Approach (DWA)	77
72 Summary	78
 XV Value-Based Deep Reinforcement Learning	 78
73 Markov Decision Processes	78
73.1 Formal Definition	78
73.2 Markov and Stationarity Assumptions	79
73.3 MDP Formulation for Mobile Robots	79
74 Value Functions and Q-Values	79
74.1 State Value Function	79
74.2 Action-Value (Q) Function	79
75 Tabular Q-Learning	79
75.1 Model-Free RL	79
75.2 Q-Learning Update Rule	80
75.3 Limitations of Tabular Q-Learning	80
76 Neural Network Function Approximation	80
76.1 Feedforward Neural Networks	80
76.2 Activation Functions	80
76.3 Universal Approximation Theorem	81
76.4 Training: Gradient Descent and Backpropagation	81
76.5 Vanishing Gradients and ReLU	81
77 Deep Q-Learning	81
77.1 Gradient Q-Learning	81
77.2 Convergence Issues	82
78 Deep Q-Network (DQN)	82
78.1 Algorithm	82
79 DRL for Robotics	83
79.1 Challenges and Sim-to-Real Transfer	83
79.2 Continuous vs Discrete Action Spaces	83
79.3 Extensions to Non-Stationary Environments	83
80 Summary	83
 XVI Policy-Based and Actor-Critic Deep RL	 84
81 Taxonomy of Deep RL Approaches	84
82 Stochastic Policy Representations	84
83 The Policy Gradient Theorem	85
83.1 Objective	85
83.2 Theorem Statement	85

84 REINFORCE (Monte Carlo Policy Gradient)	85
84.1 Algorithm	85
84.2 Interpretation of the Update	86
84.3 Limitation: High Variance	86
85 Baseline and Variance Reduction	86
85.1 Adding a Baseline	86
85.2 Advantage Function	86
85.3 REINFORCE with Baseline	86
86 Actor-Critic Methods	86
86.1 From Monte Carlo to Temporal Difference	86
86.2 Actor-Critic (AC) Algorithm	87
86.3 Advantage Actor-Critic (A2C)	87
87 Deterministic Policy Gradient (DPG)	87
88 Trust Region Methods	88
88.1 Step-Size Problem in Policy Gradient	88
88.2 Trust Region Optimization	88
88.3 KL Divergence as Policy Distance	89
89 Trust Region Policy Optimization (TRPO)	89
90 Proximal Policy Optimization (PPO)	89
90.1 Clipped Surrogate Objective	89
90.2 PPO Algorithm	89
91 Soft Actor-Critic (SAC)	90
91.1 Entropy Regularization	90
91.2 Soft Greedy Policy	90
91.3 Benefits	90
92 Summary	90
 XVII Planning under Uncertainty	 91
93 From Path Planning to Decision-Theoretic Planning	91
93.1 The Two Regimes	91
93.2 Problem Taxonomy	91
94 Markov Decision Processes (MDPs)	92
94.1 Formal Definition	92
94.2 Policies	92
94.3 Cumulative Reward and Planning Horizon	92
94.4 Value and Q-Functions	92
94.5 Optimal Policy and Bellman Equations	93
95 Solving MDPs	93
95.1 Value Iteration	93
95.2 Policy Iteration	93
96 Partially Observable MDPs (POMDPs)	94

96.1 Motivation: Partial Observability	94
96.2 Belief State	94
96.3 POMDP Formulation	94
96.4 Exact Solution: α -Vectors	94
97 Monte Carlo Tree Search (MCTS)	95
97.1 Core Idea	95
98 POMCP: Monte Carlo Planning in POMDPs	96
98.1 Challenges of Exact POMDP Solvers	96
98.2 POMCP Approach	96
98.3 Comparison: MDP vs POMDP Planning	97
99 Summary	97

Part I

Mobile Robotics: Introduction

1 Autonomous Mobile Robots

1.1 The Three Key Questions

Mobile robotics revolves around three fundamental questions that any autonomous system must answer:

1. **Where am I?** → Perception + Localization + Mapping
2. **Where am I going?** → Planning + Path Planning
3. **How do I get there?** → Obstacle Avoidance + Kinematics

To answer these questions, the robot must:

- have a **model** of the environment
- **perceive** and analyze the environment
- find its **position/situation** in the environment
- **plan** and execute the movement

1.2 General Control Scheme

The general architecture of an autonomous mobile robot can be decomposed into two main pipelines:

- **Perception pipeline:** Sensing → Information Extraction and Interpretation → Localization and Map Building
- **Motion control pipeline:** Cognition and Path Planning → Path Execution → Acting

The *Localization/Map Building* block feeds a position estimate and a global map to the *Cognition* block, which uses mission commands from the knowledge base to plan a path. The path is then passed to *Path Execution*, which sends actuator commands to the *Acting* block that interacts with the real-world environment. The loop is closed by sensing the resulting state.

2 Kinematics and Motion Control

2.1 Wheel Types and Constraints

Wheeled mobile robots are subject to two fundamental kinematic constraints:

- **Rolling constraint:** the wheel rolls without slipping in the rolling direction — velocity component along the wheel plane must be consistent with wheel rotation.
- **No-sliding constraint:** the wheel does not slip laterally — zero velocity component orthogonal to the wheel plane.

Different wheel types (fixed standard, steerable, castor, Swedish/Mecanum) impose different combinations of these constraints and determine the robot's degrees of freedom and maneuverability.

2.2 Forward and Inverse Kinematics

Let (x, y, θ) be the robot pose in the global frame and $\varphi_1, \dots, \varphi_n$ the wheel rotation speeds. The **forward kinematics** (wheels $\rightarrow$ body velocity) and **inverse kinematics** (body velocity $\rightarrow$ wheels) are:

$$\begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \end{bmatrix} = f(\dot{\varphi}_1, \dots, \dot{\varphi}_n, \theta, \text{geometry}) \quad \begin{bmatrix} \dot{\varphi}_1 \\ \vdots \\ \dot{\varphi}_n \end{bmatrix} = f(\dot{x}, \dot{y}, \dot{\theta})$$

The geometry of the robot (wheel placement, radius, orientation) determines the specific form of f .

3 Perception

3.1 Sensing Modalities

Perception is the process of extracting meaningful information from raw sensor data. Common sensors on mobile robots include:

- **Lidar** (2D or 3D): provides range measurements by emitting laser pulses and measuring time-of-flight. Produces dense point clouds or scan profiles used for obstacle detection, mapping, and localization.
- **Cameras**:
 - *Stereo*: two horizontally offset cameras enabling triangulation-based depth estimation.
 - *Omnidirectional*: wide field-of-view imaging via catadioptric optics.
 - *RGB-D*: combines color image with per-pixel depth (structured light or ToF).

Laser scans are frequently overlaid on occupancy grid maps for real-time spatial reasoning.

4 Localization and Mapping

4.1 Localization: The Core Problem

Localization is the task of estimating the robot's pose (x, y, θ) relative to a known or partially known map. It is inherently probabilistic: sensors are noisy, and motion estimation via encoders accumulates error over time.

4.2 Markov Localization

A classical probabilistic approach to localization proceeds through a **sense–act–sense** cycle:

1. **SEE**: robot queries its sensors $\rightarrow$ observes a landmark (e.g. a door). The belief distribution is updated (sharpened at consistent hypotheses).
2. **ACT**: robot moves forward. Motion is estimated via encoders; uncertainty grows (belief spreads).
3. **SEE**: robot observes again. The new observation is fused with the propagated belief via Bayes' rule.

Belief Update

At each step, the robot maintains a probability distribution over poses — the **belief** $\text{bel}(x_t)$. It is updated by:

$$\text{bel}(x_t) = \eta p(z_t | x_t) \int p(x_t | u_t, x_{t-1}) \text{bel}(x_{t-1}) dx_{t-1}$$

where z_t is the sensor observation, u_t is the control input, and η is a normalizing constant.

4.3 Visual Odometry

An alternative to encoder-based motion estimation is **visual odometry**: estimating ego-motion from consecutive camera frames. Modern methods such as *Direct Sparse Odometry* (DSO) operate directly on pixel intensities without requiring explicit feature matching, achieving robust and accurate trajectory estimation.

5 Cognition

5.1 Obstacle Avoidance

Obstacle avoidance employs **local or reactive methods** to steer the robot toward its goal while avoiding collisions in real time. Two representative approaches:

- **Bug algorithms** (e.g. Bug2): the robot moves in a straight line toward the goal and follows obstacle boundaries when a collision is imminent. Simple and provably complete for certain environments.
- **Vector Field Histogram (VFH)**: builds a polar histogram of obstacle density around the robot; selects a steering direction in a sector with low obstacle density aligned with the goal direction.

5.2 Global Path Planning

Global path planning computes a **collision-free path** from the robot's current position to the goal using a prior map. Representative methods:

- **Voronoi diagram**: paths are planned along the medial axis of the free space, maximizing clearance from obstacles.
- **Cell decomposition**: the free space is partitioned into cells; a graph is built over adjacent free cells and a graph-search algorithm (e.g. A*) finds the optimal path.

5.3 Decision Making

Beyond reactive and planned navigation, a robot may need to **regulate its behavior online** — e.g. adjust velocity to complete a path as fast as possible while avoiding accidents under uncertainty.

Online Monte Carlo Planning (in particular POMCP — Partially Observable Monte Carlo Planning) addresses this by:

- modeling the task as a POMDP (Partially Observable Markov Decision Process)
- using Monte Carlo Tree Search to plan online without a full model of the environment

- incorporating **prior knowledge** and handling **state uncertainty** explicitly through belief states

6 Learning

6.1 Deep Reinforcement Learning for Robotic Navigation

When explicit maps are unavailable or the environment is non-stationary, **Deep Reinforcement Learning (DRL)** provides a powerful paradigm for learning navigation policies directly from experience.

Mapless navigation for wheeled robots: a DRL agent (e.g. DDQN with discrete action space) is trained in simulation (Unity) and transferred to a real wheeled platform. Simulation significantly reduces training time while preserving real-world success rate.

Mapless navigation for ASVs (Autonomous Surface Vehicles): DRL is applied to navigation in challenging non-stationary scenarios where real-world experiments are difficult. **Formal verification** methods are additionally used to identify critical configurations and guarantee safety properties of the learned policy.

Part II

Wheeled Locomotion: Kinematics

7 Introduction to Kinematics for Mobile Robots

7.1 Wheeled Locomotion: Motivation

Wheeled locomotion is:

- very efficient on flat/hard surfaces
- restricted to man-made infrastructures
- the de-facto standard for mobile robots

7.2 Mobile Robots vs. Manipulator Arms

- **Robotic arms:** fixed to the ground; single chain of actuated links (usually).
- **Mobile robots:** position is unbounded; subject to rolling and sliding constraints at the wheel-ground contact point.

Key consequences for mobile robot kinematics:

- encoder values do not map to unique poses — the workspace is unbounded
- no direct way to measure pose: positions must be integrated over time (path-dependent)
- integration accumulates errors in position/motion estimates
- the constraints imposed by wheels on mobility must be understood

7.3 Non-Holonomic Systems

Non-Holonomic System

A system is **non-holonomic** if the kinematic constraints cannot be integrated to a finite position constraint. Knowing the travel distance of each wheel is insufficient to determine the final pose — one must know the full trajectory as a function of time.

This is in sharp contrast with robotic arms, where encoder positions map directly to the end-effector pose.

7.4 Forward and Differential Kinematics

For mobile robots, the relevant kinematic problems are:

- **Forward kinematics:** given actuator positions (or speeds), determine the corresponding pose (or body velocity).
- **Inverse kinematics:** given a desired pose, determine the corresponding actuator positions — required for motion control.
- **Differential forward kinematics:** given actuator speeds $\dot{\varphi}_i$, determine body velocity $\dot{\xi}$.
- **Differential inverse kinematics:** given a desired body velocity, determine the required actuator speeds.

Because non-holonomic constraints prevent direct integration, wheels provide only *differential* inverse kinematics:

$$\begin{bmatrix} \dot{x} \\ \dot{y} \end{bmatrix} = \begin{bmatrix} \dot{\varphi} r \\ 0 \end{bmatrix}$$

8 Wheel Arrangement

8.1 Three Design Concepts

The choice of wheel configuration involves three competing properties:

- **Stability** (static configuration): three wheels are sufficient to guarantee stability, as the center of gravity lies within their support triangle. More than three wheels require a flexible suspension system (hyperstatic configuration).
- **Maneuverability:** how the robot can move in the environment.
- **Controllability:** how actuators must be commanded to achieve motion.

No single configuration maximizes all three — the selection depends on the application.

8.2 Basic Wheel Types

- **Standard (fixed) wheel** — 2 DOF: rotation around the (motorized) wheel axle and the contact point.
- **Castor wheel** — 3 DOF: rotation around the wheel axle, the contact point, and the castor axle (offset pivot).
- **Swedish (Mecanum) wheel** — 3 DOF: rotation around the wheel axle, the contact point, and around the passive rollers mounted on the rim. Allows motion in any direction.
- **Spherical wheel** — 3 DOF: rotation around the three orthogonal axes.

8.3 Wheel Configurations and Stability

- Stability guaranteed with three wheels: center of gravity (CoG) must lie within the triangle formed by the ground contact points.
- Four or more wheels improve stability but create a hyperstatic arrangement requiring flexible suspension.
- Bigger wheels overcome higher obstacles, at the cost of higher torque or gear reduction (increased complexity).
- Most arrangements are non-holonomic $\Rightarrow$ higher control effort.
- Actuation and steering on the same wheel leads to complex design and additional odometry errors.

Common layouts include 2-, 3-, 4-, and 6-wheel configurations. A notable example is the **CMU Uranus**: four independent Swedish wheels at 45° , enabling motion along any trajectory on the plane and simultaneous spinning about the vertical axis.

9 Mobile Robot Kinematics

9.1 Representing Robot Position

The robot chassis is modeled as a rigid body on a planar environment. Two reference frames are used:

- **Global frame** $\{x_I, y_I\}$: inertial (world) frame.
- **Local frame** $\{x_R, y_R\}$: robot-fixed frame.

The robot pose is described by a reference point P with 3 DOF:

$$\xi_I = \begin{bmatrix} x \\ y \\ \theta \end{bmatrix}$$

where (x, y) is the position of P in the global frame and θ is the heading angle.

9.2 Frame Mapping: Rotation Matrix

Orthogonal rotation matrices map between local and global frames:

$$R(\theta) = \begin{bmatrix} \cos \theta & \sin \theta & 0 \\ -\sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

The velocity in the robot frame relates to the global velocity by:

$$\dot{\xi}_R = R(\theta) \dot{\xi}_I$$

9.3 Forward Kinematic Model (Differential Drive)

For a differential drive robot with wheel radius r , axle half-length l , wheel spinning speeds $\dot{\varphi}_1$ (right) and $\dot{\varphi}_2$ (left):

Forward Kinematics: Differential Drive

Local frame velocity:

$$\begin{bmatrix} \dot{x}_R \\ \dot{y}_R \\ \dot{\theta}_R \end{bmatrix} = \begin{bmatrix} \frac{r\dot{\varphi}_1}{2} + \frac{r\dot{\varphi}_2}{2} \\ 0 \\ \frac{r\dot{\varphi}_1}{2l} - \frac{r\dot{\varphi}_2}{2l} \end{bmatrix}$$

Global frame velocity (via $\dot{\xi}_I = R(\theta)^{-1}\dot{\xi}_R$):

$$\begin{bmatrix} \dot{x}_I \\ \dot{y}_I \\ \dot{\theta}_I \end{bmatrix} = \begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \frac{r\dot{\varphi}_1}{2} + \frac{r\dot{\varphi}_2}{2} \\ 0 \\ \frac{r\dot{\varphi}_1}{2l} - \frac{r\dot{\varphi}_2}{2l} \end{bmatrix}$$

9.4 Wheel Kinematic Constraints

Individual wheel constraints are derived under assumptions: wheel in vertical plane, single contact point, no sliding. Two constraint types are imposed for each wheel:

Fixed Standard Wheel

Parameters: α (wheel orientation w.r.t. robot frame), β (steering angle, fixed here), l (distance from P), r (radius).

- **Rolling constraint:** all motion in the wheel plane must be due to rotation,

$$[\sin(\alpha + \beta) \quad -\cos(\alpha + \beta) \quad (-l) \cos \beta] R(\theta) \dot{\xi}_I - r\dot{\varphi} = 0$$

- **Sliding (no-slip) constraint:** no motion perpendicular to wheel plane,

$$[\cos(\alpha + \beta) \quad \sin(\alpha + \beta) \quad l \sin \beta] R(\theta) \dot{\xi}_I = 0$$

Swedish Wheel

Rolling constraint is modified by the roller angle γ ; there is no constraint on the orthogonal direction (passive rollers absorb lateral motion):

$$[\sin(\alpha + \beta + \gamma) \quad -\cos(\alpha + \beta + \gamma) \quad (-l) \cos(\beta + \gamma)] R(\theta) \dot{\xi}_I - r\dot{\varphi} \cos \gamma = 0$$

Complete Robot Model

For a robot with $M = N_f + N_s$ wheels (N_f fixed, N_s steerable), only fixed and steerable standard wheels impose constraints. Stacking all constraints:

$$J_1(\beta_s) R(\theta) \dot{\xi}_I - J_2 \dot{\varphi} = 0 \quad (\text{rolling})$$

$$C_1(\beta_s) R(\theta) \dot{\xi}_I = 0 \quad (\text{no lateral slip})$$

where $J_2 = \text{diag}(r_1, \dots, r_M)$ and J_1, C_1 encode the geometric parameters of each wheel.

10 Mobile Robot Maneuverability

10.1 Degrees of Mobility and Steerability

Maneuverability

Robot maneuverability $\delta_M = \delta_m + \delta_s$ where:

- δ_m : **degree of mobility** — DoF that can be directly manipulated through changes in wheel velocity (from the sliding constraint null space).
- δ_s : **degree of steerability** — number of independently controllable steering parameters.

10.2 Five Basic Three-Wheel Configurations

Configuration	δ_M	δ_m	δ_s
Omnidirectional	3	3	0
Differential drive	2	2	0
Omni-steer	3	2	1
Tricycle	2	1	1
Two-steer	3	1	2

11 Motion Control

11.1 Overview

The objective of a kinematic controller is to follow a trajectory described by position and/or velocity profiles as a function of time. Motion control is challenging because:

- mobile robots are typically non-holonomic
- they are Multi-Input Multi-Output (MIMO) systems

Most controllers (including those covered here) operate at the kinematic level and do not account for system dynamics.

11.2 Open Loop Control

Given a trajectory, decompose it into **motion segments** of clearly defined shape (straight lines or circular arcs). Pre-compute a smooth trajectory for each segment.

Drawbacks:

- pre-computing trajectories that incorporate robot constraints is non-trivial
- dynamic changes in the environment cannot be handled
- resulting overall trajectory may not be smooth at segment transitions

11.3 Feedback Control

Find a control matrix K (with $k_{ij} = k(t, e)$) such that:

$$\begin{bmatrix} v(t) \\ \omega(t) \end{bmatrix} = K \cdot e = K \cdot \begin{bmatrix} x \\ y \\ \theta \end{bmatrix}$$

drives the error to zero: $\lim_{t \rightarrow \infty} e(t) = 0$.

Kinematic Position Control for Differential Drive

The kinematics of a differential drive in the inertial frame are:

$$\begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \end{bmatrix} = \begin{bmatrix} \cos \theta & 0 \\ \sin \theta & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} v \\ \omega \end{bmatrix}$$

Assume w.l.o.g. the goal is at the origin of the inertial frame. Let α denote the angle between the robot's x_R axis and the vector from the wheel axle center to the goal.

Polar Coordinate Transformation

Transform to polar coordinates centered at the goal:

$$\rho = \sqrt{\Delta x^2 + \Delta y^2}, \quad \alpha = -\theta + \text{atan2}(\Delta y, \Delta x), \quad \beta = -\theta - \alpha$$

The system dynamics in polar coordinates (for $\alpha \in I_1 = (-\frac{\pi}{2}, \frac{\pi}{2})$):

$$\begin{bmatrix} \dot{\rho} \\ \dot{\alpha} \\ \dot{\beta} \end{bmatrix} = \begin{bmatrix} -\cos \alpha & 0 \\ \frac{\sin \alpha}{\rho} & -1 \\ -\frac{\sin \alpha}{\rho} & 0 \end{bmatrix} \begin{bmatrix} v \\ \omega \end{bmatrix}$$

For $\alpha \in I_2 = (-\pi, -\frac{\pi}{2}] \cup [\frac{\pi}{2}, \pi]$, use $v = -v$ (backward motion).

Remark. The transformation is undefined at $x = y = 0$ (the goal itself). For $\alpha \in I_1$ the forward direction points toward the goal; for $\alpha \in I_2$ the backward direction does. One can always initialize so that $\alpha \in I_1$, but the control law must ensure it stays there.

Control Law

Kinematic Feedback Control Law

$$v = k_\rho \rho, \quad \omega = k_\alpha \alpha + k_\beta \beta$$

This drives the robot to the equilibrium $(\rho, \alpha, \beta) = (0, 0, 0)$. Stability conditions for $\alpha \in I_1$:

- *Local exponential stability:* $k_\rho > 0, k_\beta < 0, k_\alpha - k_\rho > 0$
- *Strong stability* (no direction reversal): additionally $k_\alpha + \frac{5}{3}k_\beta - \frac{2}{\pi}k_\rho > 0$

This drives the robot to $(\rho, \alpha, \beta) = (0, 0, 0)$. The closed-loop dynamics for $\alpha \in I_1$:

$$\begin{bmatrix} \dot{\rho} \\ \dot{\alpha} \\ \dot{\beta} \end{bmatrix} = \begin{bmatrix} -k_\rho \rho \cos \alpha \\ k_\rho \sin \alpha - k_\alpha \alpha - k_\beta \beta \\ -k_\rho \sin \alpha \end{bmatrix}$$

Stability conditions:

- *Local exponential stability:* $k_\rho > 0, k_\beta < 0, k_\alpha - k_\rho > 0$
- *Strong stability* (no direction reversal during approach): additionally $k_\alpha + \frac{5}{3}k_\beta - \frac{2}{\pi}k_\rho > 0$, which guarantees $\alpha \in I_1$ for all t if $\alpha(0) \in I_1$.

The control signal v always has constant sign — the robot either drives forward or backward without ever inverting direction (“parking without reversing”).

Part III

Perception: Introduction and Sensors

12 Introduction to Perception

12.1 Perception in the Control Loop

Perception is the subsystem responsible for extracting meaningful information from raw sensor data and feeding it to higher-level reasoning. In the general control scheme, the **perception pipeline** comprises three stages:

1. **Sensing:** raw data acquisition from the physical environment.
2. **Information extraction and interpretation:** feature detection, object recognition, scene understanding.
3. **Localization and map building:** estimating robot pose and constructing an environment model.

The output of this pipeline (position estimate + local map) feeds the cognition block, closing the sense-act loop.

12.2 Perception as Information Compression

Perception can be viewed as a hierarchy of increasing abstraction, each level compressing information relative to the one below:

Level	Examples
Raw data	Vision, laser, sound
Features	Corners, lines, colors
Objects	Doors, pedestrians, bicycles
Places/Situations	Kitchen, car overtaking

Moving up the hierarchy requires increasingly powerful models and semantic reasoning: from geometric *models* for features, to *models/semantics* for objects, to *reasoning about relationships* for situations.

12.3 Challenges

Perception is extremely challenging in practice due to:

- **Noise:** all sensors are subject to measurement noise and quantization error.
- **Partial observation:** sensors have limited field of view, range, and resolution.
- **Context dependency:** interpreting raw data often requires situational context that is difficult to encode algorithmically.

Humans excel at perception precisely because of common-sense knowledge, something that remains difficult to replicate in artificial systems.

13 Sensor Classification

13.1 What They Measure

- **Proprioceptive sensors:** measure quantities *internal* to the robot — e.g., motor speed, battery status, joint angles, wheel odometry.
- **Exteroceptive sensors:** measure quantities from the *environment* — e.g., distance to obstacles, light intensity, object positions.

13.2 How They Measure

- **Passive sensors:** measure energy naturally present in the environment (e.g., cameras detecting ambient light). Their output is highly dependent on environmental conditions.
- **Active sensors:** emit energy and measure the reaction from the environment (e.g., lidar emitting laser pulses, ultrasonic sensors emitting sound). Generally higher performance, but may interfere with each other or the environment.

13.3 Sensor Classification Tables

The following tables categorize common sensors by function, whether they are proprioceptive (PC) or exteroceptive (EC), and whether they are active (A) or passive (P).

Category	Sensor	PC/EC	A/P
Tactile sensors (contact, proximity)	Contact switches, bumpers	EC	P
	Optical barriers	EC	A
	Noncontact proximity sensors	EC	A
Wheel/motor sensors (speed, position)	Brush encoders	PC	P
	Potentiometers	PC	P
	Synchros, resolvers	PC	A
	Optical encoders	PC	A
	Magnetic encoders	PC	A
	Inductive encoders	PC	A
	Capacitive encoders	PC	A
Heading sensors (orientation)	Compass	EC	P
	Gyroscopes	PC	P
	Inclinometers	EC	A/P
Ground-based beacons (localization)	GPS	EC	A
	Active optical or RF beacons	EC	A
	Active ultrasonic beacons	EC	A
	Reflective beacons	EC	A
Active ranging (range, geometry)	Reflectivity sensors	EC	A
	Ultrasonic sensor	EC	A
	Laser rangefinder	EC	A
	Optical triangulation (1D)	EC	A
	Structured light (2D)	EC	A
Motion/speed sensors	Doppler radar	EC	A
	Doppler sound	EC	A
Vision-based sensors	CCD/CMOS camera(s)	EC	P
	Visual ranging packages	EC	P
	Object tracking packages	EC	P

14 Individual Sensor Types

14.1 Encoders

An encoder is an electro-mechanical device that converts position (linear or angular) to an analogue or digital signal. In robotics, **optical incremental encoders** are the standard:

- A disk with alternating opaque and transparent segments is attached to the motor shaft.
- Two photo-detectors (channels A and B) are offset by a quarter period (**quadrature encoding**).
- The four-state sequence of $(A, B) \in \{(high, low), (high, high), (low, high), (low, low)\}$ encodes both displacement and direction.
- Resolution is typically specified in pulses per revolution (PPR); quadrature decoding multiplies effective resolution by 4.

Encoders are proprioceptive and passive — they impose no load on the environment and require no emitted energy.

14.2 Heading Sensors

Heading sensors determine the robot's **orientation** (yaw) and inclination (roll, pitch) with respect to a reference frame.

- **Compass** (exteroceptive, passive): senses the absolute direction of Earth's magnetic field. Gives yaw directly but is susceptible to ferromagnetic interference.
- **Gyroscope** (proprioceptive, passive): senses *relative* orientation change. When fused with velocity information, enables **dead reckoning** (deduced reckoning) — integrating angular rate over time to estimate heading.
- **Inclinometer**: measures static tilt relative to gravity via a damped pendulum or fluid.

Gyroscopes

Two main technologies:

- **Mechanical**: exploits the angular momentum of a rapidly spinning wheel. Force applied on one axis generates a reaction (precession) on the orthogonal axis. High accuracy but high cost and significant drift over time.
- **Optical (fiber-optic / ring laser)**: two laser beams travel in opposite directions around a closed optical path. The beam counter-propagating to the rotation direction travels a slightly shorter path (Sagnac effect), producing a measurable phase shift proportional to angular velocity. Lower drift than mechanical; no moving parts.

14.3 Accelerometers

An accelerometer measures all external forces acting on the robot, including gravity. The working principle is a **spring-mass-damper** system: a proof mass m attached to a spring k and damper c deflects under acceleration; the deflection is measured capacitively or piezoelectrically.

Modern accelerometers are fabricated as **MEMS** (Micro Electro-Mechanical Systems), enabling miniaturization and mass production (applications: airbags, game controllers, smartphones).

Remark. Gravity must be subtracted before integrating accelerometer data to obtain velocity and position. Without careful calibration, double integration accumulates large errors.

14.4 Inertial Measurement Units (IMU)

An IMU integrates gyroscopes and accelerometers (and optionally magnetometers) to jointly estimate:

- relative position (x, y, z)
- orientation: roll, pitch, yaw
- velocity and acceleration

all with respect to an inertial reference frame. The processing pipeline is:

$$\underbrace{\text{Rate gyroscope}}_{\dot{\theta}} \xrightarrow{\int} \text{Orientation} \quad \underbrace{\text{Accelerometer}}_{\mathbf{a}_{\text{body}}} \xrightarrow{R^{-1}} \mathbf{a}_{\text{nav}} \xrightarrow{-g} \mathbf{a}_{\text{linear}} \xrightarrow{\int} \mathbf{v} \xrightarrow{\int} \mathbf{p}$$

Key issues:

- Gravity must be compensated in the navigation frame.
- Initial velocity must be known.
- **Drift** is the dominant source of error: integrating small biases accumulates unbounded error over time. IMUs must be periodically re-calibrated or fused with absolute sensors (GPS, cameras).

14.5 Beacons and GPS

Beacons are artificial landmarks that provide point references to compute the position of a mobile platform.

Outdoor: GPS

The Global Positioning System (GPS) consists of a constellation of satellites broadcasting timing signals. A receiver triangulates its position using signals from at least 4 satellites (the fourth satellite is needed to solve for the unknown receiver clock offset).

- Standard civilian GPS accuracy: 5–20 meters.
- Differential GPS (DGPS): a fixed reference station with known position broadcasts correction signals → centimeter-level accuracy achievable.
- Critical limitation: **occlusions** (indoors, urban canyons, tunnels) break satellite visibility.

Indoor: RFID, WiFi, Bluetooth, UWB

Indoor localization systems are based on the same triangulation/trilateration principles as GPS but use different signal technologies:

- Most technologies are sensitive to multipath propagation and interference.
- **UWB** (Ultra-Wideband): short-pulse ToF ranging with centimeter-level precision; more robust to multipath than narrowband systems.

14.6 Range Sensors

Range sensors are key sensors for localization, mapping, and obstacle detection. They are popular in robotics due to their low cost and ease of interpretation. Two families:

Time-of-Flight (ToF)

Principle: emit a signal, measure the round-trip travel time t , compute distance as $d = c \cdot t$.

- **Sonars:** use sound ($c \approx 0.3 \text{ m/ms}$). At 3 m, the echo returns in $\approx 10 \text{ ms}$ — easy to measure. Wide beam opening angle is a limitation (poor angular resolution).
- **Lidars:** use light ($c \approx 0.3 \text{ m/ns}$). At 3 m, the round-trip takes $\approx 10 \text{ ns}$ — measuring nanosecond intervals requires dedicated electronics. High angular resolution and range accuracy. Used for 2D scan profiles or 3D point clouds.
- **Time-of-Flight cameras:** per-pixel ToF measurement using modulated infrared illumination, producing dense depth images.

ToF Accuracy Factors

Quality of ToF sensors depends on:

- **Inaccuracies in time measurement** (dominant for lidars)
- **Opening angle of the beam** (dominant for sonars — poor spatial resolution)
- **Target interactions:** reflectivity, opacity, surface angle (specular vs. diffuse reflection)

Speed of sound $\approx 0.3 \text{ m/ms}$; speed of light $\approx 0.3 \text{ m/ns}$. EM signals travel $10^6 \times$ faster than sound.

Geometric Active Ranging: Structured Light

Structured light sensors project a known light pattern (e.g., a line, grid, or coded texture using a laser or projector with a prism) onto the scene. A camera displaced from the emitter observes the projected pattern and detects deformations caused by scene geometry.

- The **correspondence problem** is simplified because the projected pattern is known a priori.
- Depth is recovered via **triangulation**: the displacement of observed pattern features relative to their expected positions encodes range.
- Common patterns: single stripe (1D), multiple stripes, coded patterns (binary or phase-shifted).
- Modern RGB-D cameras (e.g., Intel RealSense, original Microsoft Kinect) use structured infrared light.

Part IV

Perception: Vision Basics

15 Introduction to Vision

15.1 Human Vision

Vision is the most powerful sense for perceiving the three-dimensional world. The human retina occupies only $\approx 1000 \text{ mm}^2$ yet is extraordinarily dense:

- ≈ 120 million **rods** (brightness, peripheral, low-light vision)

- ≈ 7 million **cones** (color, high-acuity central vision)

In terms of spatial resolution, the human eye is equivalent to a digital camera with around **500 Megapixels**. The resulting data rate reaching the visual cortex is approximately **3 GB/s**. A large fraction of the human brain is devoted to processing this stream — an evolutionary investment that makes biological vision essentially effortless and extraordinarily robust.

For robots, this must be replicated algorithmically. **Computer vision** is the field concerned with extracting meaningful information from images and videos.

15.2 Challenges

Matching the robustness of biological vision is an open research problem. Three fundamental sources of variability make it hard:

- **Viewpoint changes:** the same 3D scene produces radically different images when viewed from different angles.
- **Illumination changes:** color and appearance of objects change dramatically under different lighting conditions.
- **Intra-class variations:** objects of the same semantic class (e.g., chairs) can look very different from one another.

These challenges motivate the use of learned representations (deep networks) rather than hand-crafted descriptors for high-level vision tasks.

16 Camera and Image Formation

16.1 Digital Cameras: CCD and CMOS

All digital cameras capture images using an array of **light-sensitive picture elements (pixels)**.

- **CCD (Charge Coupled Device):** each pixel is a discharging capacitor that accumulates charge proportional to the number of photons striking it during the exposure time. Charge is then read out sequentially. CCDs offer high image quality but are relatively expensive and power-hungry.
- **CMOS (Complementary Metal Oxide Semiconductor):** each pixel has a dedicated transistor circuit that measures and amplifies the signal in parallel. CMOS chips are simpler, cheaper, and consume less power than CCDs, at some cost in image quality. They are now the dominant technology in consumer devices and robotics.

Both sensors return a scalar per pixel (light intensity) — they do not directly capture color.

16.2 Color Cameras

To acquire color images, filters are used to separate wavelengths:

- **Single-chip Bayer filter:** a color filter array (typically 2×2 blocks of R, G, G, B) is placed over the pixel array. Each pixel measures only one color channel; the missing channels are interpolated (*demosaicing*). This is cheap but reduces effective spatial resolution.
- **Three-chip cameras:** a beam splitter directs each primary color to a dedicated sensor chip. All three channels are sampled at full resolution; the outputs are combined electronically. Higher quality and resolution than single-chip, but significantly more expensive.

16.3 Pinhole Camera Model

Placing a film directly in front of an object without any optics produces a hopelessly blurred image: each film point receives rays from the entire scene simultaneously. To form a useful image, the number of rays reaching each film point must be reduced.

The **pinhole camera model** is the simplest way to achieve this:

- All rays passing through a single point — the **center of projection** (or *optical center*) C — are captured.
- An **image plane** is placed at distance f (the *focal length*) from C ; the scene projects onto it, inverted.

This model is the foundation for almost all geometric reasoning in computer vision. It is simple, analytically tractable, and a good approximation for most cameras.

16.4 Why Lenses? The Thin Lens Equation

A perfect pinhole admits only one ray per image point, producing a perfectly sharp but very **dim** image (and diffraction effects become non-negligible). Increasing the aperture lets in more light but blurs the image.

A **lens** solves this: it focuses all rays from a scene point onto a single image point, allowing a large aperture without blurring. Key geometric properties of a thin lens:

- Rays passing through the **optical center** are not deviated.
- Rays parallel to the **optical axis** converge to the **focal point** at distance f behind the lens.

These two properties, combined with similar triangles, yield the **thin lens equation**:

Thin Lens Equation

$$\frac{1}{f} = \frac{1}{z} + \frac{1}{e}$$

where z is the distance of the object from the lens, e is the distance behind the lens at which a focused image is formed, and f is the focal length.

16.5 Pinhole Approximation

In practice, the object distance $z \gg f$, so from the thin lens equation:

$$\frac{1}{e} = \frac{1}{f} - \frac{1}{z} \approx \frac{1}{f} \implies e \approx f.$$

The image plane is placed at distance f from the lens (focused at infinity). Under this approximation, the lens behaves like a pinhole at distance f from the focal plane. The **pinhole approximation** then gives the relationship between real-world height h and its projection h' :

$$\frac{h'}{h} = \frac{f}{z} \implies h' = \frac{f}{z} h.$$

This is the basic scaling law of perspective imaging: objects appear smaller by a factor proportional to their distance.

17 Perspective Projection

17.1 From Scene Points to Image Plane

Assume the camera reference frame coincides with the world reference frame. A scene point is $P = (x_c, y_c, z_c)^\top$; its projection onto the image plane is $p = (x, y)$.

By similar triangles (the pinhole model with focal length f):

$$\frac{x}{x_c} = \frac{y}{y_c} = \frac{f}{z_c} \implies x = f \frac{x_c}{z_c}, \quad y = f \frac{y_c}{z_c}.$$

This is a **perspective projection**: depth z_c divides the world coordinates, introducing the depth-dependent scaling that makes distant objects appear smaller. Crucially, cameras measure **angles**, not distances.

Remark. The image plane is conventionally represented in front of the optical center C (rather than behind it) to avoid the inversion. This produces an image with the same orientation as the scene.

17.2 From Image Coordinates to Pixel Coordinates

The image plane coordinates (x, y) are in metric units. To obtain **pixel coordinates** (u, v) we need:

- An **origin offset**: the optical center C projects to the pixel (u_0, v_0) (the *principal point*).
- A **scale factor** k (pixels per meter) accounting for pixel size.

$$u = u_0 + kx = u_0 + k \frac{f x_c}{z_c}, \quad v = v_0 + ky = v_0 + k \frac{f y_c}{z_c}.$$

17.3 The Intrinsic Camera Matrix

The mapping from 3D camera-frame coordinates to pixel coordinates is linear in **homogeneous coordinates**. Define $\alpha = kf$ (focal length in pixels). Then:

Perspective Projection in Homogeneous Coordinates

$$\lambda \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \underbrace{\begin{bmatrix} \alpha & 0 & u_0 \\ 0 & \alpha & v_0 \\ 0 & 0 & 1 \end{bmatrix}}_K \begin{bmatrix} x_c \\ y_c \\ z_c \end{bmatrix}$$

where $\lambda = z_c$ is a scale factor and K is the **intrinsic camera matrix** (or *calibration matrix*).

The matrix K encodes all internal geometric properties of the camera: focal length α and principal point (u_0, v_0) . It does not depend on scene geometry.

17.4 General Case: Extrinsic Parameters

In general, the world and camera frames do not coincide. A scene point $P_w = (x_w, y_w, z_w)^\top$ in the world frame must first be mapped to the camera frame via a rigid-body transform:

$$\begin{bmatrix} x_c \\ y_c \\ z_c \end{bmatrix} = R \begin{bmatrix} x_w \\ y_w \\ z_w \end{bmatrix} + \mathbf{t}$$

where $R \in \text{SO}(3)$ is the camera rotation and $\mathbf{t} \in \mathbb{R}^3$ is the camera translation relative to the world. The full projection is then:

$$\lambda \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = K [R \mid \mathbf{t}] \begin{bmatrix} x_w \\ y_w \\ z_w \\ 1 \end{bmatrix} = M \tilde{P}_w$$

where $M = K[R \mid \mathbf{t}]$ is the 3×4 **camera projection matrix**. The parameters $(R, \mathbf{t})$ are called **extrinsic parameters** — they describe the camera pose in the world.

- **Intrinsic parameters:** K — internal geometry of the camera (α, u_0, v_0) .
- **Extrinsic parameters:** $(R, \mathbf{t})$ — position and orientation of the camera in the world.

18 Radial Distortion

Real lenses deviate from the ideal thin lens model, introducing geometric **distortions**. The most common is **radial distortion**:

- **Barrel distortion:** image magnification decreases with distance from the optical center — the image appears to bulge outward. Common in wide-angle lenses.
- **Pincushion distortion:** magnification increases with distance — the image appears to be pinched inward. Common in telephoto lenses.

The distortion maps ideal (undistorted) pixel coordinates (u, v) to observable (distorted) coordinates (u_d, v_d) as a nonlinear function of the radial distance from the optical center. For most lenses a **quadratic model** is sufficient:

Radial Distortion Model

$$\begin{bmatrix} u_d \\ v_d \end{bmatrix} = (1 + k_1 r^2) \begin{bmatrix} u - u_0 \\ v - v_0 \end{bmatrix} + \begin{bmatrix} u_0 \\ v_0 \end{bmatrix}$$

where the radial distance from the optical center is:

$$r^2 = (u - u_0)^2 + (v - v_0)^2$$

and k_1 is the **radial distortion coefficient** (negative for barrel, positive for pincushion distortion).

Higher-order terms $(k_2 r^4, \dots)$ can be added when stronger distortion correction is needed (e.g., fisheye lenses). Distortion correction must be applied *before* using the pinhole projection model.

19 Camera Calibration

19.1 Goal

Camera calibration is the process of accurately estimating the intrinsic parameters K (and the distortion coefficient k_1) and optionally the extrinsic parameters $(R, \mathbf{t})$ for a given camera. Without calibration, metric measurements and 3D reconstruction from images are not possible.

The approach: collect a set of n known **correspondences** $\{(P_i, p_i)\}$ — 3D world points with known coordinates and their observed pixel locations — then solve for the parameters by inverting the projection model.

19.2 Direct Linear Transform (DLT)

The projection model $\lambda p_i = M \tilde{P}_i$ involves the 3×4 matrix M . Since scale is irrelevant, M has 11 degrees of freedom (we can set $m_{34} = 1$). Each point correspondence gives two equations:

$$u_i = \frac{m_{11}x_w + m_{12}y_w + m_{13}z_w + m_{14}}{m_{31}x_w + m_{32}y_w + m_{33}z_w + m_{34}}, \quad v_i = \frac{m_{21}x_w + m_{22}y_w + m_{23}z_w + m_{24}}{m_{31}x_w + m_{32}y_w + m_{33}z_w + m_{34}}.$$

With ≥ 6 point correspondences these equations become a linear system in the m_{ij} , solvable via **linear least squares** (e.g., SVD). This is the **Direct Linear Transform (DLT)**.

Once M is estimated, the intrinsic and extrinsic parameters are recovered by decomposing $M = K[R \mid \mathbf{t}]$:

- The 3×3 submatrix $M_{1:3,1:3} = KR$ is decomposed via **QR factorization** into an upper triangular matrix K and an orthogonal matrix R .
- The translation is then $\mathbf{t} = K^{-1}[m_{14}, m_{24}, m_{34}]^\top$.

19.3 Zhang's Planar Grid Method

The DLT requires a 3D calibration object with known geometry, which is inconvenient in practice. Zhang (1998) proposed a far more practical approach using a **planar checkerboard pattern**:

- The world coordinate system is placed on the calibration plane, so $z_w = 0$ for all points. The projection simplifies to:

$$\lambda \begin{bmatrix} u_d \\ v_d \\ 1 \end{bmatrix} = K \begin{bmatrix} r_1 & r_2 & \mathbf{t} \end{bmatrix} \begin{bmatrix} x_w \\ y_w \\ 1 \end{bmatrix}$$

where r_1, r_2 are the first two columns of R .

- The user takes **several images** of the checkerboard at different positions and orientations. Checkerboard corners are easy to detect sub-pixel accurately.
- For each image a different homography between the pattern plane and the image plane is estimated. Stacking homographies from multiple views provides enough constraints to solve for K and the per-image extrinsics.
- Radial distortion coefficients are estimated jointly in a nonlinear refinement step.

Zhang's Method in Practice

- **Input:** several images of a planar checkerboard pattern at distinct poses; detected corner pixel coordinates.
- **Output:** K , k_1 (distortion), and $(R_i, \mathbf{t}_i)$ for each image.
- **Minimum:** at least 3 images of the pattern; in practice 10–20 give more robust results.
- **Implementations:** MATLAB Camera Calibration Toolbox, OpenCV `calibrateCamera`.

Zhang's method is by far the most widely used calibration technique in practice due to its simplicity — the calibration target is a printout — and its flexibility: no special 3D rig is needed.

Part V**Perception: Stereo Vision****20 Structure from Stereo: Motivation**

A single camera image cannot provide depth information. The projection from 3D to 2D is a many-to-one mapping — any point along the ray from the optical center to an image pixel is consistent with that pixel. Recovering depth requires additional constraints.

Two broad strategies exist:

- **Depth from camera geometry:** e.g., depth from focus — the amount of defocus of a point encodes its distance.
- **Depth from multiple viewpoints:** use two or more images taken from different positions to triangulate 3D structure.
 - **Structure from stereo:** two cameras at known relative positions observe the scene simultaneously.
 - **Structure from motion:** a single moving camera observes the scene at different times; the relative pose is unknown and must be estimated jointly with the structure.

Depth (range) information is indispensable for mobile robots: it drives mapping, localization, and obstacle avoidance.

20.1 Stereo Vision vs. Structure from Motion

	Stereo Vision	Structure from Motion
Images captured	simultaneously	at different times
Relative camera pose	known (from calibration)	unknown — must be estimated
What is estimated	3D structure	structure + motion (pose)

Both techniques recover 3D scene geometry from image correspondences; they differ in what is assumed known.

21 Stereo Vision: Working Principle

21.1 Triangulation

The core idea of stereo vision is to observe the same 3D point P from two cameras at known relative positions, then recover P as the **intersection of the two back-projected rays**. Each camera provides a ray from its optical center through the image point; their intersection in 3D is the scene point.

Triangulation is the problem of determining the 3D position of a point given:

- its image coordinates in two (or more) views,
- the intrinsic parameters and relative pose of the cameras.

Setting up the projection equations for left and right cameras:

$$\lambda_l \begin{bmatrix} u_l \\ v_l \\ 1 \end{bmatrix} = K_l \begin{bmatrix} I & \mathbf{0} \end{bmatrix} \begin{bmatrix} X_w \\ Y_w \\ Z_w \\ 1 \end{bmatrix}, \quad \lambda_r \begin{bmatrix} u_r \\ v_r \\ 1 \end{bmatrix} = K_r \begin{bmatrix} R & \mathbf{t} \end{bmatrix} \begin{bmatrix} X_w \\ Y_w \\ Z_w \\ 1 \end{bmatrix}$$

where K_l, K_r are the intrinsic matrices, $(R, \mathbf{t})$ is the relative pose of the right camera with respect to the left (known from calibration), and λ_l, λ_r are the unknown depth scales. Solving this system yields $P_w = (X_w, Y_w, Z_w)^\top$.

21.2 Simplified Case: Aligned Cameras

To build intuition, consider the idealized setup: two identical cameras with focal length f , optical axes parallel, image planes coplanar, separated by a horizontal **baseline** b . Let $P = (X, Y, Z)$ be a scene point. By similar triangles applied to each camera:

$$u_l = f \frac{X}{Z}, \quad u_r = f \frac{X - b}{Z}.$$

Subtracting gives the **disparity**:

$$d := u_l - u_r = f \frac{X}{Z} - f \frac{X - b}{Z} = \frac{f b}{Z}.$$

Solving for depth:

Depth from Disparity

$$Z = \frac{f \cdot b}{d}, \quad d = u_l - u_r$$

where f is the focal length (in pixels), b is the baseline (distance between the two optical centers), and d is the disparity (difference in horizontal image coordinates of the corresponding projections).

Two key observations:

- **Depth is inversely proportional to disparity**: closer objects produce larger disparity; distant objects produce small disparity, making them harder to distinguish.
- **Depth resolution scales with baseline**: a longer b amplifies disparity differences, improving depth resolution — but makes the correspondence problem harder (larger apparent shifts between views).

21.3 General Case

In practice the idealized aligned setup is unachievable:

- No two manufactured cameras are truly identical.
- Aligning both optical axes precisely on a common horizontal axis is mechanically very difficult.

The general case requires knowing the **relative pose** ($R, \mathbf{t}$), the **intrinsic parameters** (K_l, K_r), and the **distortion coefficients** of each camera — all obtained via **stereo calibration** (an extension of Zhang’s method applied jointly to a synchronized stereo pair). Once calibrated, the general triangulation equations can be solved directly.

22 Correspondence Search

Given two calibrated cameras, the practical challenge is: *which pixel in the right image corresponds to a given pixel in the left image?* This is the **correspondence problem**, and it must be solved before triangulation can proceed.

22.1 Area-Based Methods

For each pixel in the left image, define a small **patch** (window) centered on it and search for the most similar patch in the right image. Common similarity measures:

- **Normalized Cross-Correlation (NCC)**: measures linear correlation between two patches, normalized by their individual energies. Robust to affine illumination changes.
- **Sum of Squared Differences (SSD)**: $\sum_{(i,j) \in W} (I_l(i, j) - I_r(i + \Delta u, j + \Delta v))^2$. Simple and fast, but sensitive to illumination changes.
- **Sum of Absolute Differences (SAD)**: $\sum_{(i,j) \in W} |I_l(i, j) - I_r(i + \Delta u, j + \Delta v)|$. Similar to SSD but more robust to outlier pixels.

Limitation: exhaustive search over all candidate displacements is computationally expensive.

22.2 Feature-Based Methods

Rather than matching every pixel, extract **distinctive keypoints** in both images and match only those. Features should be repeatable (detectable under different viewpoints) and invariant to illumination, scale, and orientation changes. Common examples:

- **Geometric**: edges, corners, line segments.
- **Descriptor-based**: SIFT (Scale-Invariant Feature Transform), SURF (Speeded-Up Robust Features), MSER (Maximally Stable Extremal Regions).

Advantages over area-based methods: faster, more robust. **Drawback**: depth is recovered only at keypoints; a **dense depth map** requires interpolation between matches.

22.3 Epipolar Constraint

Exhaustive 2D search is unnecessary. The **epipolar constraint** reduces correspondence search from 2D to 1D.

Epipolar Geometry

Given a point p_l in the left image, the set of all 3D points consistent with p_l (varying depth) projects onto a **line** in the right image — the **epipolar line** of p_l . The correct correspondence p_r must lie on this line.

The plane containing both optical centers C_l, C_r and the 3D point P is the **epipolar plane**. Its intersections with the two image planes are the epipolar lines.

The **epipoles** e_l and e_r are the projections of each camera's optical center onto the other camera's image plane. All epipolar lines pass through their respective epipole.

By restricting search to epipolar lines, the problem reduces from a 2D search to a **1D search**, dramatically cutting computational cost and false match rate.

23 Epipolar Rectification

In the general (uncalibrated or non-aligned) setup, epipolar lines are oblique and differ per pixel. **Epipolar rectification** is a homographic re-sampling of both images such that:

- all epipolar lines become **horizontal** (parallel to the u -axis),
- corresponding epipolar lines share the **same row index** v in both images.

After rectification, correspondence search reduces to a **horizontal 1D scan** per row — highly efficient and amenable to hardware implementation.

23.1 Rectification Pipeline

1. **Remove radial distortion:** apply the inverse distortion model to both images using the calibrated coefficients, yielding undistorted images.
2. **Compensate for rotation and translation:** apply a homographic warp to each undistorted image, derived from the known $(R, \mathbf{t})$, so that the virtual image planes become coplanar and parallel to the baseline vector.

After step 2, any point in the left image and its correspondent in the right image have the same v -coordinate: $v_l = v_r$. Disparity $d = u_l - u_r$ is purely horizontal.

Remark. Rectification needs to be computed only once per stereo rig for a fixed calibration. At runtime, each stereo frame is warped using precomputed lookup tables (one per camera), making the per-frame cost negligible.

24 Disparity Maps

After rectification, correspondence search along rows produces a **disparity map**: an image in which each pixel (u, v) stores the disparity $d(u, v) = u_l - u_r$ of the corresponding scene point.

- **Brighter pixels** (large d) correspond to **closer** objects — large disparity means the projections are far apart horizontally, which by $Z = fb/d$ implies small depth.
- **Darker pixels** (small d) correspond to **farther** objects.
- **Invalid pixels:** regions visible in only one camera (occlusions) or textureless regions (where NCC/SSD/SAD are unreliable) are assigned an invalid depth label.

Every valid disparity pixel can be back-projected to a 3D point via $Z = fb/d$, $X = Zu_l/f$, $Y = Zv/f$, yielding a dense **point cloud** of the visible scene.

25 Structure from Motion

Structure from motion (SfM) generalizes stereo to the case where the relative camera pose is **unknown**. Given image correspondences across multiple views, SfM simultaneously estimates:

- **Structure**: the 3D positions $\{P_i\}$ of scene points.
- **Motion**: the camera poses $\{(R_j, \mathbf{t}_j)\}$ for each view.

The standard formulation minimizes the **reprojection error** — the sum of squared distances between observed image points and the re-projected 3D points:

$$\min_{\{P_i\}, \{R_j, \mathbf{t}_j\}} \sum_{i,j} \|\pi(K_j, R_j, \mathbf{t}_j, P_i) - p_{ij}\|^2$$

where $\pi(\cdot)$ is the perspective projection and p_{ij} is the observed image coordinate of point i in view j . This is a non-linear least-squares problem solved via **bundle adjustment** (typically Levenberg–Marquardt).

SfM is the backbone of large-scale 3D reconstruction pipelines (e.g., COLMAP) and is closely related to **visual odometry** and **visual SLAM**, where camera trajectory estimation must happen in real time.

Part VI

Perception: Line Extraction from Range Data

26 Introduction

Range sensors (lidars, sonars) produce sparse or dense **point clouds** representing the geometry of the surrounding environment. Raw point clouds are not directly useful for higher-level tasks such as localization or map matching; they must first be processed to extract compact **geometric features**.

The most fundamental and widely used feature from 2D range data is the **line segment**: walls, corridors, and table edges all appear as lines in a planar laser scan. Line features are used for:

- laser scan matching for localization,
- map building (line-based maps),
- detecting and navigating along structured environments.

Feature fitting is in general an over-determined problem: the number of data points exceeds the number of model parameters, and measurements are noisy. The standard approach is to cast it as a **least-squares optimization**: find the model parameters that minimize the sum of squared discrepancies between the data and the model.

Before considering specific algorithms, three sub-problems must be addressed:

1. **How many lines** are present in the data?
2. **Which points** belong to which line (data association)?

3. How to estimate the line parameters robustly?

27 Probabilistic Line Fitting

27.1 Line Representation in Polar Coordinates

A natural parameterization for lines in the context of range data uses **polar (normal) form**: a line is described by (r, α) , where r is the perpendicular distance from the origin to the line, and α is the angle of the normal to the line. This avoids the singularities of slope-intercept form for vertical lines.

Given n range measurements in polar coordinates (ρ_i, θ_i) (from the laser), the **signed distance** from point i to the line (r, α) is:

$$d_i = \rho_i \cos(\theta_i - \alpha) - r.$$

The line converts to Cartesian via $x \cos \alpha + y \sin \alpha = r$.

27.2 Least-Squares Estimation

The optimal line parameters (r^*, α^*) minimize the **sum of squared distances**:

$$S(r, \alpha) = \sum_{i=1}^n (\rho_i \cos(\theta_i - \alpha) - r)^2.$$

Setting the partial derivatives to zero yields a closed-form solution. From $\partial S / \partial r = 0$:

$$r = \frac{1}{n} \sum_{i=1}^n \rho_i \cos(\theta_i - \alpha).$$

Substituting into $\partial S / \partial \alpha = 0$ and applying trigonometric identities gives:

Closed-Form Line Fit (Polar Coordinates)

$$\alpha^* = \frac{1}{2} \arctan \left(\frac{-\sum_i \rho_i^2 \sin 2\theta_i + \frac{2}{n} \sum_i \rho_i \cos \theta_i \cdot \sum_i \rho_i \sin \theta_i}{\sum_i \rho_i^2 \cos 2\theta_i - \frac{1}{n} [(\sum_i \rho_i \cos \theta_i)^2 - (\sum_i \rho_i \sin \theta_i)^2]} \right)$$

$$r^* = \frac{1}{n} \sum_{i=1}^n \rho_i \cos(\theta_i - \alpha^*)$$

This gives the **maximum-likelihood line estimate** under the assumption that measurement errors are i.i.d. Gaussian. It is the statistical foundation used by all line-fitting algorithms below.

28 Line Extraction Algorithms

28.1 Split and Merge

The **split and merge** algorithm recursively subdivides a set of points until every subset is well fit by a single line, then merges collinear adjacent segments.

The key design choice is the threshold t_h : too small and noise causes unnecessary splits; too large and distinct lines get merged. The final merge step combines segments whose fitted lines are nearly collinear (small angle and distance between endpoints).

Algorithm 1 Split and Merge

Input: Set of points S , distance threshold t_h
Output: Set of line segments
 Push S onto list L
while L is not empty **do**
 $S_i \leftarrow \text{pop}(L)$
 $\ell \leftarrow \text{fitLine}(S_i)$
 $d_{\max} \leftarrow \max_{p \in S_i} \text{dist}(p, \ell)$
 if $d_{\max} > t_h$ **then**
 Split S_i at the farthest point into S_{i1}, S_{i2}
 Push S_{i1} and S_{i2} onto L
 end if
end while
 Merge collinear adjacent segments

Complexity: $O(N \log N)$ on average (balanced splits). In practice it is very fast — around 1500 Hz on typical laser scan sizes.

28.2 Line Regression

Line regression is a sliding-window approach: a window of N_f consecutive points is moved across the ordered point set, fitting a line to each window. Adjacent fitted lines are then merged if collinear.

Algorithm 2 Line Regression

Input: Ordered set of points S , window size N_f
Output: Set of line segments
 Initialize sliding window of N_f points
for all windows W of N_f consecutive points **do**
 Fit a line to points in W (least squares)
end for
 Merge collinear adjacent segments

Complexity: $O(N \cdot N_f)$. Slower than split and merge (~ 400 Hz) due to redundant overlapping fits, but straightforward to implement.

28.3 RANSAC

RANSAC (RANdom SAMple Consensus) is a general-purpose robust fitting algorithm designed to work well in the presence of **outliers** — data points that do not belong to any line. It is iterative and non-deterministic: with more iterations, the probability of finding a good solution increases.

Choosing the Number of Iterations k

The number of iterations needed depends on the fraction of inliers in the data. Let w be the (estimated) fraction of inliers. Under the assumption that the two sampled points are drawn independently:

$$P(\text{both sampled points are inliers}) = w^2, \quad P(\text{at least one outlier}) = 1 - w^2.$$

The probability that *all* k iterations sample at least one outlier (i.e., RANSAC never finds a

Algorithm 3 RANSAC for Line Extraction**Input:** Set of points S , number of iterations k , threshold t_h **Output:** Best line and its inlier set**for** $i = 1$ to k **do** Randomly sample 2 points (p_1, p_2) from S Fit line ℓ through (p_1, p_2) $\text{InlierSet}_i \leftarrow \{p \in S : \text{dist}(p, \ell) < t_h\}$ **end for****return** line with largest $|\text{InlierSet}|$, refit to all inliers

clean pair) is $(1 - w^2)^k$. Setting this equal to $1 - p$, where p is the desired probability of finding at least one outlier-free sample:

RANSAC: Required Number of Iterations

$$k = \frac{\log(1 - p)}{\log(1 - w^2)}$$

where p is the desired success probability and w is the estimated fraction of inliers.

Example 28.1. With $p = 0.99$ (99% success) and $w = 0.5$ (50% inliers): $k = \log(0.01)/\log(0.75) \approx 16$ iterations suffice.

To extract **multiple lines**, RANSAC is re-run iteratively: after each run, the inliers of the found line are removed from S and the algorithm repeats on the remaining points.

Drawbacks: no optimality guarantee on the number of inliers or on the fit quality within the inlier set. Results differ between runs (non-deterministic). **Complexity:** $O(S \cdot N \cdot k)$ where S is the number of lines — significantly slower (~ 30 Hz) than split and merge or regression.

28.4 Hough Transform

The **Hough transform** takes a fundamentally different approach: it maps the problem from *data space* (where we seek a line through points) to a *parameter space* (where points vote for line parameters), and finds the line as the most-voted parameter cell.

Duality between Points and Lines

In the slope-intercept parameterization $y = mx + b$:

- A **line** $y = m_0x + b_0$ in data space $\leftrightarrow$ a **point** (m_0, b_0) in parameter space.
- A **point** (x_0, y_0) in data space $\leftrightarrow$ a **line** $b = -x_0m + y_0$ in parameter space.
- Two collinear points in data space $\leftrightarrow$ two lines in parameter space that **intersect** at the parameters (m^*, b^*) of the line through them.

In general, n collinear points produce n lines in parameter space all passing through the same point (m^*, b^*) — this intersection is detected by voting.

Polar Parameterization

The slope-intercept form has two problems: m is unbounded (vertical lines have $m \rightarrow \infty$), and b is also unbounded. The standard fix is the **polar (Hesse normal) form**:

$$x \cos \theta + y \sin \theta = \rho, \quad 0 \leq \theta < \pi, \rho \in \mathbb{R}.$$

Under this parameterization, a point (x, y) in data space maps to a **sinusoid** $\rho = x \cos \theta + y \sin \theta$ in the (θ, ρ) parameter space (bounded since $\rho \leq \sqrt{x^2 + y^2}$ for fixed (x, y)).

Algorithm 4 Hough Transform

Input: Points S , accumulator dimensions $N_r \times N_c$, threshold t_h

Output: Detected line(s)

Initialize accumulator $H[N_r][N_c] \leftarrow 0$

for all points $p = (x, y) \in S$ **do**

for all discrete values θ **do**

$\rho \leftarrow x \cos \theta + y \sin \theta$

$H(\theta, \rho) \leftarrow H(\theta, \rho) + 1$

end for

end for

$(\theta^*, \rho^*) \leftarrow \arg \max_{\theta, \rho} H(\theta, \rho)$

if $H(\theta^*, \rho^*) > t_h$ **then**

 Inliers $\leftarrow$ all points that voted for (θ^*, ρ^*)

 Refit line to inliers; **return** line

end if

Complexity: $O(S \cdot N \cdot N_c + N_r \cdot N_c)$, where N_c is the number of θ -bins. In practice the Hough transform is the slowest of the algorithms considered (~ 10 Hz) due to the inner loop over all θ values per point.

29 Algorithm Comparison

The following table summarizes the empirical performance of the four algorithms on typical laser scan data ($N = 722$ points, $S = 7$ extracted segments on average), from Nguyen et al. (IROS 2005):

Algorithm	Complexity	Speed (Hz)	False Positives	Precision
Split and Merge	$O(N \log N)$	~ 1500	10%	+++
Line Regression	$O(N \cdot N_f)$	~ 400	10%	+++
RANSAC	$O(S \cdot N \cdot k)$	~ 30	30%	++++
Hough Transform	$O(S \cdot N \cdot N_c + N_r N_c)$	~ 10	30%	++++

- **Split and Merge** and **Line Regression** are fast and have low false positive rates, but are sensitive to the ordering of points and to outliers in the data.
- **RANSAC** and **Hough Transform** are inherently robust to outliers (they find the *best-supported* model regardless of noise), achieving higher precision at the cost of speed. RANSAC is more widely used in practice due to its simplicity and easy extension to other geometric primitives (planes, circles, etc.).

Part VII

Localization: Recursive State Estimation

30 Introduction to Localization

30.1 The Localization Problem

Localization is the task of estimating the robot's pose (position and orientation) in the environment based on sensor observations and a prior map. It sits at the core of the perception pipeline: without a reliable pose estimate, planning and map-based reasoning are impossible.

Two related problems are distinguished:

- **Localization:** the map is given; only the pose must be estimated.
- **Simultaneous Localization and Mapping (SLAM):** the map and the pose are both unknown and must be estimated jointly.

30.2 Why Probabilistic?

Two fundamental sources of uncertainty prevent a deterministic solution:

- **Noisy sensors:** both proprioceptive measurements (wheel odometry, IMU) and exteroceptive measurements (lidar, camera) are corrupted by noise. A single range reading does not give a precise distance — it gives a *distribution* over distances.
- **Imperfect maps:** the stored environment model differs from the real world (dynamic objects, construction, measurement errors at build time).

A deterministic approach would produce a single point estimate $x = 0.5\text{m}$ that ignores all uncertainty. A **probabilistic approach** instead maintains a full probability distribution over poses — e.g., $X \sim \mathcal{N}(0.5, 1)$ — capturing both the best estimate and the confidence in it. This distinction is critical: a robot that knows it is uncertain can act cautiously or seek additional observations.

31 Probability Background

31.1 Discrete and Continuous Random Variables

Let X be a random variable and x a realization.

- **Discrete:** $x \in \{x_1, \dots, x_n\}$. The probability mass function gives $P(X = x_i) \geq 0$ with $\sum_i P(X = x_i) = 1$.
- **Continuous:** X takes values on $\mathbb{R}$ (or a subset). The probability density function $p(x) \geq 0$ satisfies $\int p(x) dx = 1$, and $P(X \in [a, b]) = \int_a^b p(x) dx$.

31.2 Joint, Conditional, and Marginal Probability

For two random variables X and Y :

- **Joint:** $P(x, y) = P(X = x \text{ and } Y = y)$.
- **Conditional:** $P(x | y) = P(x, y)/P(y)$, equivalently $P(x, y) = P(x | y)P(y)$.
- **Independence:** $P(x, y) = P(x)P(y)$, equivalently $P(x | y) = P(x)$.

- **Marginal:** $p(x) = \int P(x, y) dy$ (continuous), or $P(x) = \sum_y P(x, y)$ (discrete).
- **Total probability:** $p(x) = \int P(x | y)P(y) dy$.

31.3 Bayes' Rule

From the product rule $P(x, y) = P(x | y)P(y) = P(y | x)P(x)$:

Bayes' Rule

$$P(x | y) = \frac{P(y | x) P(x)}{P(y)} = \eta P(y | x) P(x)$$

where the normalizing constant $\eta = 1/P(y) = 1/\sum_x P(y | x)P(x)$ ensures the result is a valid distribution.

The terms have a natural interpretation: $P(x)$ is the **prior** (belief before observing y), $P(y | x)$ is the **likelihood** (probability of the observation given the state), and $P(x | y)$ is the **posterior** (updated belief after observing y).

Incorporating previous knowledge: when a third variable z is already known,

$$P(x | y, z) = \eta P(y | x, z) P(x | z).$$

31.4 Conditional Independence

X is conditionally independent of Y given Z if:

$$P(x, y | z) = P(x | z) P(y | z) \iff P(x | z, y) = P(x | z).$$

Knowing z renders knowledge of y irrelevant for inferring x . This property is central to the Markov assumption used in the Bayes filter.

32 The Bayes Filter

32.1 Setup and Belief

The robot interacts with the world through a sequence of **controls** $u_1, u_2, \dots$ and **observations** $z_1, z_2, \dots$. At time t , the history is $h_t = u_1, z_1, \dots, u_t, z_t$.

The **belief** $\text{Bel}(x_t)$ is the posterior distribution over the robot's state x_t (here: pose) given all past controls and observations:

$$\text{Bel}(x_t) = P(x_t | u_{1:t}, z_{1:t}).$$

Three models are required:

- **Sensor model** $P(z_t | x_t)$: probability of receiving observation z_t when in state x_t .
- **Motion model** $P(x_t | u_t, x_{t-1})$: probability of transitioning to state x_t from x_{t-1} under control u_t .
- **Prior** $P(x_0)$: initial belief over the state.

32.2 The Markov Assumption

Computing $\text{Bel}(x_t)$ directly from the full history is intractable. The **Markov assumption** makes it manageable by asserting that the current state x_t contains all relevant information about the past:

Markov Assumption

$$\begin{aligned} P(z_t | x_t, z_{t-1}, u_{1:t}) &= P(z_t | x_t) \\ P(x_t | x_{t-1}, z_{t-1}, u_{1:t}) &= P(x_t | x_{t-1}, u_t) \end{aligned}$$

The current observation depends only on the current state (not on history). The current state depends only on the previous state and the current control.

This defines a **Dynamic Bayesian Network** (DBN): states $x_0 \rightarrow x_1 \rightarrow x_2 \rightarrow \dots$ form a Markov chain driven by controls u_t , with observations z_t depending only on x_t .

32.3 Derivation of the Recursive Update

Starting from the definition and applying Bayes' rule, total probability, and the Markov assumption:

$$\begin{aligned} \text{Bel}(x_t) &= P(x_t | u_{1:t}, z_{1:t}) \\ &= \eta P(z_t | x_t, u_{1:t}, z_{t-1}) P(x_t | u_{1:t}, z_{t-1}) && [\text{Bayes}] \\ &= \eta P(z_t | x_t) P(x_t | u_{1:t}, z_{t-1}) && [\text{Markov (1)}] \\ &= \eta P(z_t | x_t) \int P(x_t | u_{1:t}, z_{t-1}, x_{t-1}) P(x_{t-1} | u_{1:t}, z_{t-1}) dx_{t-1} && [\text{Total prob.}] \\ &= \eta P(z_t | x_t) \int P(x_t | u_t, x_{t-1}) P(x_{t-1} | u_{1:t-1}, z_{t-1}) dx_{t-1} && [\text{Markov (2)}] \\ &= \eta P(z_t | x_t) \int P(x_t | u_t, x_{t-1}) \text{Bel}(x_{t-1}) dx_{t-1} \end{aligned}$$

The last step recognizes $P(x_{t-1} | u_{1:t-1}, z_{t-1}) = \text{Bel}(x_{t-1})$, making the update **recursive**: the new belief depends only on the previous belief, the new control, and the new observation — not on the full history.

32.4 Prediction and Correction

The full update is split into two steps that have clear physical interpretations:

Bayes Filter: Predict-Correct

Prediction step (propagate belief through the motion model):

$$\overline{\text{Bel}}(x_t) = \int P(x_t | u_t, x_{t-1}) \text{Bel}(x_{t-1}) dx_{t-1}$$

Correction step (incorporate the new observation):

$$\text{Bel}(x_t) = \eta P(z_t | x_t) \overline{\text{Bel}}(x_t)$$

- **Prediction:** applying a control u_t moves the robot and increases uncertainty. The predicted belief $\overline{\text{Bel}}(x_t)$ is a convolution of the previous belief with the motion model — it spreads out.
- **Correction:** observing z_t sharpens the belief by multiplying pointwise by the sensor likelihood $P(z_t | x_t)$ and renormalizing.

32.5 Worked Example: Door State Estimation

A robot faces a door. State: *open* or *closed*.

Models:

$$\begin{aligned} P(\text{light}=\text{high} \mid \text{open}) &= 0.6, & P(\text{light}=\text{high} \mid \text{closed}) &= 0.3 \\ P(\text{range}=\text{near} \mid \text{open}) &= 0.5, & P(\text{range}=\text{near} \mid \text{closed}) &= 0.6 \end{aligned}$$

Prior: $P(\text{open}) = P(\text{closed}) = 0.5$.

After first observation (light = high):

$$P(\text{open} \mid \text{light}=\text{high}) = \eta \cdot 0.6 \cdot 0.5 = \eta \cdot 0.30$$

$$P(\text{closed} \mid \text{light}=\text{high}) = \eta \cdot 0.3 \cdot 0.5 = \eta \cdot 0.15$$

Normalizing: $\eta = 1/(0.30 + 0.15) = 1/0.45$, so $P(\text{open} \mid \text{light}=\text{high}) \approx 0.67$.

After second observation (range = near), using the posterior from step 1 as the new prior:

$$P(\text{open}) \leftarrow 0.67, \quad P(\text{closed}) \leftarrow 0.33$$

$$P(\text{open} \mid \text{light}=\text{high}, \text{range}=\text{near}) = \eta \cdot 0.5 \cdot 0.67 \approx 0.50$$

$$P(\text{closed} \mid \text{light}=\text{high}, \text{range}=\text{near}) = \eta \cdot 0.6 \cdot 0.33 \approx 0.50$$

The second observation provides conflicting evidence (a range reading of *near* is more consistent with *closed*), pulling the posterior back toward 50/50.

Incorporating an action (robot closes the door):

Transition model: $P(\text{open} \mid \text{close}, \text{open}) = 0.1$, $P(\text{closed} \mid \text{close}, \text{open}) = 0.9$, $P(\text{open} \mid \text{close}, \text{closed}) = 0$, $P(\text{closed} \mid \text{close}, \text{closed}) = 1$.

With prior $P(\text{open}) = 5/8$, $P(\text{closed}) = 3/8$:

$$\begin{aligned} P(\text{closed after close}) &= P(\text{closed} \mid \text{close}, \text{open}) \cdot \frac{5}{8} + P(\text{closed} \mid \text{close}, \text{closed}) \cdot \frac{3}{8} \\ &= 0.9 \cdot \frac{5}{8} + 1.0 \cdot \frac{3}{8} = \frac{4.5+3}{8} = \frac{7.5}{8} = 0.9375. \end{aligned}$$

The action greatly increases the probability that the door is closed, as expected.

33 Realizations of the Bayes Filter

The Bayes filter is a **general framework**. Practical implementations differ in how they represent and update the belief distribution:

Filter	Belief Representation	Model Assumptions
Kalman Filter (KF)	Gaussian (mean + covariance)	Linear motion and sensor models
Extended KF (EKF)	Gaussian	Linearized nonlinear models
Unscented KF (UKF)	Gaussian	Nonlinear models (sigma points)
Particle Filter (PF)	Weighted sample set	Arbitrary nonlinear/non-Gaussian
Histogram Filter	Discrete grid	Arbitrary (discretized state space)

- **Kalman-family filters:** represent the belief as a Gaussian $\mathcal{N}(\mu_t, \Sigma_t)$. The prediction and correction steps have closed-form solutions under linear (or linearized) models. Very efficient but limited to unimodal distributions — they cannot represent global uncertainty or multiple hypotheses simultaneously.
- **Particle filters:** represent the belief as a set of N weighted samples (particles) $\{x_t^{[i]}, w_t^{[i]}\}$. They make no parametric assumption and can approximate arbitrary distributions, including multi-modal ones. The correction step reweights samples by the sensor likelihood; a **resampling** step then draws new particles proportional to the weights, concentrating them in high-probability regions. Particle filters are the basis of **Monte Carlo Localization (MCL)**, which demonstrated real-time localization in large environments (e.g., the Smithsonian Museum) without requiring linearization.

Part VIII

Localization: Kalman Filter

34 The Kalman Filter

34.1 Overview

The **Kalman filter** (KF) is a concrete realization of the Bayes filter for the case where:

- the belief is represented as a **Gaussian distribution**,
- motion and observation models are **linear** with **Gaussian noise**.

Under these assumptions the KF is the **optimal** estimator — it minimizes the mean squared error and the belief remains exactly Gaussian at every step. The key insight is that Gaussians are closed under linear transformations, products, and marginalization, so every operation in the predict–correct cycle produces another Gaussian.

34.2 The Gaussian Belief

The state belief is a multivariate Gaussian:

$$\text{Bel}(x_t) = \mathcal{N}(x_t; \mu_t, \Sigma_t) = \det(2\pi\Sigma_t)^{-1/2} \exp\left(-\frac{1}{2}(x_t - \mu_t)^\top \Sigma_t^{-1}(x_t - \mu_t)\right)$$

where $\mu_t \in \mathbb{R}^n$ is the **mean** (best state estimate) and $\Sigma_t \in \mathbb{R}^{n \times n}$ is the **covariance matrix** (encoding uncertainty). Representing and updating the belief reduces to tracking (μ_t, Σ_t) .

34.3 Linear Motion and Observation Models

The Kalman filter assumes the following parametric structure:

KF System Models

Motion model:

$$x_t = A_t x_{t-1} + B_t u_t + \epsilon_t, \quad \epsilon_t \sim \mathcal{N}(0, R_t)$$

Observation model:

$$z_t = C_t x_t + \delta_t, \quad \delta_t \sim \mathcal{N}(0, Q_t)$$

The matrices have the following roles:

- A_t ($n \times n$): state transition matrix — describes how the state evolves from x_{t-1} to x_t in the absence of controls.
- B_t ($n \times l$): control-input matrix — maps the control $u_t \in \mathbb{R}^l$ to a state increment.
- C_t ($k \times n$): observation matrix — maps the state to the expected observation $z_t \in \mathbb{R}^k$.
- R_t ($n \times n$): process noise covariance — uncertainty injected by the motion.
- Q_t ($k \times k$): measurement noise covariance — uncertainty in the sensor.

Under these definitions the models become Gaussians:

$$P(x_t \mid u_t, x_{t-1}) = \mathcal{N}(x_t; A_t x_{t-1} + B_t u_t, R_t)$$

$$P(z_t \mid x_t) = \mathcal{N}(z_t; C_t x_t, Q_t)$$

34.4 Closure under Gaussian Belief

Because the product of two Gaussians is Gaussian, and because a linear transformation of a Gaussian is Gaussian, it follows by induction that:

- if $\text{Bel}(x_{t-1}) = \mathcal{N}(\mu_{t-1}, \Sigma_{t-1})$, then after the prediction step $\overline{\text{Bel}}(x_t)$ is Gaussian,
- after the correction step $\text{Bel}(x_t)$ is Gaussian (product of two Gaussians, renormalized).

The belief stays Gaussian for all t if initialized as Gaussian.

34.5 The Kalman Filter Algorithm

The predict–correct cycle computes the new Gaussian parameters (μ_t, Σ_t) in closed form:

Kalman Filter Algorithm

Input: $\mu_{t-1}, \Sigma_{t-1}, u_t, z_t$

Prediction:

$$\begin{aligned}\bar{\mu}_t &= A_t \mu_{t-1} + B_t u_t \\ \bar{\Sigma}_t &= A_t \Sigma_{t-1} A_t^\top + R_t\end{aligned}$$

Kalman gain:

$$K_t = \bar{\Sigma}_t C_t^\top (C_t \bar{\Sigma}_t C_t^\top + Q_t)^{-1}$$

Correction:

$$\begin{aligned}\mu_t &= \bar{\mu}_t + K_t (z_t - C_t \bar{\mu}_t) \\ \Sigma_t &= (I - K_t C_t) \bar{\Sigma}_t\end{aligned}$$

Output: μ_t, Σ_t

34.6 Interpreting Each Step

1. **Predict mean** $\bar{\mu}_t = A_t \mu_{t-1} + B_t u_t$: propagate the previous estimate through the linear motion model.
2. **Predict covariance** $\bar{\Sigma}_t = A_t \Sigma_{t-1} A_t^\top + R_t$: transform the previous uncertainty through the dynamics and add process noise. Uncertainty *grows* in the prediction step.

3. **Kalman gain** K_t : trades off between the predicted uncertainty $\bar{\Sigma}_t$ and the measurement noise Q_t . If measurements are very accurate ($Q_t \approx 0$) then $K_t \rightarrow C_t^{-1}$ and the state is set to the measurement. If motion is very accurate ($R_t \approx 0$, $\bar{\Sigma}_t \approx 0$) then $K_t \rightarrow 0$ and the measurement is ignored.
4. **Correct mean**: $z_t - C_t \bar{\mu}_t$ is the **innovation** — the discrepancy between the actual observation and the predicted observation. The correction moves the mean toward the measurement by an amount proportional to K_t .
5. **Correct covariance**: $(I - K_t C_t) \bar{\Sigma}_t$ reduces uncertainty. Uncertainty *shrinks* in the correction step.

35 Mobile Robot Localization with the KF

35.1 Markov Localization

Markov Localization is the application of the Bayes filter to robot pose estimation given a map m . The map enters the observation model: the sensor likelihood becomes $P(z_t | x_t, m)$, giving the probability of receiving observation z_t from state x_t given the known map.

The algorithm is the standard Bayes filter predict–correct cycle:

1. $\bar{\text{bel}}(x_t) = \int P(x_t | u_t, x_{t-1}) \text{bel}(x_{t-1}) dx_{t-1}$ (prediction)
2. $\text{bel}(x_t) = \eta P(z_t | x_t, m) \bar{\text{bel}}(x_t)$ (correction)

Markov Localization is a *framework* — the choice of belief representation and models gives different concrete algorithms.

35.2 KF Localization for Feature-Based Maps

When the map consists of identifiable **landmarks** (features with known positions), the KF provides an efficient and principled solution. The state is the robot pose $x_t = (x, y, \theta)^\top$. Observations are range and bearing measurements to nearby landmarks: $z_t^i = (r^i, \phi^i)^\top$.

Key requirement: landmarks must be **uniquely identifiable** — the data association problem (which observation corresponds to which landmark) must be solved. Under known correspondences, each landmark observation independently constrains the pose through the correction step.

The belief is a Gaussian $\text{Bel}(x_t) = \mathcal{N}(\mu_t, \Sigma_t)$ representing the estimated pose and its uncertainty. The covariance ellipse Σ_t shrinks with each observation and grows with each motion.

35.3 Localization Taxonomy

Localization problems differ along several axes:

- **Local vs. global**: local localization (position tracking) assumes a good initial estimate; global localization starts with a uniform prior over all poses. The “kidnapped robot” problem (robot is moved without warning) is the hardest global case.
- **Static vs. dynamic environments**: dynamic elements (people, moving objects) violate the static map assumption and require special handling.
- **Passive vs. active**: a passive robot localizes from observations collected incidentally; an active robot plans actions to reduce its uncertainty (information-theoretic planning).

- **Single vs. multi-robot:** multiple robots can cooperate by sharing observations or using each other as landmarks.

36 Extended Kalman Filter

36.1 Motivation

The standard KF requires linear models. In practice, robot kinematics and sensor geometry are nonlinear. For a differential-drive robot, the motion model involves $\cos \theta$ and $\sin \theta$; range-bearing observations involve arctan and square roots. The **Extended Kalman Filter** (EKF) extends the KF to nonlinear models via **local linearization**.

36.2 Nonlinear Models

The general case replaces the linear matrices with arbitrary differentiable functions:

$$\begin{aligned} x_t &= g(u_t, x_{t-1}) + \epsilon_t, & \epsilon_t &\sim \mathcal{N}(0, R_t) \\ z_t &= h(x_t) + \delta_t, & \delta_t &\sim \mathcal{N}(0, Q_t) \end{aligned}$$

where $g : \mathbb{R}^l \times \mathbb{R}^n \rightarrow \mathbb{R}^n$ is the (nonlinear) motion function and $h : \mathbb{R}^n \rightarrow \mathbb{R}^k$ is the (nonlinear) observation function.

36.3 First-Order Taylor Linearization

The EKF approximates g and h by their first-order Taylor expansions around the current mean estimate:

EKF Linearization

Motion linearization around μ_{t-1} :

$$g(u_t, x_{t-1}) \approx g(u_t, \mu_{t-1}) + G_t(x_{t-1} - \mu_{t-1}), \quad G_t = \frac{\partial g(u_t, \mu_{t-1})}{\partial x_{t-1}}$$

Observation linearization around $\bar{\mu}_t$:

$$h(x_t) \approx h(\bar{\mu}_t) + H_t(x_t - \bar{\mu}_t), \quad H_t = \frac{\partial h(\bar{\mu}_t)}{\partial x_t}$$

G_t and H_t are **Jacobian matrices** evaluated at the current mean. The Jacobian of a vector-valued function $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$ is the $m \times n$ matrix of partial derivatives:

$$F_x = \begin{pmatrix} \partial f_1 / \partial x_1 & \cdots & \partial f_1 / \partial x_n \\ \vdots & \ddots & \vdots \\ \partial f_m / \partial x_1 & \cdots & \partial f_m / \partial x_n \end{pmatrix}$$

36.4 EKF Algorithm

The EKF algorithm is structurally identical to the KF, with the linear matrices replaced by the nonlinear functions and their Jacobians:

Extended Kalman Filter Algorithm

Input: $\mu_{t-1}, \Sigma_{t-1}, u_t, z_t$

Prediction:

$$\begin{aligned}\bar{\mu}_t &= g(u_t, \mu_{t-1}) \\ \bar{\Sigma}_t &= G_t \Sigma_{t-1} G_t^\top + R_t\end{aligned}$$

Kalman gain:

$$K_t = \bar{\Sigma}_t H_t^\top (H_t \bar{\Sigma}_t H_t^\top + Q_t)^{-1}$$

Correction:

$$\begin{aligned}\mu_t &= \bar{\mu}_t + K_t(z_t - h(\bar{\mu}_t)) \\ \Sigma_t &= (I - K_t H_t) \bar{\Sigma}_t\end{aligned}$$

Output: μ_t, Σ_t

The key differences from the KF:

- Predict mean: use $g(u_t, \mu_{t-1})$ instead of $A_t \mu_{t-1} + B_t u_t$.
- Predict covariance: use G_t (Jacobian) instead of A_t .
- Correct mean: use $h(\bar{\mu}_t)$ instead of $C_t \bar{\mu}_t$ in the innovation.
- Correct covariance: use H_t (Jacobian) instead of C_t .

36.5 When Does EKF Work?

The EKF approximation is valid when:

- The functions g and h are **locally linear** around the mean — i.e., the true function is well approximated by its tangent at the current estimate.
- The **uncertainty is small** — a wide distribution magnifies linearization errors because the tails of the Gaussian explore regions where the linear approximation fails.

When these conditions are violated (high uncertainty, highly nonlinear functions), the EKF can diverge. Alternative approaches include the **Unscented Kalman Filter** (UKF), which propagates a set of carefully chosen *sigma points* through the nonlinear function without requiring an explicit Jacobian, and **particle filters**, which make no linearity or Gaussianity assumptions at all.

36.6 Extensions

- **Unknown correspondences:** when landmark identity is ambiguous, the **maximum-likelihood** correspondence estimate is used: each observation is assigned to the landmark that maximizes $P(z_t | x_t, m)$.
- **Multiple Hypothesis Tracking (MHT):** maintain a mixture of Gaussians, one per data-association hypothesis, to handle ambiguous correspondences without committing to a single assignment.
- **Information filter:** an algebraically equivalent reformulation of the KF using the *information matrix* $\Omega_t = \Sigma_t^{-1}$ and *information vector* $\xi_t = \Sigma_t^{-1} \mu_t$. Some operations (e.g.,

incorporating independent observations, sparsity exploitation) are more efficient in this parameterization.

Part IX

Localization: Particle Filters and Monte Carlo Localization

37 Non-Parametric Belief Representation

37.1 Motivation

The Kalman filter family requires the belief to be Gaussian — a unimodal, symmetric distribution fully described by a mean and covariance. This is inadequate for:

- **Global localization:** initially the robot has no idea where it is; the belief is uniform or multi-modal (many plausible poses).
- **Highly nonlinear models:** linearization errors can cause the Gaussian to drift away from the true posterior.
- **Non-Gaussian sensor noise:** real sensors (e.g., sonars with specular reflections, lidars with mixed pixels) produce non-Gaussian likelihoods.

Particle filters address all of these by representing the belief as a set of weighted samples — a non-parametric, fully general approximation that can represent *any* distribution.

37.2 Sample-Based Distribution Representation

A probability distribution $p(x)$ is approximated by a **particle set**:

$$\mathcal{X} = \left\{ \left(x^{[j]}, \omega^{[j]} \right) \right\}_{j=1}^M$$

where $x^{[j]}$ is the j -th **particle** (a concrete state hypothesis) and $\omega^{[j]} \geq 0$ is its **importance weight**, normalized so that $\sum_j \omega^{[j]} = 1$. The discrete approximation is:

$$p(x) \approx \sum_{j=1}^M \omega^{[j]} \delta_{x^{[j]}}(x)$$

where $\delta_{x^{[j]}}$ is a Dirac delta centered at $x^{[j]}$. As $M \rightarrow \infty$, the approximation converges to the true distribution under mild regularity conditions. The quality of the approximation is controlled by the number of particles M : more particles give a finer representation but increase computational cost.

38 Importance Sampling

38.1 The Core Idea

To update the belief recursively we need to draw samples from the new posterior, which in general we cannot sample from directly. **Importance sampling** solves this: we draw samples from a tractable **proposal distribution** $g(x)$ and correct for the difference using weights.

Importance Sampling

To approximate samples from a target distribution $f(x)$:

1. Draw $x^{[j]} \sim g(x)$ from the proposal.
2. Assign importance weight $\omega^{[j]} = f(x^{[j]})/g(x^{[j]})$.

Precondition: $f(x) > 0 \Rightarrow g(x) > 0$ — the proposal must cover the support of the target.

The weighted samples $\{(x^{[j]}, \omega^{[j]})\}$ then represent f despite having been drawn from g . The choice of proposal affects efficiency: g close to f yields lower variance weights and a better approximation with fewer samples.

39 The Particle Filter Algorithm

39.1 Predict, Weight, Resample

The particle filter implements recursive Bayesian estimation in three stages per timestep:

1. **Prediction (sampling):** propagate each particle through the motion model to obtain a new hypothesis.
2. **Correction (weighting):** weight each new particle by how well it explains the current observation.
3. **Resampling:** draw a new particle set from the weighted set, concentrating particles in high-probability regions.

Particle Filter Algorithm

Input: $\mathcal{X}_{t-1}, u_t, z_t$ (previous particles, control, observation)

Phase 1 — sample and weight:

```

 $\bar{\mathcal{X}}_t = \emptyset$ 
for  $m = 1$  to  $M$  do
  sample  $x_t^{[m]} \sim P(x_t \mid u_t, x_{t-1}^{[m]})$ 
   $\omega_t^{[m]} = P(z_t \mid x_t^{[m]})$ 
  add  $(x_t^{[m]}, \omega_t^{[m]})$  to  $\bar{\mathcal{X}}_t$ 
end for

```

Phase 2 — resample:

```

 $\mathcal{X}_t = \emptyset$ 
for  $m = 1$  to  $M$  do
  draw index  $j$  with probability  $\propto \omega_t^{[j]}$ 
  add  $x_t^{[j]}$  to  $\mathcal{X}_t$ 
end for
return  $\mathcal{X}_t$ 

```

The proposal is the **motion model** $P(x_t \mid u_t, x_{t-1}^{[m]})$ — easy to sample from. The importance weight is the **observation likelihood** $P(z_t \mid x_t^{[m]})$ — it measures how consistent the sampled state is with the observation.

39.2 Why This Works

The ratio of target to proposal is:

$$\omega^{[m]} = \frac{\text{Bel}(x_t^{[m]})}{P(x_t^{[m]} | u_t, x_{t-1}^{[m]})} \propto P(z_t | x_t^{[m]})$$

because $\text{Bel}(x_t) \propto P(z_t | x_t) \overline{\text{Bel}}(x_t)$ and $\overline{\text{Bel}}(x_t)$ is exactly the distribution we sampled from. The observation likelihood therefore serves directly as the importance weight.

40 Resampling

40.1 Purpose

After weighting, many particles have very small weights and contribute negligibly to the belief. Resampling eliminates them and duplicates high-weight particles, keeping the effective sample count high. Without resampling, the particle set degenerates over time: a few particles accumulate all the weight while the rest are wasted.

40.2 Roulette Wheel Sampling

The naive approach — a roulette wheel where each slot has area proportional to $\omega^{[j]}$ — selects particles independently via binary search. This is easy to implement but:

- **Complexity:** $O(M \log M)$ (binary search per sample).
- **High variance:** if all weights are equal the roulette wheel may nonetheless select some particles many times and miss others entirely, introducing unnecessary noise.

40.3 Low-Variance (Systematic) Resampling

The preferred method. Instead of M independent random draws, a single random offset $r \sim \mathcal{U}(0, M^{-1})$ is drawn and the M selection points are placed at:

$$r, \quad r + M^{-1}, \quad r + 2M^{-1}, \quad \dots, \quad r + (M - 1)M^{-1}.$$

Low-Variance Resampling Algorithm

```

 $r \leftarrow \mathcal{U}(0, M^{-1})$ 
 $c \leftarrow \omega_1^{[1]}; \quad j \leftarrow 1$ 
for  $m = 1$  to  $M$  do
   $u \leftarrow r + (m - 1) \cdot M^{-1}$ 
  while  $u > c$  do
     $j \leftarrow j + 1; \quad c \leftarrow c + \omega_t^{[j]}$ 
  end while
  add  $x_t^{[j]}$  to  $\mathcal{X}_t$ 
end for
return  $\mathcal{X}_t$ 

```

Properties:

- **Complexity:** $O(M)$ — the M selection points are processed in a single left-to-right pass over the cumulative weight array.

- **Low variance:** the evenly spaced selection points guarantee that if all weights are equal, every particle is selected exactly once — no spurious variance is introduced.
- **Recommendation:** always use low-variance resampling over roulette-wheel sampling.

41 Monte Carlo Localization

41.1 Algorithm

Monte Carlo Localization (MCL) is the particle filter applied to mobile robot pose estimation. Each particle $x_t^{[m]} = (x, y, \theta)^\top$ is a pose hypothesis. The posterior belief over poses is:

$$\text{bel}(x_t) \approx \sum_{m=1}^M \omega_t^{[m]} \delta_{x_t^{[m]}}(x_t).$$

The three models required are:

- **Motion model** $P(x_t \mid u_t, x_{t-1})$: samples a new pose given the previous pose and the executed control (e.g., odometry-based sampling).
- **Observation model** $P(z_t \mid x_t, m)$: scores a pose hypothesis by how well the sensor reading z_t matches what would be expected from that pose given map m .
- **Prior** $P(x_0)$: for global localization, initialized as uniform over the map; for tracking, initialized as a tight Gaussian around the known starting pose.

MCL is a direct specialization of the generic particle filter; the map m enters only in the observation model.

41.2 The MCL Cycle

One timestep of MCL proceeds as follows:

1. **Motion update:** for each particle $x_{t-1}^{[m]}$, sample a new pose $x_t^{[m]} \sim P(x_t \mid u_t, x_{t-1}^{[m]})$ from the motion model. This *spreads* the particle cloud in accordance with motion uncertainty.
2. **Observation update:** weight each particle by $\omega_t^{[m]} = P(z_t \mid x_t^{[m]}, m)$. Particles at poses consistent with the observation receive high weights; inconsistent ones receive low weights.
3. **Resampling:** draw M particles from the weighted set (using low-variance resampling). Particles with high weights survive and are duplicated; low-weight particles are discarded.

Over successive iterations the particle cloud *converges*: it collapses from a diffuse cloud covering the whole map to a tight cluster around the true pose as observations accumulate.

41.3 Global Localization

MCL handles the global localization problem naturally by initializing particles uniformly across all free space in the map. After a few observations and motion steps, the cloud rapidly concentrates around the consistent pose hypotheses. This is impossible for the KF/EKF, which require a unimodal Gaussian prior.

Remark. The kidnapped robot problem — where the robot is physically moved to an unknown location mid-mission — can be addressed by injecting a fraction of random particles at each step, ensuring the filter never fully commits to a single hypothesis.

42 Practical Considerations

42.1 Pros and Cons of Particle Filters

Advantages	Disadvantages
Handle non-Gaussian distributions directly	Problematic in high-dimensional state spaces
Handle data-association ambiguity naturally	Particle depletion under high uncertainty
Support multiple sensing modalities easily	Computationally expensive for large M
Easy to implement	Approximation quality depends on M
Robust to model mismatch	

42.2 Particle Depletion

When the observation likelihood is very peaked (informative sensor), nearly all particles receive near-zero weight after the correction step. The resampling step then draws only from the few surviving high-weight particles, catastrophically reducing diversity. Mitigation strategies include:

- using a large M ,
- adaptive particle counts (more particles when uncertainty is high),
- injecting random particles to maintain diversity.

42.3 Variants

- **Rao-Blackwellized Particle Filters:** analytically marginalize out part of the state (e.g., the map in SLAM) and maintain only a particle filter over the remaining lower-dimensional state (e.g., the trajectory). This reduces the effective dimensionality and makes the filter tractable for SLAM.
- **Real-time particle filters:** handle sensor streams arriving at different frame rates by decoupling the prediction and correction steps.
- **Delayed-state particle filters:** handle delays in sensor data by conditioning on the delayed observation at the time it arrives.

MCL is the **gold standard** for indoor mobile robot localization. It was used to localize the Minerva robot autonomously in the Smithsonian Museum and has been deployed in numerous real-world systems since.

Part X

Simultaneous Localization and Mapping

43 The SLAM Problem

43.1 Motivation

Localization (Parts VII to IX) assumes a known map. Mapping assumes a known trajectory. In practice, neither is available: a robot deployed in an unknown environment must build its map *and* localize within it simultaneously. This is the **Simultaneous Localization and Mapping (SLAM)** problem.

SLAM is harder than either task in isolation because the two sub-problems are coupled: the map depends on the estimated trajectory, and the trajectory estimate depends on the map. Errors in one propagate to the other.

43.2 Problem Formulations

Let $x_{0:T} = \{x_0, x_1, \dots, x_T\}$ be the robot trajectory, m the map, $z_{1:T}$ the sequence of observations, and $u_{1:T}$ the control inputs.

Full SLAM

Estimate the *entire* trajectory and the map jointly:

$$P(x_{0:T}, m \mid z_{1:T}, u_{1:T}).$$

This is a high-dimensional distribution over all past poses. It is usually solved offline (batch).

Online SLAM

Marginalize out past poses and track only the current pose and map:

$$P(x_t, m \mid z_{1:t}, u_{1:t}).$$

This is the incremental formulation suited to real-time systems.

43.3 Key Challenges

- **Correlation:** the robot's poses and landmark positions are correlated. Ignoring correlations (e.g., treating each observation independently) leads to overconfident, inconsistent estimates.
- **Data association:** deciding which measurement corresponds to which landmark is ambiguous, especially in repetitive environments. Wrong associations degrade the map catastrophically.
- **Scalability:** the state grows with the number of landmarks and poses; naive approaches are $O(n^2)$ in landmark count.
- **Loop closure:** when the robot revisits a location after a long traverse, accumulated drift must be corrected globally, not just locally.

Remark. SLAM was once considered one of the fundamental open problems in robotics. The key theoretical insight — that the joint distribution over all poses and landmarks can be maintained consistently — came from the late 1980s and 1990s (Smith, Self, Cheeseman; Dissanayake et al.). Scalable solutions emerged in the 2000s.

44 Grid-Based SLAM with Rao-Blackwellized Particle Filters

44.1 Factoring the Joint Distribution

An occupancy-grid map m is high-dimensional but, given a *known* trajectory $x_{0:t}$, each cell is conditionally independent of all others given the observations. This permits an exact factorization:

Rao-Blackwellization for SLAM

$$p(x_{0:t}, m \mid z_{1:t}, u_{1:t}) = \underbrace{p(x_{0:t} \mid z_{1:t}, u_{1:t})}_{\text{particle filter over trajectory}} \cdot \underbrace{p(m \mid x_{0:t}, z_{1:t})}_{\text{exact grid map per particle}}.$$

Each particle carries a complete hypothesis of the trajectory *and* its own individual map. The map for particle i is updated with standard occupancy-grid techniques conditioned on the particle's pose sequence.

44.2 Particle Filter over Trajectories

The trajectory distribution $p(x_{0:t} \mid z_{1:t}, u_{1:t})$ is maintained by a particle filter:

Algorithm 5 Rao-Blackwellized SLAM Particle Filter (one step)

Input: particle set $\mathcal{X}_{t-1} = \{(x_{0:t-1}^{[i]}, m_{t-1}^{[i]}, \omega^{[i]})\}_{i=1}^N$, control u_t , observation z_t
for $i = 1$ **to** N **do**
 Sample $x_t^{[i]} \sim p(x_t \mid x_{t-1}^{[i]}, u_t)$ (motion model proposal)
 Update map: $m_t^{[i]} = \text{GridUpdate}(m_{t-1}^{[i]}, x_t^{[i]}, z_t)$
 Compute weight: $\omega_t^{[i]} = p(z_t \mid x_t^{[i]}, m_t^{[i]})$
end for
 Normalize weights: $\omega^{[i]} \leftarrow \omega^{[i]} / \sum_j \omega^{[j]}$
 Resample N particles with replacement according to $\{\omega^{[i]}\}$
Return: $\mathcal{X}_t$

44.3 Improved Proposal via Scan Matching

The plain motion-model proposal is inefficient when the sensor is highly informative (narrow likelihood) relative to the motion noise (wide prior). Incorporating the most recent observation into the proposal dramatically reduces the number of particles needed.

Scan matching (e.g., ICP or correlation-based scan matching) is used to compute a refined pose estimate $\hat{x}_t^{[i]}$ by aligning the incoming laser scan against particle i 's map. The proposal is then a Gaussian centered on $\hat{x}_t^{[i]}$:

$$q(x_t \mid x_{0:t-1}^{[i]}, z_t, u_t) = \mathcal{N}(\hat{x}_t^{[i]}, \hat{\Sigma}_t^{[i]}).$$

Weights are adjusted accordingly to account for the change of proposal:

$$\omega_t^{[i]} \propto \frac{p(z_t \mid x_t^{[i]}, m_{t-1}^{[i]}) p(x_t^{[i]} \mid x_{t-1}^{[i]}, u_t)}{q(x_t^{[i]} \mid \dots)}.$$

The GMapping algorithm (Grisetti et al., 2007) implements this approach and remains a widely used open-source SLAM system.

45 Graph-Based SLAM

45.1 Pose Graph Representation

Graph-based SLAM separates the problem into two phases:

1. **Front-end:** process raw sensor data to extract relative pose constraints between robot poses.

2. **Back-end:** optimize a graph of poses to find the globally consistent trajectory.

Pose Graph

- **Nodes:** robot poses $x_i \in SE(2)$ (or $SE(3)$).
- **Edges:** a constraint between nodes i and j is a measured relative transformation Z_{ij} with information matrix $\Omega_{ij} \succ 0$. Sequential odometry edges encode motion; loop-closure edges encode sensor-derived matches between non-consecutive poses.

The key insight is that once the graph is built, the map can be recovered from the optimized trajectory without being part of the optimization itself (for feature-based maps, landmarks become nodes; for grid maps, the grid is reconstructed after optimization).

45.2 Front-End: ICP Scan Matching

The Iterative Closest Point (ICP) algorithm computes the rigid transformation T that best aligns two point clouds $\mathcal{A}$ and $\mathcal{B}$:

Algorithm 6 ICP

```

Initialize  $T \leftarrow T_0$  (e.g., from odometry)
repeat
  for each point  $a \in \mathcal{A}$  do
     $b^* \leftarrow \arg \min_{b \in \mathcal{B}} \|T a - b\|^2$  (nearest-neighbor matching)
  end for
   $T \leftarrow \arg \min_T \sum_k \|T a_k - b_k^*\|^2$  (closed-form SVD solution)
until convergence
Return:  $T$ , residual error

```

ICP converges to a local minimum, so a good initialization (from odometry or scan matching on a local map) is critical. The output relative pose and its uncertainty form an edge in the pose graph.

45.3 Back-End: Gauss-Newton Optimization

45.3.1 Error Function

For an edge (i, j) , the error measures the discrepancy between the observed relative transformation Z_{ij} and the one implied by the current pose estimates x_i, x_j :

Edge Error

$$e_{ij}(x_i, x_j) = \text{t2v}\left(Z_{ij}^{-1} (X_i^{-1} X_j)\right),$$

where $X_i \in SE(2)$ is the matrix form of pose x_i , and $\text{t2v}(\cdot)$ converts a homogeneous transformation to a vector $(\Delta x, \Delta y, \Delta \theta)^\top$.

When x_i and x_j are consistent with the measurement Z_{ij} , the argument of t2v is the identity and $e_{ij} = 0$.

45.3.2 Objective Function

The total cost is a sum of weighted squared errors over all edges $\mathcal{E}$:

SLAM Objective

$$x^* = \arg \min_x \sum_{(i,j) \in \mathcal{E}} e_{ij}(x_i, x_j)^\top \Omega_{ij} e_{ij}(x_i, x_j),$$

where $\Omega_{ij} = \Sigma_{ij}^{-1}$ is the information matrix (inverse covariance) of the constraint.

This is a nonlinear least-squares problem. The Gauss-Newton method solves it by iteratively linearizing the error functions.

45.3.3 Linearization

Around the current estimate $\bar{x}$, linearize each error:

$$e_{ij}(x + \Delta x) \approx e_{ij}(\bar{x}) + J_{ij} \Delta x,$$

where $J_{ij} = \frac{\partial e_{ij}}{\partial x} \big|_{\bar{x}}$ is the Jacobian of the error with respect to *all* poses (sparse: non-zero only in the columns corresponding to x_i and x_j).

Substituting into the cost and minimizing over Δx gives the **normal equations**:

Normal Equations (Gauss-Newton)

$$H \Delta x = -b,$$

$$H = \sum_{(i,j) \in \mathcal{E}} J_{ij}^\top \Omega_{ij} J_{ij},$$

$$b = \sum_{(i,j) \in \mathcal{E}} J_{ij}^\top \Omega_{ij} e_{ij}.$$

The Hessian H is a sparse, positive semi-definite matrix. Its sparsity pattern reflects the graph topology: $H_{ij} \neq 0$ only when nodes i and j share an edge. The system is solved with sparse Cholesky factorization; the poses are updated as $\bar{x} \leftarrow \bar{x} + \Delta x$ and the procedure is iterated until convergence.

45.4 Sparsity and Scalability

- Each edge contributes only to 4×4 (2D) or 6×6 (3D) diagonal blocks of H and the corresponding off-diagonal blocks.
- For a trajectory with n poses and $O(n)$ edges (odometry-only), H is block-tridiagonal and solvable in $O(n)$ time.
- Loop closures add fill-in but the matrix remains sparse for realistic graphs. Variable reordering (e.g., nested dissection) preserves sparsity.
- This sparsity is the fundamental reason graph-based SLAM scales to large environments with thousands of poses.

Remark. The iSAM (Kaess et al., 2008) and g^2o (Kümmerle et al., 2011) frameworks implement efficient incremental and batch graph-based SLAM, respectively. The approach extends naturally to 3D ($SE(3)$ poses) and is the backbone of modern LiDAR and visual SLAM systems.

46 SLAM in Practice

46.1 Comparison of Approaches

Approach	Map type	Strengths	Limitations
RBPF (GMapping)	Occupancy grid	Dense, intuitive	Particle degeneracy at scale
Graph-based	Feature/grid	Scalable, sparse solve	Requires good front-end
EKF-SLAM	Feature	Principled covariance	$O(n^2)$ cost in landmarks

46.2 Data Association

Data association — determining which observation corresponds to which previously seen landmark or scan — is the most failure-prone step in any SLAM system. Common strategies:

- **Nearest-neighbor (NN)**: match each measurement to the closest known landmark. Fast but fragile in ambiguous environments.
- **Joint compatibility branch and bound (JCBB)**: jointly validate assignment hypotheses using chi-squared gating. More robust but exponential in the worst case.
- **Scan-to-scan / scan-to-map matching**: ICP-based; avoids explicit landmark extraction by working directly with point clouds.
- **Descriptor matching**: use appearance descriptors (ORB, SIFT, etc.) for loop closure detection in visual SLAM.

46.3 Loop Closure

Loop closure is the event of revisiting a previously mapped location after a long traverse. It is the mechanism by which globally consistent maps are built:

1. Detect a potential loop closure (e.g., via place recognition, scan matching score, or appearance similarity).
2. Compute the relative pose constraint via ICP or descriptor-based alignment.
3. Add the constraint as a new edge in the pose graph.
4. Re-run back-end optimization to distribute the correction globally.

Without loop closure, odometric drift accumulates and the map becomes globally inconsistent even if locally accurate.

Part XI

Occupancy Grid Maps

47 Map Representations

47.1 Why Maps Matter

A map is a representation of the environment that a robot uses for virtually every high-level task: localization, obstacle avoidance, path planning, and activity planning. Choosing the right representation is a fundamental design decision.

Type	Representation	Key trade-off
Feature maps	Geometric primitives (corners, lines, landmarks)	Compact but requires feature extraction
Volumetric maps	Occupancy of discretized space (grids)	Complete but memory intensive

Occupancy grid maps are **volumetric**: they directly model which regions of space are occupied without relying on any intermediate feature-extraction step.

47.2 Mapping with and without Known Poses

Two problem settings arise depending on whether the robot trajectory is known:

Mapping with Known Poses

Given sensor data $z_{1:t}$ and a known trajectory $x_{1:t}$, recover the most likely map:

$$m^* = \arg \max_m P(m \mid x_{1:t}, z_{1:t}).$$

This is the setting studied in this part; it decouples mapping from localization.

Mapping without Known Poses (SLAM)

Without known poses, both the map and trajectory must be estimated jointly:

$$m^* = \arg \max_m P(m \mid u_{1:t}, z_{1:t}).$$

This is the full SLAM problem (Part X).

48 Occupancy Grid Maps

48.1 Definition and Assumptions

An occupancy grid divides the environment into a regular grid of N cells $\{m_1, m_2, \dots, m_N\}$. Each cell m_i is a binary random variable:

$$m_i = \begin{cases} 1 & \text{cell is occupied,} \\ 0 & \text{cell is free.} \end{cases}$$

Three modelling assumptions make the problem tractable:

1. **Binary occupancy**: each cell is either completely occupied or completely free; the area of one cell is small enough that this is a good approximation.
2. **Static environment**: occupancy states do not change over time. A cell that is occupied remains occupied and vice versa.
3. **Cell independence**: the random variables $\{m_i\}$ are mutually independent. This is the critical assumption that allows the joint map distribution to factorize.

48.2 Factorization of the Map Distribution

Under the independence assumption, the joint posterior over the entire map decomposes into a product of per-cell marginals:

Map Posterior Factorization

$$P(m \mid z_{1:t}, x_{1:t}) = \prod_{i=1}^N P(m_i \mid z_{1:t}, x_{1:t}).$$

This is a crucial simplification: instead of maintaining a joint distribution over 2^N map configurations (exponential in N), we maintain N independent Bernoulli posteriors.

49 Binary Bayes Filter for Static States

49.1 Derivation of the Update Rule

Each cell is estimated independently using a **static-state binary Bayes filter**. Because the state (occupancy) does not change, there is no prediction step; only the correction step is needed.

Starting from Bayes' rule and applying the Markov assumption (the current measurement z_t is conditionally independent of past measurements given the current pose and cell state):

$$P(m_i \mid z_{1:t}, x_{1:t}) \propto P(z_t \mid m_i, x_t) P(m_i \mid z_{1:t-1}, x_{1:t-1}).$$

To avoid numerical issues with repeated multiplication of small probabilities, we work with the **odds ratio**:

$$\text{Odds}(m_i \mid z_{1:t}, x_{1:t}) = \frac{P(m_i \mid z_{1:t}, x_{1:t})}{1 - P(m_i \mid z_{1:t}, x_{1:t})}.$$

Computing the ratio $P(m_i \mid z_{1:t}, x_{1:t}) / P(\neg m_i \mid z_{1:t}, x_{1:t})$ via Bayes' rule applied twice (once for occupied, once for free) and cancelling the normalizer yields:

Odds Update Rule

$$\frac{P(m_i \mid z_{1:t}, x_{1:t})}{1 - P(m_i \mid z_{1:t}, x_{1:t})} = \frac{P(m_i \mid z_t, x_t)}{1 - P(m_i \mid z_t, x_t)} \cdot \frac{P(m_i \mid z_{1:t-1}, x_{1:t-1})}{1 - P(m_i \mid z_{1:t-1}, x_{1:t-1})} \cdot \frac{1 - P(m_i)}{P(m_i)}.$$

The three factors are: the *inverse sensor model* odds, the *previous posterior* odds, and the *prior* odds (inverted).

49.2 Log Odds Representation

Taking the logarithm of the odds turns the product into a sum, making the update $O(1)$ per cell per time step:

Log Odds Definition and Update

Define the log odds of a probability p as:

$$l(p) = \log \frac{p}{1-p}.$$

Recover probability from log odds:

$$p = 1 - \frac{1}{1 + e^l}.$$

The occupancy mapping update in log odds form is:

$$l_{t,i} = \underbrace{l(m_i | z_t, x_t)}_{\text{inverse sensor model}} + \underbrace{l_{t-1,i}}_{\text{previous log odds}} - \underbrace{l_0}_{\text{prior log odds}},$$

where $l_0 = l(P(m_i))$ encodes the prior belief (typically $l_0 = 0$ for $P(m_i) = 0.5$).

Remark. The log odds form is the reason occupancy mapping is computationally cheap. Each cell update is a single floating-point addition. The inverse sensor model precomputes a lookup table indexed by range and angle, so the total cost per scan is proportional to the number of cells in the sensor's perceptual field — far fewer than N in practice.

50 Occupancy Grid Mapping Algorithm

Algorithm 7 Occupancy Grid Mapping

Input: $\{l_{t-1,i}\}$ (log odds for all cells), pose x_t , measurement z_t

Output: $\{l_{t,i}\}$ (updated log odds)

for all cells m_i **do**

if m_i is in the perceptual field of z_t **then**

$l_{t,i} \leftarrow l_{t-1,i} + \text{InverseSensorModel}(m_i, x_t, z_t) - l_0$

else

$l_{t,i} \leftarrow l_{t-1,i}$

end if

end for

Return $\{l_{t,i}\}$

Efficiency: only cells inside the sensor's perceptual field receive an update; cells outside are untouched. For a laser rangefinder with a narrow beam, this is a small fraction of the total grid.

51 Inverse Sensor Models

51.1 Role of the Inverse Sensor Model

The **inverse sensor model** $P(m_i | z_t, x_t)$ — equivalently its log odds $l(m_i | z_t, x_t)$ — answers: given that the sensor at pose x_t returned measurement z_t , how does that update the belief that cell m_i is occupied?

It is the reverse of the *forward* sensor model $P(z_t | m_i, x_t)$; deriving it analytically via Bayes' rule is possible but requires a prior and a forward model. In practice, simple geometric models are used directly.

51.2 Laser Rangefinder Model

A laser rangefinder returns a distance z_r along beam direction ϕ . The geometric interpretation is:

- **Occupied:** the cell through which the beam terminates (at range $z_r \pm \alpha/2$, where α is a thickness parameter) is very likely occupied $\rightarrow$ return l_{occ} .
- **Free:** cells along the beam path between the sensor and the endpoint are very likely free $\rightarrow$ return l_{free} .
- **Unknown:** cells beyond the beam endpoint or outside the beam cone $\rightarrow$ return l_0 (no update).

Algorithm 8 Inverse Range Sensor Model (Laser)

```

Let  $(x_i, y_i)$  be the center of cell  $m_i$ 
 $r \leftarrow \sqrt{(x_i - x)^2 + (y_i - y)^2}$  (distance to cell center)
 $\theta \leftarrow \text{atan2}(y_i - y, x_i - x) - \phi_{\text{robot}}$  (bearing to cell)
 $k \leftarrow \arg \min_k |\theta - \phi_k|$  (nearest beam index)
if  $r > \min(z_{\text{max}}, z_k + \alpha/2)$  or  $|\theta - \phi_k| > \beta/2$  then
    return  $l_0$  (outside perceptual field)
end if
if  $z_k < z_{\text{max}}$  and  $|r - z_k| < \alpha/2$  then
    return  $l_{\text{occ}}$  (at beam endpoint: occupied)
end if
if  $r < z_k$  then
    return  $l_{\text{free}}$  (between sensor and endpoint: free)
end if

```

Parameters: z_{max} is the maximum sensor range; α is the hit-region half-thickness; β is the beam's angular width; $l_{\text{occ}} > l_0 > l_{\text{free}}$.

51.3 Sonar Model

Ultrasonic sensors emit a cone-shaped acoustic pulse. The model is qualitatively similar to the laser model but smeared angularly:

- Cells on the beam axis near range z_r are most likely occupied.
- Cells within the cone but not on the axis are less certain (reduced l_{occ} weight).
- Cells inside the cone and between sensor and z_r are free.

Because of the wide beam angle, sonar maps are noisier and boundaries are less sharp compared to laser-based maps.

52 Map Quality and Practical Considerations

52.1 Sensor Comparison

Sensor	Resolution	Map quality
24-beam sonar array	Wide cone, noisy	Blurry boundaries, uncertain occupancy
Laser rangefinder	Narrow beam, precise	Sharp boundaries, accurate free space

The quality of the occupancy map is bounded by sensor precision. A laser scanner with sub-centimetre range accuracy and 0.25° angular resolution produces maps visually indistinguishable from architectural blueprints, whereas sonar maps are only suitable for coarse obstacle avoidance.

52.2 Comparison with Feature Maps

Feature maps (lines, corners, landmarks) are compact and support closed-form probabilistic updates (EKF-SLAM), but they require reliable feature extraction, which fails in featureless or repetitive environments. Occupancy grids require no feature detection: any sensor that returns range data or image intensity can be used. The cost is memory: a $100\text{ m} \times 100\text{ m}$ environment at 5 cm resolution requires 4×10^6 cells, which is manageable in 2D but prohibitive in 3D without volumetric compression (e.g., OctoMap).

52.3 Summary of Key Properties

- **Non-parametric:** the grid itself is the map; no structural assumptions about the environment.
- **Efficient update:** $O(K)$ per scan where K is the number of cells in the perceptual field, not the total grid size N .
- **Numerically stable:** log odds avoids probability underflow from repeated multiplication.
- **Extendable:** the same framework supports 3D grids, probabilistic truncated signed distance fields (TSDF), and multi-resolution representations.
- **Limitation:** assumes a static world; dynamic obstacles violate the static-state assumption and must be handled by extensions (e.g., dynamic occupancy grids with particle-based cell state tracking).

Part XII

Probabilistic Observation Models

53 Role of the Observation Model

The observation model $P(z_t | x_t, m)$ is the core ingredient of the Bayes filter correction step. Recall the recursive state estimation equations:

Bayes Filter: Prediction and Correction

$$\begin{aligned}\overline{\text{Bel}}(x_t) &= \int P(x_t | x_{t-1}, u_t) \text{Bel}(x_{t-1}) dx_{t-1}, \\ \text{Bel}(x_t) &= \eta P(z_t | x_t) \overline{\text{Bel}}(x_t).\end{aligned}$$

The *motion model* $P(x_t | x_{t-1}, u_t)$ propagates belief through controls. The *observation model* $P(z_t | x_t, m)$ weights the predicted belief by how well a measurement z_t is explained by each candidate pose x_t given the map m . The quality of localization is therefore directly determined by the accuracy of this model.

54 Range Sensors and the Scan Factorization

Range sensors (laser scanners, ultrasound) are the primary sensor type in range-based localization. A full scan z_t at time t consists of K individual beam measurements:

$$z_t = \{z_t^1, z_t^2, \dots, z_t^K\}.$$

Under the **conditional independence assumption** — each beam is independent of all others given the robot pose and map — the joint scan likelihood factorizes:

Scan Likelihood Factorization

$$P(z_t | x_t, m) = \prod_{k=1}^K P(z_t^k | x_t, m).$$

This reduces the problem from modelling a joint distribution over all beams simultaneously (exponential in K) to modelling each beam independently. All models below specify $P(z_t^k | x_t, m)$ for a single beam k .

55 Beam-Based Model (Advanced Ray-Cast Model)

55.1 Physical Decomposition

A laser rangefinder can fail in several physically distinct ways. The beam-based model captures all of them via a **mixture of four components**:

Beam-Based Mixture Model

$$P(z_t^k | x_t, m) = z_{\text{hit}} p_{\text{hit}} + z_{\text{short}} p_{\text{short}} + z_{\text{rand}} p_{\text{rand}} + z_{\text{max}} p_{\text{max}},$$

with mixing weights satisfying $z_{\text{hit}} + z_{\text{short}} + z_{\text{rand}} + z_{\text{max}} = 1$.

Let z_t^{k*} denote the *expected* range for beam k at pose x_t , obtained by ray-casting the map m .

55.2 Component 1: Correct Range with Gaussian Noise (p_{hit})

Physical cause: the beam hits the correct obstacle and the measurement is corrupted only by small sensor noise.

$$p_{\text{hit}}(z_t^k | x_t, m) = \begin{cases} \eta \mathcal{N}(z_t^k; z_t^{k*}, \sigma_{\text{hit}}^2) & 0 \leq z_t^k \leq z_{\text{max}}, \\ 0 & \text{otherwise,} \end{cases}$$

where $\eta = (\int_0^{z_{\text{max}}} \mathcal{N}(z; z_t^{k*}, \sigma_{\text{hit}}^2) dz)^{-1}$ normalizes the truncated Gaussian to $[0, z_{\text{max}}]$. The parameter σ_{hit} captures the sensor's range precision.

55.3 Component 2: Unexpected Obstacle (p_{short})

Physical cause: an unmodelled object (person, furniture, dynamic obstacle) sits between the robot and the mapped obstacle, causing an early return.

$$p_{\text{short}}(z_t^k | x_t, m) = \begin{cases} \eta \lambda_{\text{short}} e^{-\lambda_{\text{short}} z_t^k} & 0 \leq z_t^k \leq z_t^{k*}, \\ 0 & \text{otherwise,} \end{cases}$$

with normalization $\eta = (1 - e^{-\lambda_{\text{short}} z_t^{k*}})^{-1}$. The exponential distribution assigns higher probability to short readings than to readings near the true range z_t^{k*} , consistent with an obstacle that could be anywhere between the robot and the mapped surface.

55.4 Component 3: Random Measurement (p_{rand})

Physical cause: completely erratic readings due to multi-path reflections, sensor crosstalk, or electronic noise.

$$p_{\text{rand}}(z_t^k \mid x_t, m) = \begin{cases} \frac{1}{z_{\text{max}}} & 0 \leq z_t^k \leq z_{\text{max}}, \\ 0 & \text{otherwise.} \end{cases}$$

A uniform distribution over $[0, z_{\text{max}}]$ models complete uncertainty: the reading carries no information about the environment.

55.5 Component 4: Maximum Range Failure (p_{max})

Physical cause: the beam misses all obstacles (e.g., hits glass, dark surface, or goes out the window) and the sensor saturates at z_{max} .

$$p_{\text{max}}(z_t^k \mid x_t, m) = \begin{cases} 1 & z_t^k = z_{\text{max}}, \\ 0 & \text{otherwise.} \end{cases}$$

This is a point mass (Dirac delta) at the maximum range value. In practice, a reading of exactly z_{max} is a strong signal that the beam failed to detect any obstacle.

55.6 Parameter Learning

The mixing weights ($z_{\text{hit}}, z_{\text{short}}, z_{\text{rand}}, z_{\text{max}}$) and the noise parameters ($\sigma_{\text{hit}}, \lambda_{\text{short}}$) can be learned from real sensor data via maximum likelihood estimation. In practice, they are often set by hand after visual inspection of histograms of residuals $z_t^k - z_t^{k*}$.

Remark. The beam-based model is the most physically accurate model for laser rangefinders. Its main drawback is computational cost: for each candidate pose x_t , each of the K beams requires a ray-cast query against the map to compute z_t^{k*} . For particle filters with thousands of particles, this dominates the runtime.

56 Likelihood Field Model

56.1 Motivation

The likelihood field model trades some accuracy for a large speedup by replacing per-particle ray-casting with a precomputed lookup table.

56.2 Construction

Given a 2D occupancy grid map m , the **likelihood field** $\ell(p)$ for a point $p = (x_p, y_p)$ in the map frame is defined as:

$$\ell(p) = \exp\left(-\frac{d^2(p, m)}{2\sigma^2}\right),$$

where $d(p, m)$ is the Euclidean distance from p to the nearest occupied cell in m . This is precomputed once for the entire map and stored as a 2D image.

56.3 Sensor Update

For a candidate robot pose $x_t = (x, y, \theta)$ and a range reading z_t^k at beam angle ϕ_k relative to the robot:

1. Project the beam endpoint into the map frame:

$$\begin{pmatrix} x_z \\ y_z \end{pmatrix} = \begin{pmatrix} x \\ y \end{pmatrix} + z_t^k \begin{pmatrix} \cos(\theta + \phi_k) \\ \sin(\theta + \phi_k) \end{pmatrix}.$$

2. Look up the precomputed likelihood at the projected point: $\ell(x_z, y_z)$.
3. The per-beam likelihood is:

$$P(z_t^k | x_t, m) = z_{\text{hit}} \ell(x_z, y_z) + z_{\text{rand}} \frac{1}{z_{\text{max}}} + z_{\text{max}} \mathcal{I}(z_t^k = z_{\text{max}}).$$

(The p_{short} component is typically omitted for speed.)

No ray-casting against the map is required: the entire update is a table lookup followed by a simple formula.

Remark. The likelihood field ignores the physical structure of the beam path (it does not check whether the beam is occluded before reaching the endpoint). This can cause wrong weights when walls obstruct the line of sight, but in practice the approach is robust and widely used in real-time systems.

56.4 Comparison of Beam and Likelihood Field Models

Model	Accuracy	Speed	Primary use
Simple ray-cast	Low	Fast	Idealized/debug
Beam-based (mixture)	High	Slow	Offline, accurate localization
Likelihood field	Medium	Fast	Real-time particle filter

57 Landmark-Based Observation Models

57.1 Range-Bearing Model

When the environment contains identifiable landmarks (beacons, visual markers, mapped corners), a range-bearing sensor at robot pose $x_t = (x, y, \theta)^\top$ observing landmark j at map position $(m_{j,x}, m_{j,y})^\top$ produces:

Range-Bearing Observation Model

$$\begin{pmatrix} r_t^j \\ \phi_t^j \end{pmatrix} = \begin{pmatrix} \sqrt{(m_{j,x} - x)^2 + (m_{j,y} - y)^2} \\ \text{atan2}(m_{j,y} - y, m_{j,x} - x) - \theta \end{pmatrix} + \begin{pmatrix} \epsilon_{r^j} \\ \epsilon_{\phi^j} \end{pmatrix},$$

where $\epsilon_{r^j} \sim \mathcal{N}(0, \sigma_r^2)$ and $\epsilon_{\phi^j} \sim \mathcal{N}(0, \sigma_\phi^2)$ are independent Gaussian noise terms.

The likelihood of an observation (r_t^j, ϕ_t^j) given pose x_t and known landmark j is:

$$P(z_t^j | x_t, m) = \mathcal{N}(r_t^j; \hat{r}^j, \sigma_r^2) \mathcal{N}(\phi_t^j; \hat{\phi}^j, \sigma_\phi^2),$$

where $\hat{r}^j$ and $\hat{\phi}^j$ are the expected range and bearing computed from x_t and the map.

57.2 Bearing-Only Model

If the sensor provides only bearing (angle) with no range information:

$$\phi_t^j = \text{atan2}(m_{j,y} - y, m_{j,x} - x) - \theta + \epsilon_{\phi^j}.$$

A single bearing observation constrains the robot to lie on a ray from the landmark direction but leaves its distance along that ray uncertain. The posterior after one bearing observation is therefore a ring-shaped distribution in pose space (a distribution over (x, y, θ) concentrated along a manifold at the observed angle from the landmark). This is illustrated clearly by particle filters: the particle cloud forms an arc consistent with observing a landmark at angle ϕ from distance unknown.

Example 57.1. In the AIBO RoboCup soccer domain, robots self-localize on the pitch using coloured beacons and white-line detections. A single landmark observed at bearing 30° and range 5 m constrains the robot to be on a circle of radius 5 m centred at the landmark, at an angle consistent with the observed bearing. Multiple such observations rapidly collapse the distribution to a narrow cluster.

58 Summary

- The observation model $P(z_t | x_t, m)$ is the likelihood of a measurement given a pose; it is the key component of the Bayes filter correction step.
- For range sensors, the independence assumption over beams yields $P(z_t | x_t, m) = \prod_k P(z_t^k | x_t, m)$, making the model tractable.
- The **beam-based model** is a four-component mixture capturing Gaussian sensor noise, unexpected short returns, random errors, and max-range failures. It is physically accurate but requires per-particle ray-casting.
- The **likelihood field model** precomputes a distance-to-obstacle map and replaces ray-casting with a table lookup, enabling real-time sensor updates in particle filters at modest accuracy cost.
- **Landmark range-bearing models** give Gaussian likelihoods directly from the geometric observation equation; bearing-only observations produce ring-shaped posteriors, requiring multiple observations to localize.

Part XIII

Probabilistic Motion Models

59 Motion Uncertainty and the Motion Model

59.1 Why Model Uncertainty?

Real robot motion is never perfect. Even when a robot commands a specific displacement, the actual trajectory deviates due to:

- terrain irregularities (bumps, carpet, slopes),
- wheel slippage and unequal wheel diameters,

- discretization and quantization in encoders,
- violations of the kinematic rolling constraint under high lateral forces.

These errors compound: a robot navigating over several metres accumulates significant pose uncertainty. The **probabilistic motion model** $P(x_t \mid u_t, x_{t-1})$ encodes this uncertainty as a probability distribution over the next pose x_t , given control input u_t and previous pose x_{t-1} .

59.2 Role in the Bayes Filter

The motion model is the prediction step of the Bayes filter:

$$\overline{\text{Bel}}(x_t) = \int P(x_t \mid u_t, x_{t-1}) \text{Bel}(x_{t-1}) dx_{t-1}.$$

Without sensing, integrating $P(x_t \mid u_t, x_{t-1})$ over time causes uncertainty to grow without bound. Sensor corrections (the correction step) are required to keep the belief concentrated.

59.3 Two Model Families

Model	Input	Use case
Odometry-based	Encoder readings (δ_{rot1} , δ_{trans} , δ_{rot2})	When wheel encoders are available
Velocity-based	Velocity commands (v , ω)	When encoders are unavailable (dead reckoning)

60 Odometry Motion Model

60.1 Motion Decomposition

Odometry decomposes each motion step into three primitive actions: an initial in-place rotation δ_{rot1} , a straight-line translation δ_{trans} , and a final in-place rotation δ_{rot2} . Let the previous pose be $x_{t-1} = (\bar{x}, \bar{y}, \bar{\theta})^\top$ and the current pose $x_t = (x', y', \theta')^\top$ as reported by the encoders. Then:

Odometry Decomposition

$$\begin{aligned} \delta_{\text{trans}} &= \sqrt{(x' - \bar{x})^2 + (y' - \bar{y})^2}, \\ \delta_{\text{rot1}} &= \text{atan2}(y' - \bar{y}, x' - \bar{x}) - \bar{\theta}, \\ \delta_{\text{rot2}} &= \theta' - \bar{\theta} - \delta_{\text{rot1}}. \end{aligned}$$

The odometry reading is $u_t = (\delta_{\text{rot1}}, \delta_{\text{trans}}, \delta_{\text{rot2}})^\top$.

60.2 Noise Model

Each of the three odometry components is corrupted by zero-mean noise whose standard deviation scales with the magnitudes of the motion components (larger motions cause larger errors):

Odometry Noise Model

$$\begin{aligned}\hat{\delta}_{\text{rot1}} &= \delta_{\text{rot1}} + \varepsilon(\alpha_1|\delta_{\text{rot1}}| + \alpha_2|\delta_{\text{trans}}|), \\ \hat{\delta}_{\text{trans}} &= \delta_{\text{trans}} + \varepsilon(\alpha_3|\delta_{\text{trans}}| + \alpha_4(|\delta_{\text{rot1}}| + |\delta_{\text{rot2}}|)), \\ \hat{\delta}_{\text{rot2}} &= \delta_{\text{rot2}} + \varepsilon(\alpha_1|\delta_{\text{rot2}}| + \alpha_2|\delta_{\text{trans}}|),\end{aligned}$$

where $\varepsilon(\sigma^2)$ denotes zero-mean noise with variance σ^2 , and $\alpha_1, \dots, \alpha_4 > 0$ are platform-specific noise coefficients determined empirically.

Interpretation of noise coefficients:

- α_1 : rotation error proportional to rotation (encoder inaccuracy in turning).
- α_2 : rotation error proportional to translation (heading drift during straight motion).
- α_3 : translation error proportional to translation (wheel-radius mismatch, slippage).
- α_4 : translation error proportional to rotation (turning radius variations).

60.3 Noise Distributions

Two distributions are commonly used for $\varepsilon(\sigma^2)$:

Gaussian: $\varepsilon_{\sigma^2}(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{x^2}{2\sigma^2}\right)$. Unbounded support; natural for continuous measurement noise.

Triangular: $\varepsilon_{\sigma^2}(x) = \max\left\{0, \frac{1}{\sqrt{6}\sigma} - \frac{|x|}{6\sigma^2}\right\}$. Bounded support on $[-\sqrt{6}\sigma, \sqrt{6}\sigma]$; piecewise linear; often more realistic for hardware noise and cheaper to compute.

Corresponding sampling algorithms:

$$\text{sample_normal}(\sigma^2) = \frac{1}{12} \sum_{i=1}^{12} \text{rand}(-\sigma, \sigma), \quad \text{sample_triangular}(\sigma^2) = \frac{\sqrt{6}}{2} [\text{rand}(-\sigma, \sigma) + \text{rand}(-\sigma, \sigma)],$$

where $\text{rand}(a, b)$ draws uniformly from $[a, b]$. The normal sampler uses the central limit theorem (sum of 12 uniforms); the triangular sampler uses convolution of two uniforms.

60.4 Computing $P(x_t \mid u_t, x_{t-1})$: Density Evaluation

Given a query pair (x_{t-1}, x_t) and odometry u_t , the true motion components are:

$$\begin{aligned}\delta_{\text{trans}}^* &= \sqrt{(x_t^x - x_{t-1}^x)^2 + (x_t^y - x_{t-1}^y)^2}, \\ \delta_{\text{rot1}}^* &= \text{atan2}(x_t^y - x_{t-1}^y, x_t^x - x_{t-1}^x) - x_{t-1}^\theta, \\ \delta_{\text{rot2}}^* &= x_t^\theta - x_{t-1}^\theta - \delta_{\text{rot1}}^*.\end{aligned}$$

The probability is then the product of three independent noise densities:

Odometry Model: Density Evaluation

$$\begin{aligned}
p_1 &= \text{prob}(\delta_{\text{rot1}}^* - \delta_{\text{rot1}}, \alpha_1|\delta_{\text{rot1}}| + \alpha_2|\delta_{\text{trans}}|), \\
p_2 &= \text{prob}(\delta_{\text{trans}}^* - \delta_{\text{trans}}, \alpha_3|\delta_{\text{trans}}| + \alpha_4(|\delta_{\text{rot1}}| + |\delta_{\text{rot2}}|)), \\
p_3 &= \text{prob}(\delta_{\text{rot2}}^* - \delta_{\text{rot2}}, \alpha_1|\delta_{\text{rot2}}| + \alpha_2|\delta_{\text{trans}}|), \\
P(x_t | u_t, x_{t-1}) &= p_1 \cdot p_2 \cdot p_3,
\end{aligned}$$

where $\text{prob}(a, \sigma^2)$ evaluates the chosen noise density (Gaussian or triangular) at a with variance σ^2 .

60.5 Sampling from $P(x_t | u_t, x_{t-1})$: Particle Generation

For particle filters, we need to draw samples $x_t \sim P(x_t | u_t, x_{t-1})$ rather than evaluate the density. The algorithm adds noise to each odometry component and then applies the kinematic update:

Algorithm 9 Sample Odometry Motion Model

Input: $x_{t-1} = (x, y, \theta)^\top$, odometry $u_t = (\delta_{\text{rot1}}, \delta_{\text{trans}}, \delta_{\text{rot2}})^\top$
 Extract raw odometry components from u_t
 $\hat{\delta}_{\text{rot1}} \leftarrow \delta_{\text{rot1}} - \text{sample}(\alpha_1|\delta_{\text{rot1}}| + \alpha_2|\delta_{\text{trans}}|)$
 $\hat{\delta}_{\text{trans}} \leftarrow \delta_{\text{trans}} - \text{sample}(\alpha_3|\delta_{\text{trans}}| + \alpha_4(|\delta_{\text{rot1}}| + |\delta_{\text{rot2}}|))$
 $\hat{\delta}_{\text{rot2}} \leftarrow \delta_{\text{rot2}} - \text{sample}(\alpha_1|\delta_{\text{rot2}}| + \alpha_2|\delta_{\text{trans}}|)$
 $x' \leftarrow x + \hat{\delta}_{\text{trans}} \cos(\theta + \hat{\delta}_{\text{rot1}})$
 $y' \leftarrow y + \hat{\delta}_{\text{trans}} \sin(\theta + \hat{\delta}_{\text{rot1}})$
 $\theta' \leftarrow \theta + \hat{\delta}_{\text{rot1}} + \hat{\delta}_{\text{rot2}}$
Return $(x', y', \theta')^\top$

Each call generates one sample; running this for N particles in parallel implements the prediction step of a particle filter.

61 Velocity-Based Motion Model**61.1 Kinematics**

When wheel encoders are unavailable, the robot's commanded linear velocity v and angular velocity ω serve as the motion input. For a robot following a circular arc of radius v/ω over time interval Δt , the ideal (noiseless) state update is:

$$\begin{aligned}
x' &= x - \frac{v}{\omega} \sin \theta + \frac{v}{\omega} \sin(\theta + \omega \Delta t), \\
y' &= y + \frac{v}{\omega} \cos \theta - \frac{v}{\omega} \cos(\theta + \omega \Delta t), \\
\theta' &= \theta + \omega \Delta t.
\end{aligned}$$

61.2 Noise Model

Additive zero-mean noise is applied to both velocity components before integrating the kinematics:

Velocity Noise Model

$$\begin{aligned}\hat{v} &= v + \varepsilon(\alpha_1|v| + \alpha_2|\omega|), \\ \hat{\omega} &= \omega + \varepsilon(\alpha_3|v| + \alpha_4|\omega|), \\ \hat{\gamma} &= \varepsilon(\alpha_5|v| + \alpha_6|\omega|).\end{aligned}$$

The term $\hat{\gamma}$ is an additional independent noise component that models final-heading uncertainty (e.g., slippage affecting orientation but not the arc trajectory). It is absent in a naive two-parameter noise model, which underestimates heading error.

61.3 State Update with Noise

Velocity Model: Noisy State Update

$$\begin{aligned}x' &= x - \frac{\hat{v}}{\hat{\omega}} \sin \theta + \frac{\hat{v}}{\hat{\omega}} \sin(\theta + \hat{\omega}\Delta t), \\ y' &= y + \frac{\hat{v}}{\hat{\omega}} \cos \theta - \frac{\hat{v}}{\hat{\omega}} \cos(\theta + \hat{\omega}\Delta t), \\ \theta' &= \theta + \hat{\omega}\Delta t + \hat{\gamma}\Delta t.\end{aligned}$$

The ratio $\hat{v}/\hat{\omega}$ is the noisy radius of curvature. The $\hat{\gamma}\Delta t$ term adds independent rotational drift to the final heading.

Remark. The velocity model is less accurate than the odometry model when encoders are available, because it relies on commanded velocities rather than direct wheel measurements. However, it is general enough to apply to any wheeled or aerial platform described by (v, ω) .

61.4 Resulting Distributions

The distribution $P(x_t | u_t, x_{t-1})$ under either model is in general non-Gaussian and multi-modal when projected onto 2D. Running the sample algorithm N times from a single starting pose and visualising the particle cloud reveals:

- **Straight motion:** the cloud is elongated along the direction of travel (translation noise dominates).
- **Turning motion:** the cloud fans out in orientation (rotation noise dominates).
- **Long sequences:** the cloud diffuses rapidly; without sensor corrections, particles spread across the entire reachable space.

62 Summary

- The motion model $P(x_t | u_t, x_{t-1})$ is the prediction ingredient of the Bayes filter; it propagates uncertainty forward in time.
- **Odometry model:** decomposes motion into $(\delta_{\text{rot1}}, \delta_{\text{trans}}, \delta_{\text{rot2}})$; noise scales with motion magnitude via four coefficients $\alpha_{1:4}$.
- **Velocity model:** integrates noisy (v, ω) along a circular arc; an additional γ term models heading drift not captured by velocity noise alone.

- Both models support two operations: *density evaluation* (used in grid-based filters) and *sampling* (used in particle filters). The sampling form is the prediction step of Monte Carlo Localization.
- Gaussian and triangular distributions are the standard choices for the noise density; the triangular distribution is bounded and cheaper to sample.

Part XIV

Robot Motion Planning and Navigation

63 Overview and Historical Context

63.1 What is Motion Planning?

Motion planning is the problem of determining a set of actions (a plan) that brings a robot from a known start state to a known goal state while satisfying constraints. The term encompasses several sub-disciplines:

- **Action planning:** find a sequence of symbolic actions that realize a given objective.
- **Path planning:** find a valid geometric path from one configuration to another; local reactive avoidance handles inaccuracies and dynamic obstacles.
- **Planning under uncertainty:** find a policy that maximizes long-term reward (MDPs, POMDPs).

63.2 Historical Development

Era	Paradigm	Characteristics
Mid-1970s	Classic robotics	Exact models, no sensing (Voronoi diagrams)
Mid-1980s	Reactive planning	No models; sensor-only; high-speed avoidance
Early 1990s	Hybrid/hierarchical	Models at high level, reactive at low level
Mid-1990s+	Probabilistic planning	Uncertainty in models and sensors throughout

Modern systems combine a **global planner** (low frequency, map-based, e.g. Dijkstra or A*) with a **local planner** (high frequency, reactive, e.g. DWA), as in the ROS navigation stack.

64 Configuration Space

64.1 Definitions

Let $\mathcal{W} = \mathbb{R}^m$ be the **work space** with obstacle region $\mathcal{O} \subset \mathcal{W}$. For a robot in configuration $q \in \mathcal{C}$, let $A(q) \subset \mathcal{W}$ denote the set of points occupied by the robot.

Configuration Space Partition

$$\begin{aligned}\mathcal{C}_{\text{free}} &= \{q \in \mathcal{C} \mid A(q) \cap \mathcal{O} = \emptyset\}, \\ \mathcal{C}_{\text{obs}} &= \mathcal{C} \setminus \mathcal{C}_{\text{free}}.\end{aligned}$$

The **motion planning problem**: find a continuous path $\tau : [0, 1] \rightarrow \mathcal{C}_{\text{free}}$ with $\tau(0) = q_i$ (start) and $\tau(1) = q_G$ (goal).

64.2 Mobile Robot C-Space Simplifications

A mobile robot's configuration is $(x, y, \theta) \in \mathbb{R}^2 \times S^1$. In practice, two simplifications reduce this to 2D:

1. **Ignore kinematic constraints** (treat the robot as holonomic — valid for differential-drive robots that can rotate in place; non-holonomic constraints are then delegated to the local planner, e.g. DWA).
2. **Treat the robot as a point by inflating obstacles** by the robot's radius (Minkowski sum).

Under both simplifications, the C-space reduces to a 2D grid with inflated obstacles, which is identical to the occupancy map used for localization.

65 Combinatorial C-Space Representations**65.1 General Strategy**

Combinatorial approaches decompose $\mathcal{C}_{\text{free}}$ and capture its connectivity in a graph, then apply a graph-search algorithm. They are **complete** but suffer from the **curse of dimensionality**: the number of cells grows exponentially with the dimension of $\mathcal{C}$, making them intractable for manipulation or molecular planning.

65.2 Visibility Graph

Connect every pair of vertices (obstacle corners, start, goal) with a straight line segment that lies entirely in $\mathcal{C}_{\text{free}}$. Path planning reduces to a shortest-path search on this graph.

- + Complete; geometrically optimal path length; fast for polygonal environments.
- Paths graze obstacle surfaces; unsafe without inflation.

65.3 Voronoi Diagram

Partition $\mathcal{C}_{\text{free}}$ into regions equidistant from two or more obstacles. The Voronoi edges form a roadmap that maximizes clearance from obstacles.

- + Complete; maximizes distance from obstacles (safe paths).
- Path length is not optimal; sparse coverage problematic for short-range sensors.

65.4 Exact Cell Decomposition

Decompose $\mathcal{C}_{\text{free}}$ into cells whose boundaries coincide with geometric critical points of the obstacles. Adjacent cells are linked in a connectivity graph; path planning finds a sequence of adjacent cells.

- + Complete; efficient in sparse environments.
- Difficult to implement; inefficient in dense environments.

65.5 Approximate Cell Decomposition

Overlay a regular grid; every cell is labeled *free* or *occupied*. This is the standard occupancy grid representation used for mapping.

- + Unified representation with the map; de-facto standard in robotics.
- Narrow passages may disappear at coarse resolution; limits path shapes.

Adaptive (quadtree) decomposition refines grid resolution near obstacles, improving accuracy at the cost of variable-time preprocessing.

66 Potential Field Methods

Potential field methods define a scalar field $U(q)$ over $\mathcal{C}$ whose gradient drives the robot toward the goal and away from obstacles.

Potential Field Components

$$U_{\text{att}}(q) = \frac{1}{2}k_{\text{att}}\rho_{\text{goal}}^2(q),$$

$$U_{\text{rep}}(q) = \begin{cases} \frac{1}{2}k_{\text{rep}}\left(\frac{1}{\rho(q)} - \frac{1}{\rho_0}\right)^2 & \text{if } \rho(q) \leq \rho_0, \\ 0 & \text{otherwise,} \end{cases}$$

where $\rho_{\text{goal}}(q)$ is the distance to the goal, $\rho(q)$ is the distance to the nearest obstacle, and ρ_0 is the influence radius of obstacles.

The robot velocity (or force) is set proportional to the negative gradient:

$$F(q) = -\nabla U_{\text{att}}(q) - \nabla U_{\text{rep}}(q).$$

- + Closed-form control law; very fast to evaluate.
- Not complete: local minima (where $F = 0$ but $q \neq q_G$) cause the robot to stall; oscillations can occur near concave obstacles.

67 Sampling-Based Methods

67.1 Motivation

Combinatorial methods become intractable for high-dimensional $\mathcal{C}$. Sampling-based methods avoid explicit representation of $\mathcal{C}_{\text{free}}$: they *probe* $\mathcal{C}$ by sampling configurations and querying a collision-detection oracle. They are **probabilistically complete** (probability of failure decays to zero as sample count grows) but not complete in the strict sense.

67.2 Probabilistic Road Map (PRM)

1. **Learning phase:** sample random configurations in $\mathcal{C}$, retain those in $\mathcal{C}_{\text{free}}$, and attempt to connect nearby samples with a simple local planner. Successfully connected pairs become graph edges.

2. **Query phase:** connect q_i and q_G to the graph and apply a graph-search (e.g. A*).
- + Simple and effective; handles high-dimensional spaces.
 - Sub-optimal; poor coverage of narrow passages; requires dense sampling.

67.3 Rapidly Exploring Random Trees (RRT)

1. Initialize a tree $\mathcal{T}$ rooted at q_i .
 2. Sample a random configuration q_{rand} ; find the nearest tree node q_{near} .
 3. Extend q_{near} toward q_{rand} by a step size; add the new node q_{new} if the extension is collision-free.
 4. Repeat until q_G is reached or a budget is exhausted.
- + Naturally handles kinodynamic constraints; suitable for dynamic environments.
 - Sub-optimal; paths are jagged and difficult to smooth; non-deterministic.

68 Graph-Search Algorithms

68.1 Problem Setup and Complexity Metrics

Once the environment is represented as a graph, path planning is a graph-search problem. Algorithm complexity is measured by:

- b : branching factor (max successors per node).
- d : depth of the least-cost solution.
- m : maximum depth of the state space.
- C^* : cost of the optimal solution; ε : minimum edge cost.

Remark. **Tree search** re-expands nodes; **graph search** maintains a closed set to avoid revisiting nodes, reducing time complexity at the cost of memory.

68.2 Uninformed Search: Algorithm Comparison

Algorithm	Complete	Time	Space	Optimal
BFS	Yes*	b^d	b^d	Yes*
UCS (Dijkstra)	Yes	$b^{\lceil C^*/\varepsilon \rceil}$	$b^{\lceil C^*/\varepsilon \rceil}$	Yes
DFS	No	b^m	bm	No
IDS	Yes*	b^d	bd	Yes*

* If branching factor is finite and (for optimality) step costs are equal.

68.3 A* Search

A* is an informed search algorithm that expands nodes in order of $f(n) = g(n) + h(n)$, where:

- $g(n)$: exact cost from start to node n .
- $h(n)$: heuristic estimate of cost from n to goal.

A* Optimality Conditions

- **Admissible heuristic:** $h(n) \leq h^*(n)$ (never overestimates true cost) $\Rightarrow$ A* *tree search* is optimal.
- **Consistent heuristic:** $h(n) \leq c(n, a, n') + h(n')$ for all edges (n, n') $\Rightarrow$ A* *graph search* is optimal and optimally efficient.
- Consistency $\Rightarrow$ admissibility (converse does not hold).

Typical heuristics: Euclidean distance, Manhattan distance. $h \equiv 0$ reduces A* to Dijkstra/UCS.

Optimality efficiency: A* expands all nodes with $f(n) < C^*$, some with $f(n) = C^*$, and none with $f(n) > C^*$ — no other optimal algorithm expands fewer nodes (for a given h).

69 Multi-Agent Path Finding (MAPF)

69.1 Problem Definition

MAPF Problem

Given n agents on a known graph, each with a start location s_i and goal location g_i , find collision-free paths for all agents minimizing an objective:

- **Makespan:** $\max_i$ (arrival time of agent i).
- **Flowtime** (Sum of Costs): $\sum_i$ (path length of agent i).

Theorem (Yu and LaValle): MAPF is NP-hard to solve optimally for both objectives.

69.2 Naive Approach and Its Limitations

The joint state is the tuple of all agent positions. A* on the joint state space has branching factor b^n and state space $|V|^n$, which is intractable for large n .

SIC heuristic (Sum of Individual Costs): $h = \sum_i h_i$, where h_i is the single-agent optimal cost. It is easy to compute and admissible, but the joint state space remains exponential.

69.3 Independent Detection (ID)

Algorithm 10 Independent Detection

```

Plan independently for each agent
while any two agents collide do
    Group all mutually colliding agents together
    Re-plan jointly for each group using A* + SIC
end while

```

Issue: when many agents share a bottleneck (e.g., a corridor), groups merge quickly and the joint problem becomes intractable again.

69.4 Conflict-Based Search (CBS)

CBS operates on a two-level search (Sharon et al., AIJ 2015). The low level plans single-agent paths subject to *constraints*; the high level resolves *conflicts* by adding constraints.

CBS Definitions

- **Conflict:** $\langle A, B, x, t \rangle$ — agents A and B both occupy location x at time t .
- **Constraint:** $\langle \text{agent}, x, t \rangle$ — agent is forbidden from occupying x at time t .

Algorithm 11 Conflict-Based Search (CBS)

Plan independently for each agent (no constraints); form root node of constraint tree
while current node has a conflict $\langle A, B, x, t \rangle$ **do**
 Branch: create two child nodes
 In child 1: add constraint $\langle A, x, t \rangle$; replan for A
 In child 2: add constraint $\langle B, x, t \rangle$; replan for B
 Expand lowest-cost node (best-first)
end while
Return current node's paths

Key trade-off: CBS expands $O(m)$ nodes where ID expands $O(m^2)$ in bottleneck scenarios. However, in open environments with many conflicts, the joint-state approach (ID with A^*) may be faster. **Meta-Agent CBS** (CBS + A^*) switches adaptively. CBS scales to approximately 40 agents depending on the map.

70 Multi-Agent Pickup and Delivery (MAPD)

70.1 From One-Shot to Lifelong Planning

MAPF is a one-shot problem: each agent has one pre-determined goal. In **MAPD**, agents execute a continuous stream of pickup-and-delivery tasks (e.g., a warehouse with arriving orders). The system must remain responsive (low latency) while maintaining good throughput.

70.2 Well-Formed MAPD Instances

A MAPD instance is **well-formed** if:

1. The number of tasks is finite.
2. The number of non-task parking locations $\geq$ number of agents.
3. For any two parking locations, a path exists that traverses no other parking location.

These conditions guarantee that agents can always park without blocking each other, ensuring solvability.

70.3 Token Passing (TP)

Token Passing Algorithm

1. A shared *token* stores all current tasks and all currently planned paths.
2. Agents take turns holding the token (round-robin).
3. The token-holder greedily assigns itself to the best available task (shortest A^* path, no endpoint conflict with already-assigned tasks).
4. The agent plans a collision-free path using A^* , then releases the token.

TP solves all well-formed MAPD instances. **Token Passing with Task Swaps (TPTS)** extends TP by allowing an agent to take over a task from another agent if it can complete it faster.

70.4 Centralized Assignment (CENTRAL)

CENTRAL assigns agents to tasks globally using the **Hungarian method** (optimal bipartite matching), then plans all paths jointly by solving the resulting one-shot MAPF.

70.5 Algorithm Trade-offs

Metric	TP	TPTS	CENTRAL	Best
Service time	worst	mid	best	CENTRAL
Throughput	worst	mid	best	CENTRAL
Makespan	worst	mid	best	CENTRAL
Runtime/step	10 ms	200 ms	4000 ms	TP

CENTRAL achieves the best solution quality at the cost of much higher computation per time step.

Multi-Item MAPD: robots carry more than one item per task (multiple pickups $s_i^1, \dots, s_i^n$ and deliveries $g_i^1, \dots, g_i^m$). Token Passing is extended so that agents may hold multiple tasks simultaneously; the token-holder allocates itself greedily considering all active tasks and releases the token if no valid assignment exists.

71 Obstacle Avoidance

Local obstacle avoidance runs online at high frequency, reacting to sensor readings to modify the robot's trajectory without full replanning.

71.1 Bug Algorithms

The Bug family provides simple reactive strategies:

- **Bug 1:** fully circumnavigate each obstacle once, then depart from the point on its boundary closest to the goal. *Complete* but highly inefficient.
- **Bug 2:** follow the obstacle boundary but depart as soon as the straight line to the goal is unobstructed. More efficient than Bug 1 in practice but *incomplete* (can loop on certain obstacle configurations).

71.2 Dynamic Window Approach (DWA)

DWA searches in the robot's **velocity space** (v, ω) for a safe, efficient command:

DWA Velocity Space

V_s = all feasible velocities (platform limits),
 $V_d = \{(v, \omega) \mid \text{reachable in one time step given current}(v, \omega) \text{ and acceleration limits}\},$
 $V_o = \{(v, \omega) \mid \text{robot can stop before hitting an obstacle}\},$
 $V_r = V_s \cap V_d \cap V_o$ (admissible velocities).

The command is selected by maximizing an objective over V_r :

$$(v^*, \omega^*) = \arg \max_{(v, \omega) \in V_r} a \cdot \text{heading}(v, \omega) + b \cdot \text{velocity}(v, \omega) + c \cdot \text{dist}(v, \omega),$$

where heading measures alignment with the goal direction, velocity encourages forward progress, and dist penalizes proximity to obstacles. Weights a, b, c are tunable.

Key properties:

- Respects kinodynamic limits (acceleration and deceleration constraints).
- Fast enough for real-time execution (the dynamic window reduces the search to a small 2D region).
- Prone to local optima; addressed by the **Global DWA** variant, which replaces heading with a wavefront-computed distance-to-goal map.
- DWA is the default local planner in the ROS navigation stack.

72 Summary

- The **configuration space** (C-space) unifies robot geometry and workspace obstacles into a single planning domain; mobile robots commonly reduce to 2D with inflated obstacles.
- **Combinatorial methods** (visibility graphs, Voronoi, cell decomposition) are complete but scale poorly with dimension.
- **Potential fields** provide a reactive closed-form control law but are incomplete due to local minima.
- **Sampling-based methods** (PRM, RRT) are probabilistically complete and scale to high-dimensional C-spaces at the cost of sub-optimality.
- **A*** is the standard informed graph-search; admissibility guarantees optimality; consistency further guarantees optimal efficiency.
- **MAPF** is NP-hard; practical solvers decompose the problem (ID, CBS) or leverage structure (Meta-Agent CBS). CBS is preferred when conflicts are sparse; joint search (ID) when they are dense.
- **MAPD** extends MAPF to lifelong task streams; Token Passing is the fastest solver; CEN-TRAL achieves the best solution quality.
- **DWA** is the standard local obstacle avoidance method in practice, combining kinodynamic feasibility with an objective function over the admissible velocity window.

Part XV**Value-Based Deep Reinforcement Learning****73 Markov Decision Processes****73.1 Formal Definition**

An MDP is a tuple $(\mathcal{S}, \mathcal{A}, R, P)$:

- $\mathcal{S}$: finite state space, $|\mathcal{S}| = n$.
- $\mathcal{A}$: finite action space, $|\mathcal{A}| = m$.
- **Transition function:** $p(s' | s, a) = \Pr[S_{t+1} = s' | S_t = s, A_t = a]$.
- **Reward function:** $r(s, a) = \mathbb{E}[R_{t+1} | S_t = s, A_t = a]$.

73.2 Markov and Stationarity Assumptions

The Markov property states that the future is conditionally independent of the past given the present:

$$\Pr\{R_{t+1}, S_{t+1} \mid S_t, A_t\} = \Pr\{R_{t+1}, S_{t+1} \mid S_0, A_0, \dots, S_t, A_t\}.$$

Stationarity further requires this distribution to be time-invariant. Together these two assumptions make the MDP framework tractable: the entire history collapses to a single state.

73.3 MDP Formulation for Mobile Robots

MDP element	Robotics counterpart
State s	Robot pose (x, y, θ)
Action a	Control command $u = (v, \omega)$
Transition p	Motion model $p(x' \mid x, u)$
Reward r	Task-dependent (e.g., +1 at goal, $-\delta$ per step, $-C$ on collision)

74 Value Functions and Q-Values

74.1 State Value Function

The **value** of state s under policy π is the expected discounted return when starting at s and following π :

State Value Function

$$V_\pi(s) = \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t R_{t+1} \mid S_0 = s, \pi \right],$$

where $0 \leq \gamma < 1$ is the **discount factor**, downweighting future rewards exponentially.

A discount factor close to 1 gives nearly equal weight to all future rewards (far-sighted); close to 0 makes the agent myopic.

74.2 Action-Value (Q) Function

The **Q-value** extends the value function to state-action pairs:

Action-Value Function

$$Q_\pi(s, a) = \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t R_{t+1} \mid S_0 = s, A_0 = a, \pi \right] = r(s, a) + \gamma \int p(s' \mid s, a) V_\pi(s') ds'.$$

The value and Q-value are related by $V_\pi(s) = Q_\pi(s, \pi(s))$.

Once Q_π is known, the optimal policy is simply $\pi^*(s) = \arg \max_a Q_\pi(s, a)$.

75 Tabular Q-Learning

75.1 Model-Free RL

In RL, the agent does not know $p(s' \mid s, a)$ or $r(s, a)$. It must learn Q from interaction:

- **Model-based:** learn p and r first, then plan. Fewer environment interactions needed, but model complexity can introduce bias.

- **Model-free:** estimate Q (or π) directly from experience. Simpler; no model bias; typically requires more data.

75.2 Q-Learning Update Rule

Q-learning is an off-policy, model-free algorithm. After executing action a in state s , observing reward r and next state s' , the Q-table is updated:

Tabular Q-Learning Update

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right],$$

where $0 < \alpha \leq 1$ is the learning rate. The term in brackets is the **TD error**: difference between the current estimate and the bootstrapped target $r + \gamma \max_{a'} Q(s', a')$.

Algorithm 12 Tabular Q-Learning

Initialize $Q(s, a)$ arbitrarily for all s, a ; observe initial state s

loop

 Select action a using ε -greedy policy on Q

 Execute a ; observe reward r and next state s'

$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$

$s \leftarrow s'$

end loop

Q-learning converges to Q^* under mild conditions (all state-action pairs visited infinitely often, step-size satisfying the stochastic approximation conditions). The ε -greedy policy ensures exploration during training.

75.3 Limitations of Tabular Q-Learning

Tabular Q-learning stores one scalar per (s, a) pair. For continuous or very large state spaces — such as robot pose, sensor readings, or image pixels — the table is intractable. **Function approximation** is required.

76 Neural Network Function Approximation

76.1 Feedforward Neural Networks

A feedforward network with L layers computes a function $f(x; w)$ by composing affine transformations and pointwise nonlinearities:

$$h^{(l)} = \sigma(W^{(l)}h^{(l-1)} + b^{(l)}), \quad h^{(0)} = x,$$

where $W^{(l)}, b^{(l)}$ are the weight matrix and bias at layer l , and σ is an activation function. The output layer is typically linear for regression tasks such as Q-value estimation.

76.2 Activation Functions

Name	Formula	Properties
Threshold	$\mathbf{1}[x \geq 0]$	Non-differentiable
Sigmoid	$\sigma(x) = (1 + e^{-x})^{-1}$	Saturates; output in $(0, 1)$
Tanh	$\tanh(x) = (e^x - e^{-x}) / (e^x + e^{-x})$	Saturates; zero-centered
ReLU	$\max(0, x)$	No saturation for $x > 0$; sparse
Softplus	$\log(1 + e^x)$	Smooth ReLU approximation

76.3 Universal Approximation Theorem

Universal Approximation (Hornik et al., 1989; Cybenko, 1989)

A feedforward network with a linear output layer and at least one hidden layer with a continuous squashing activation (sigmoid, tanh) can approximate any continuous function on a compact subset of $\mathbb{R}^n$ to arbitrary precision, given sufficient hidden units.

This theorem justifies using neural networks as Q-function approximators. **Deep networks** (many hidden layers) achieve high expressivity with compact parameterizations, enabling representation of complex Q-value landscapes.

76.4 Training: Gradient Descent and Backpropagation

Given a dataset $\{(x_n, y_n)\}$, training minimizes the mean squared loss:

$$\mathcal{L}(w) = \sum_n (f(x_n; w) - y_n)^2.$$

The loss is non-convex; it is minimized by gradient descent. Each weight is updated by:

$$w_{ij} \leftarrow w_{ij} - \eta \frac{\partial \mathcal{L}}{\partial w_{ij}},$$

where η is the learning rate. **Backpropagation** computes $\partial \mathcal{L} / \partial w$ efficiently via the chain rule, propagating error gradients backward through the computation graph.

76.5 Vanishing Gradients and ReLU

Saturating activations (sigmoid, tanh) have derivatives bounded above by 1. In deep networks, multiplying many such derivatives during backpropagation causes gradients to decay exponentially toward the early layers — the **vanishing gradient problem**.

ReLU ($h(x) = \max(0, x)$) mitigates this: its gradient is exactly 1 for $x > 0$ and 0 elsewhere, preventing exponential decay. Additional solutions include batch normalization, skip connections (residual networks), and careful weight initialization.

77 Deep Q-Learning

77.1 Gradient Q-Learning

Replace the Q-table with a neural network $Q_w(s, a)$ parametrized by weights w . The training objective is to minimize the squared TD error:

Deep Q-Learning Objective

$$\text{Err}(w) = \left(Q_w(s, a) - \underbrace{[r + \gamma \max_{a'} Q_w(s', a')]}_{\text{target}} \right)^2.$$

The gradient with respect to w is:

$$\frac{\partial \text{Err}(w)}{\partial w} = 2 \left(Q_w(s, a) - r - \gamma \max_{a'} Q_w(s', a') \right) \frac{\partial Q_w(s, a)}{\partial w}.$$

Algorithm 13 Gradient Q-Learning

Initialize weights w randomly; observe initial state s
loopSelect and execute action a ; observe r, s'

$$\delta \leftarrow Q_w(s, a) - r - \gamma \max_{a'} Q_w(s', a')$$

$$w \leftarrow w - \alpha \delta \frac{\partial Q_w(s, a)}{\partial w}$$

 $s \leftarrow s'$ **end loop**

77.2 Convergence Issues

With nonlinear approximation, gradient Q-learning is not guaranteed to converge. Each weight update changes Q_w everywhere in state-action space, not just at the current (s, a) — updates intended to reduce error at one point can increase it at others, causing oscillations or divergence.

Two structural remedies stabilize training:

77.2.1 Experience Replay

Instead of updating on the latest transition, store all experienced tuples (s, a, r, s') in a **replay buffer**. At each step, sample a random mini-batch from the buffer and update on it.

- **Decorrelates** successive samples: consecutive environment steps are highly correlated; random mini-batches break this dependency, improving gradient stability.
- **Data efficiency**: each experience can be used multiple times.

77.2.2 Target Network

Maintain a second network $Q_{w'}$ — the **target network** — whose weights w' are held fixed for c steps. The update becomes:

$$w \leftarrow w - \alpha \left(Q_w(s, a) - r - \gamma \max_{a'} Q_{w'}(s', a') \right) \frac{\partial Q_w(s, a)}{\partial w}.$$

Every c steps, synchronize: $w' \leftarrow w$. Fixing the target prevents the moving-target problem, where chasing a target that itself changes at every step causes instability.

78 Deep Q-Network (DQN)**78.1 Algorithm**

DQN (Mnih et al., Nature 2015) combines gradient Q-learning, deep neural networks, experience replay, and a target network:

DQN Key Innovations

1. **Deep neural network** as Q -function approximator — enables high-dimensional state spaces (e.g., raw pixel inputs for Atari games).
2. **Experience replay** — breaks temporal correlations, improves sample efficiency.
3. **Target network** — stabilizes the regression target, prevents divergence.

DQN achieved above human-level performance on a majority of 49 Atari games using raw pixel inputs, demonstrating that a single architecture and learning algorithm can master diverse sequential decision tasks.

Algorithm 14 Deep Q-Network (DQN)

```

Initialize weights  $w, w'$  randomly; initialize replay buffer  $\mathcal{B}$ ; observe  $s$ 
loop
  Select  $a$  via  $\varepsilon$ -greedy on  $Q_w$ ; execute; observe  $r, s'$ 
  Add  $(s, a, r, s')$  to  $\mathcal{B}$ 
  Sample mini-batch  $\mathcal{M} \subset \mathcal{B}$ 
  for each  $(s, a, r, s') \in \mathcal{M}$  do
     $\delta \leftarrow Q_w(s, a) - r - \gamma \max_{a'} Q_{w'}(s', a')$ 
     $w \leftarrow w - \alpha \delta \frac{\partial Q_w(s, a)}{\partial w}$ 
  end for
   $s \leftarrow s'$ 
  Every  $c$  steps:  $w' \leftarrow w$ 
end loop

```

79 DRL for Robotics

79.1 Challenges and Sim-to-Real Transfer

Training directly on a physical robot is slow, costly, and potentially unsafe. The standard approach is **sim-to-real transfer**:

1. Train the neural network agent in a high-fidelity simulator (e.g., Unity, Gazebo).
2. Transfer the learned weights to the real robot, possibly with a fine-tuning step.

The main challenge is the **reality gap**: discrepancies between simulator physics and the real world degrade policy performance. Domain randomization (varying simulator parameters during training) is a common mitigation.

79.2 Continuous vs Discrete Action Spaces

Most DRL algorithms for robotics use continuous action spaces (PPO, DDPG), which naturally handle continuous motor commands. However, continuous approaches are computationally expensive.

Discrete DQN for mapless navigation (Macchesini and Farinelli, ICRA 2020): proper discretization of the action space allows DQN to train in approximately 1 hour versus 20 hours for continuous methods, with comparable navigation performance. The agent takes as input the target heading, target distance, and 19 sparse laser scan values from a 45° forward cone, and outputs a discrete angular velocity command.

79.3 Extensions to Non-Stationary Environments

In aquatic navigation, the environment is non-stationary (currents, waves). DRL agents trained with domain randomization, combined with formal verification tools to detect safety violations, have been shown to navigate successfully under such conditions (Macchesini et al., IROS 2021; Covi et al., RLC 2024).

80 Summary

- An **MDP** $(\mathcal{S}, \mathcal{A}, R, P)$ formalizes sequential decision making under uncertainty. The Markov property reduces history to the current state.

- The **Q-function** $Q_\pi(s, a)$ measures the expected discounted return of taking action a in state s under policy π . The optimal policy is $\pi^*(s) = \arg \max_a Q^*(s, a)$.
- **Tabular Q-learning** converges to Q^* but requires an explicit table — intractable for large or continuous state spaces.
- **Neural networks** are universal function approximators suitable for representing $Q_w(s, a)$. ReLU activations mitigate the vanishing gradient problem that affects sigmoid/tanh networks.
- **Gradient Q-learning** minimizes the squared TD error via gradient descent but may not converge with nonlinear approximation.
- **DQN** stabilizes learning with (i) experience replay (decorrelates updates, improves efficiency) and (ii) a target network (fixes the regression target for c steps). This combination achieved superhuman Atari performance.
- In robotics, sim-to-real transfer is the standard training pipeline. Discretizing the action space enables DQN to train orders of magnitude faster than continuous methods at comparable performance.

Part XVI

Policy-Based and Actor-Critic Deep RL

81 Taxonomy of Deep RL Approaches

The three main families of deep RL methods differ in what they represent explicitly:

Family	Explicit representation	Key characteristic
Value-based (e.g. DQN)	$Q_w(s, a)$ only	Policy is implicit: $\pi(s) = \arg \max_a Q_w(s, a)$
Policy-based (e.g. REINFORCE)	$\pi_\theta(a s)$ only	No value function; direct policy search
Actor-Critic (e.g. A2C, PPO)	Both π_θ and V_w	Actor updates policy; Critic evaluates it

Policy-based and actor-critic methods support **stochastic policies**, handle **continuous action spaces** naturally, and are better suited to robotics than pure value-based methods.

82 Stochastic Policy Representations

A parametric stochastic policy defines a conditional distribution over actions:

$$\pi_\theta(a | s) = \Pr(a | s; \theta).$$

Softmax policy (discrete actions): assign a real-valued score $h(s, a, \theta)$ to each action and normalize:

$$\pi_\theta(a | s) = \frac{\exp(h(s, a, \theta))}{\sum_{a'} \exp(h(s, a', \theta))}.$$

The score can be *linear*: $h(s, a, \theta) = \theta^\top \phi(s, a)$, or *nonlinear*: a neural network $h = \text{ANN}(s, a, \theta)$.

Gaussian policy (continuous actions):

$$\pi_\theta(a | s) = \mathcal{N}(a; \mu(s, \theta), \Sigma(s, \theta)).$$

A neural network maps the state s to the parameters (μ, σ) of the distribution. The mean μ is the nominal action; the standard deviation σ provides learnable exploration noise. No activation is applied to the output layer since μ and σ can take any real value (with σ exponentiated afterward to enforce positivity).

83 The Policy Gradient Theorem

83.1 Objective

Define the performance objective as the value of the start state s_0 under the current policy:

$$J(\theta) = V_\theta(s_0) = \sum_a \pi_\theta(a | s_0) Q_\theta(s_0, a).$$

The goal is to find $\theta^* = \arg \max_\theta J(\theta)$ via gradient ascent.

83.2 Theorem Statement

Policy Gradient Theorem

$$\nabla_\theta J(\theta) \propto \sum_s \mu_\theta(s) \sum_a \nabla_\theta \pi_\theta(a | s) Q_\theta(s, a),$$

where $\mu_\theta(s)$ is the on-policy stationary state distribution under π_θ , and $Q_\theta(s, a)$ is the action-value function under π_θ .

Remark. Using the log-derivative trick $\nabla_\theta \pi_\theta = \pi_\theta \nabla_\theta \log \pi_\theta$, the theorem can be rewritten as an expectation:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta} [\nabla_\theta \log \pi_\theta(A | S) Q_\theta(S, A)].$$

This form is directly estimable from sampled trajectories.

84 REINFORCE (Monte Carlo Policy Gradient)

84.1 Algorithm

REINFORCE estimates the policy gradient using full-episode Monte Carlo returns $G_n = \sum_{k=0}^{T-n-1} \gamma^k R_{n+k+1}$:

REINFORCE Update

$$\theta \leftarrow \theta + \alpha \gamma^n G_n \nabla_\theta \log \pi_\theta(A_n | S_n).$$

Algorithm 15 REINFORCE (Monte Carlo Policy Gradient)

```

Initialize policy parameters  $\theta$  arbitrarily
while not converged do
    Generate episode  $S_0, A_0, R_1, \dots, S_{T-1}, A_{T-1}, R_T$  following  $\pi_\theta$ 
    for  $n = 0, \dots, T-1$  do
         $G_n \leftarrow \sum_{k=n+1}^T \gamma^{k-n-1} R_k$ 
         $\theta \leftarrow \theta + \alpha \gamma^n G_n \nabla_\theta \log \pi_\theta(A_n | S_n)$ 
    end for
end while
    
```

84.2 Interpretation of the Update

The update step has three factors:

- G_n : scales the update by how good the trajectory was — good returns reinforce the actions that produced them.
- $\nabla_{\theta} \log \pi_{\theta}(A_n | S_n)$: the direction that increases the probability of action A_n in state S_n .
- $1/\pi_{\theta}(A_n | S_n)$ (implicit in the log-derivative): down-weights actions that are already selected frequently, preventing the update from over-fitting to high-probability actions regardless of their returns.

84.3 Limitation: High Variance

REINFORCE has high variance because G_n depends on the entire future trajectory, which is noisy. This is addressed by subtracting a baseline.

85 Baseline and Variance Reduction

85.1 Adding a Baseline

Any function $b(s)$ that does not depend on the action can be subtracted from $Q_{\theta}(s, a)$ without biasing the gradient:

$$\sum_a \nabla_{\theta} \pi_{\theta}(a | s) b(s) = b(s) \nabla_{\theta} \underbrace{\sum_a \pi_{\theta}(a | s)}_{=1} = 0.$$

Therefore:

$$\nabla_{\theta} J(\theta) \propto \sum_s \mu_{\theta}(s) \sum_a \nabla_{\theta} \pi_{\theta}(a | s) [Q_{\theta}(s, a) - b(s)].$$

85.2 Advantage Function

The natural baseline choice is $b(s) = V^{\pi}(s)$, which gives the **advantage function**:

Advantage Function

$$A(s, a) = Q(s, a) - V^{\pi}(s).$$

$A(s, a) > 0$ means action a is better than average in state s ; $A(s, a) < 0$ means it is worse. The policy update becomes:

$$\theta \leftarrow \theta + \alpha \gamma^n A(S_n, A_n) \nabla_{\theta} \log \pi_{\theta}(A_n | S_n).$$

85.3 REINFORCE with Baseline

Maintain a separate value-function estimator $V_w(s)$ trained alongside the policy:

86 Actor-Critic Methods

86.1 From Monte Carlo to Temporal Difference

REINFORCE requires a complete episode before updating. Actor-critic methods replace the Monte Carlo return G_n with a **one-step TD target**, enabling online updates:

$$\delta_n = R_{n+1} + \gamma V_w(S_{n+1}) - V_w(S_n).$$

Algorithm 16 REINFORCE with Baseline

```

Initialize  $\theta, w$  arbitrarily
while not converged do
  Generate episode  $S_0, A_0, R_1, \dots, S_{T-1}, A_{T-1}, R_T$  following  $\pi_\theta$ 
  for  $n = 0, \dots, T-1$  do
     $G_n \leftarrow \sum_{k=n+1}^T \gamma^{k-n-1} R_k$ 
     $\delta_n \leftarrow G_n - V_w(S_n)$ 
     $w \leftarrow w + \alpha^w \delta_n \nabla_w V_w(S_n)$ 
     $\theta \leftarrow \theta + \alpha^\theta \delta_n \nabla_\theta \log \pi_\theta(A_n | S_n)$ 
  end for
end while

```

This is the TD error — a lower-variance (but biased) estimate of the advantage.

86.2 Actor-Critic (AC) Algorithm**Algorithm 17** Actor-Critic (AC)

```

Initialize  $\theta, w$  arbitrarily;  $S \leftarrow s_0$ ;  $I \leftarrow 1$ 
while not converged do
  while  $S$  not terminal do
     $A \sim \pi_\theta(\cdot | S)$ ; take  $A$ ; observe  $R, S'$ 
     $\delta \leftarrow R + \gamma V_w(S') - V_w(S)$  (TD error)
     $w \leftarrow w + \alpha^w \delta \nabla_w V_w(S)$ 
     $\theta \leftarrow \theta + \alpha^\theta I \delta \nabla_\theta \log \pi_\theta(A | S)$ 
     $I \leftarrow \gamma I$ ;  $S \leftarrow S'$ 
  end while
end while

```

86.3 Advantage Actor-Critic (A2C)

A2C replaces the TD error with the full advantage, using a Q -value critic:

A2C Advantage Estimate

$$\delta = R + \gamma \max_{a'} Q_w(S', a') - Q_w(S, A),$$

$$A(S, A) = R + \gamma \max_{a'} Q_w(S', a') - \sum_a \pi_\theta(a | S) Q_w(S, a).$$

Updates: $w \leftarrow w + \alpha^w \delta \nabla_w Q_w(S, A)$; $\theta \leftarrow \theta + \alpha^\theta A(S, A) \nabla_\theta \log \pi_\theta(A | S)$.

The advantage properly measures how much better action A is relative to the policy's average, leading to faster convergence than the plain TD error.

87 Deterministic Policy Gradient (DPG)

For continuous action spaces, stochastic policies require integrating over actions to compute gradients. **Deterministic policies** $\pi_\theta(s) \rightarrow a$ sidestep this:

Deterministic Policy Gradient Theorem (Silver et al., ICML 2014)

$$\nabla_{\theta} J(\theta) \propto \mathbb{E}_{s \sim \mu_{\theta}} \left[\nabla_{\theta} \pi_{\theta}(s) \nabla_a Q_{\theta}(s, a) \Big|_{a=\pi_{\theta}(s)} \right].$$

The gradient flows from the Q-function through the policy: the actor is updated to move in the direction that the critic says increases Q.

Algorithm 18 Deterministic Policy Gradient (DPG)

```

Initialize  $\theta, w; S \leftarrow s_0; I \leftarrow 1$ 
while not converged do
  while  $S$  not terminal do
     $A = \pi_{\theta}(S)$ ; take  $A$ ; observe  $R, S'$ 
     $\delta \leftarrow R + \gamma Q_w(S', \pi_{\theta}(S')) - Q_w(S, A)$ 
     $w \leftarrow w + \alpha^w \delta \nabla_w Q_w(S, A)$ 
     $\theta \leftarrow \theta + \alpha^{\theta} \nabla_{\theta} \pi_{\theta}(S) \nabla_a Q_w(S, a) \Big|_{a=\pi_{\theta}(S)}$ 
     $I \leftarrow \gamma I; S \leftarrow S'$ 
  end while
end while

```

Remark. DDPG (Deep Deterministic Policy Gradient) combines DPG with the DQN stabilization techniques: experience replay and a target network, enabling stable training on high-dimensional continuous control tasks.

88 Trust Region Methods

88.1 Step-Size Problem in Policy Gradient

The learning rate α in policy gradient is critical:

- Too small: slow convergence.
- Too large: the policy changes drastically, potentially collapsing to a bad region from which recovery is difficult.

Gradient descent in parameter space θ is problematic because small $\Delta\theta$ can cause large changes in π_{θ} and hence in V .

88.2 Trust Region Optimization

Trust region methods replace unconstrained gradient steps with constrained optimization: find the best update within a region where the local approximation is trusted.

Trust Region Update

Given a surrogate objective $\tilde{f}(x)$ (e.g., quadratic Taylor approximation of f around x_0):

$$x^* = \arg \min_{x: \|x-x_0\|_2 \leq \delta} \tilde{f}(x).$$

If $f(x^*) < f(x_0)$: accept the step and enlarge δ . Otherwise: reject and shrink δ .

88.3 KL Divergence as Policy Distance

Constraining $\|\theta - \theta'\|$ is a poor measure of policy change. The preferred measure is the KL divergence between the old and new policy distributions:

$$D_{\text{KL}}(\pi_{\theta}(\cdot | s), \pi_{\theta'}(\cdot | s)) = \sum_a \pi_{\theta}(a | s) \log \frac{\pi_{\theta}(a | s)}{\pi_{\theta'}(a | s)}.$$

A small KL divergence guarantees the policy distribution has not changed much, which implies a small change in the value function.

89 Trust Region Policy Optimization (TRPO)

TRPO Objective (Schulman et al., ICML 2015)

$$\theta \leftarrow \arg \max_{\theta'} \mathbb{E}_{s \sim \mu_{\theta}, a \sim \pi_{\theta}} \left[\frac{\pi_{\theta'}(a | s)}{\pi_{\theta}(a | s)} A_{\theta}(s, a) \right] \quad \text{subject to} \quad \mathbb{E}_{s \sim \mu_{\theta}} [D_{\text{KL}}(\pi_{\theta}(\cdot | s), \pi_{\theta'}(\cdot | s))] \leq \delta.$$

The ratio $\pi_{\theta'}(a | s) / \pi_{\theta}(a | s)$ is an **importance weight** that corrects for the fact that actions were sampled under the old policy π_{θ} . The KL constraint (relaxed from $\max_s$ to $\mathbb{E}_s$ for computational tractability) prevents the new policy from deviating too far.

90 Proximal Policy Optimization (PPO)

90.1 Clipped Surrogate Objective

TRPO requires solving a constrained optimization problem at each step, which is expensive. PPO approximates the trust-region constraint by **clipping** the importance-weight ratio directly in the objective:

PPO Clipped Objective

Let $r_t(\theta) = \frac{\pi_{\theta}(A_t | S_t)}{\pi_{\theta_{\text{old}}}(A_t | S_t)}$. Then:

$$\mathcal{L}^{\text{CLIP}}(\theta) = \mathbb{E}_t [\min(r_t(\theta) A_t, \text{clip}(r_t(\theta), 1 - \varepsilon, 1 + \varepsilon) A_t)],$$

where $\text{clip}(x, 1 - \varepsilon, 1 + \varepsilon)$ clamps x to the interval $[1 - \varepsilon, 1 + \varepsilon]$.

When $A_t > 0$ (action was better than average), clipping prevents r_t from growing above $1 + \varepsilon$, limiting how much the policy shifts toward that action. When $A_t < 0$, clipping prevents r_t from falling below $1 - \varepsilon$, limiting the shift away from it. The min ensures the objective is a pessimistic lower bound.

90.2 PPO Algorithm

PPO is the **de facto standard policy gradient method** in robotics and game-playing (e.g., OpenAI's RoboSoccer, dexterous robotic hand reorientation).

Algorithm 19 Proximal Policy Optimization (PPO)

```

Initialize policy  $\theta$ , critic  $w$ 
while not converged do
  Sample  $s_0$ 
  for  $n = 0, \dots, N - 1$  do
     $a_n \sim \pi_\theta(\cdot \mid s_n)$ ; execute; observe  $r_n, s_{n+1}$ 
     $\delta' \leftarrow r_n + \gamma \max_{a'} Q_w(s_{n+1}, a') - Q_w(s_n, a_n)$ 
     $A(s_n, a_n) \leftarrow r_n + \gamma \max_{a'} Q_w(s_{n+1}, a') - \sum_a \pi_\theta(a \mid s_n) Q_w(s_n, a)$ 
     $w \leftarrow w + \alpha^w \gamma^n \delta' \nabla_w Q_w(s_n, a_n)$ 
  end for
   $\theta \leftarrow \arg \max_{\theta'} \frac{1}{N} \sum_{n=0}^{N-1} \min\{r_n(\theta') A(s_n, a_n), \text{clip}(r_n(\theta'), 1 - \varepsilon, 1 + \varepsilon) A(s_n, a_n)\}$ 
end while

```

91 Soft Actor-Critic (SAC)

91.1 Entropy Regularization

Standard RL maximizes $\mathbb{E}[\sum_t \gamma^t R_t]$. SAC adds an **entropy bonus** to the reward at each step:

Soft MDP Objective

$$J_{\text{soft}}(\pi) = \mathbb{E} \left[\sum_t \gamma^t \left(R(s_t, a_t) + \lambda H(\pi(\cdot \mid s_t)) \right) \right], \quad H(\pi(\cdot \mid s)) = - \sum_a \pi(a \mid s) \log \pi(a \mid s).$$

$\lambda > 0$ controls the trade-off between reward maximization and entropy maximization.

91.2 Soft Greedy Policy

In the soft MDP, the greedy policy with respect to the soft Q-function is the softmax:

$$\pi_{\text{greedy}}(\cdot \mid s) = \text{softmax} \left(\frac{Q^{\text{soft}}(s, \cdot)}{\lambda} \right).$$

As $\lambda \rightarrow 0$, the entropy term vanishes and the soft greedy policy recovers the standard $\arg \max$.

91.3 Benefits

- **Exploration:** maximizing entropy encourages the policy to try all actions proportionally to their value, avoiding premature convergence to suboptimal deterministic strategies.
- **Robustness:** an entropic policy generalizes better to perturbations of the environment.
- **Off-policy:** SAC is off-policy (uses a replay buffer), making it sample-efficient.

SAC is one of the most widely used off-policy baselines in robotics research, performing competitively with or better than PPO, A2C, TRPO, and DDPG on standard MuJoCo benchmarks (Hopper, Walker, HalfCheetah, Ant, Humanoid).

92 Summary

- **Policy gradient theorem:** $\nabla J \propto \mathbb{E}[\nabla_\theta \log \pi_\theta(A \mid S) Q(S, A)]$ — the gradient of the value function is an expectation estimable from sampled trajectories.

- **REINFORCE**: Monte Carlo policy gradient; unbiased but high-variance. A baseline $b(s) \approx V^\pi(s)$ reduces variance without introducing bias.
- **Advantage function** $A(s, a) = Q(s, a) - V(s)$: measures how much better an action is than the policy's average; the natural signal for actor updates.
- **Actor-Critic (AC)**: replaces MC returns with TD targets for online, low-variance updates. A2C uses the full advantage estimate for faster convergence.
- **DPG**: deterministic actor for continuous actions; gradient flows through the Q-function. DDPG adds experience replay and a target network for stability.
- **Trust regions**: constrain policy updates via KL divergence rather than parameter distance, preventing destructive large updates.
- **TRPO**: maximizes a surrogate objective subject to a KL constraint; guaranteed monotonic improvement but expensive per step.
- **PPO**: approximates the trust-region constraint by clipping the importance-weight ratio; simple, scalable, and the standard method in robotics.
- **SAC**: adds entropy regularization to the reward, producing a stochastic off-policy actor-critic that explores broadly and generalizes well; widely used in robotics.

Part XVII

Planning under Uncertainty

93 From Path Planning to Decision-Theoretic Planning

93.1 The Two Regimes

Classical path planning (Parts XII to XIV) assumes a **known state** and **deterministic actions**: the solution is a fixed sequence of movements. Planning under uncertainty relaxes both assumptions:

	Path planning	Planning under uncertainty
Goals	Explicit goal state	Utility function (reward)
Actions	Deterministic	Stochastic (transition model)
State	Fully known	Partially observable
Solution	Sequence of actions	Policy $\pi : s \rightarrow a$

The key shift is from computing a *plan* to computing a *policy* that maps every possible situation to the best action.

93.2 Problem Taxonomy

- **MDP** (Markov Decision Process): stochastic actions, fully observable state.
- **POMDP** (Partially Observable MDP): stochastic actions, state only partially observable through noisy sensors.

POMDPs strictly generalise MDPs and are the natural model for mobile robots, which always have imperfect knowledge of their own pose.

94 Markov Decision Processes (MDPs)

94.1 Formal Definition

An MDP is a tuple $(\mathcal{S}, \mathcal{A}, R, P)$:

- $\mathcal{S}$: finite state space with $|\mathcal{S}| = n$.
- $\mathcal{A}$: finite action space with $|\mathcal{A}| = m$.
- **Transition model**: $p(s' | s, a) = \Pr[S_{t+1} = s' | S_t = s, A_t = a]$.
- **Reward function**: $r(s, a) = \mathbb{E}[R_{t+1} | S_t = s, A_t = a]$.

The **Markov property** asserts that $\Pr[R_{t+1}, S_{t+1} | S_t, A_t]$ does not depend on the history $S_0, A_0, \dots, S_{t-1}, A_{t-1}$. Stationarity further requires this distribution to be time-invariant.

94.2 Policies

- **Stationary**: $\pi : \mathcal{S} \rightarrow \mathcal{A}$ — the same action is chosen in state s regardless of time.
- **Non-stationary**: $\pi : \mathcal{S} \times \mathbb{N} \rightarrow \mathcal{A}$ — actions can depend on the remaining horizon.
- **Stochastic**: $\pi(a | s)$ — a conditional distribution over actions.

For infinite-horizon discounted MDPs, an optimal stationary policy always exists.

94.3 Cumulative Reward and Planning Horizon

The agent maximizes the expected discounted return:

$$R_T = \mathbb{E} \left[\sum_{\tau=1}^T \gamma^\tau r_{t+\tau} \right].$$

Three regimes:

- $T = 1$: greedy — simple but often suboptimal.
- $1 < T < \infty$: finite horizon — typically $\gamma = 1$; optimal policy may be non-stationary.
- $T = \infty$: infinite horizon with $\gamma < 1$ — return is bounded by $r_{\max}/(1 - \gamma)$; optimal policy is stationary.

94.4 Value and Q-Functions

Value and Action-Value Functions

$$\begin{aligned} V_\pi(s) &= \mathbb{E} \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \mid S_t = s, \pi \right], \\ Q_\pi(s, a) &= \gamma \left[r(s, a) + \int p(s' | s, a) V_\pi(s') \, ds' \right], \\ V_\pi(s) &= Q_\pi(s, \pi(s)). \end{aligned}$$

94.5 Optimal Policy and Bellman Equations

The optimal policy π^* satisfies $V_{\pi^*}(s) \geq V_{\pi}(s)$ for all s and all π . Its value function V^* obeys the **Bellman optimality equation**:

Bellman Optimality Equation

$$V^*(s) = \gamma \max_a \left[r(s, a) + \int p(s' | s, a) V^*(s') ds' \right].$$

This is a *self-consistency condition*: the value of each state must equal the best immediate reward plus the discounted future value reachable from it. The optimal policy recovers as $\pi^*(s) = \arg \max_a Q^*(s, a)$.

95 Solving MDPs

95.1 Value Iteration

Value iteration turns the Bellman optimality equation into an iterative update rule. Starting from arbitrary $\hat{V}^{(0)}$, at each sweep every state is backed up:

Bellman Backup (Discrete)

$$\hat{V}^{(k+1)}(s_i) = \gamma \max_a \left[r(s_i, a) + \sum_{j=1}^N p(s_j | s_i, a) \hat{V}^{(k)}(s_j) \right].$$

Algorithm 20 Discrete Value Iteration

```

Initialize  $\hat{V}(s_i) \leftarrow r_{\min}$  for all  $i$ 
while not converged do
  for  $i = 1$  to  $N$  do
     $\hat{V}(s_i) \leftarrow \gamma \max_a \left[ r(s_i, a) + \sum_{j=1}^N p(s_j | s_i, a) \hat{V}(s_j) \right]$ 
  end for
end while
Return  $\hat{V}$ 

```

Once $\hat{V}$ has converged, extract the policy greedily:

$$\pi(s) = \arg \max_a \left[r(s, a) + \sum_{j=1}^N p(s_j | s, a) \hat{V}(s_j) \right].$$

Properties: guaranteed convergence to V^* ; convergence rate is linear in γ . Complexity per sweep is $O(|\mathcal{S}|^2|\mathcal{A}|)$.

95.2 Policy Iteration

Policy iteration alternates between two steps until the policy stabilises:

1. **Policy evaluation:** for the current fixed π , solve the linear system

$$\hat{V}(s_i) = \gamma \left[r(s_i, \pi(s_i)) + \sum_{j=1}^N p(s_j | s_i, \pi(s_i)) \hat{V}(s_j) \right]$$

for all i (can be done exactly or by truncated iteration).

2. **Policy improvement:** update π greedily with respect to $\hat{V}$. Each improvement step strictly increases V_π or terminates at optimality.

Policy iteration typically converges in fewer iterations than value iteration, but each step is more expensive (requires solving a linear system).

96 Partially Observable MDPs (POMDPs)

96.1 Motivation: Partial Observability

In most realistic robot scenarios, the state s is *not* directly observable. Sensors provide noisy, ambiguous measurements z ; the robot cannot know its exact pose. Defining a policy $\pi(s)$ over the true state is therefore ill-posed.

A common but flawed workaround is to replace the true state with the **maximum likelihood estimate** $\hat{s}_t = \arg \max_s p(s_t \mid z_{1:t}, u_{1:t})$, ignoring estimation uncertainty. This can lead to catastrophic decisions near regions of high belief ambiguity.

96.2 Belief State

The principled solution is to maintain a **belief**:

$$b_t(s) = p(s_t \mid z_{1:t}, u_{1:t}).$$

The belief is a probability distribution over states, updated recursively via Bayes filtering (as in Parts VII to IX). The belief is a **sufficient statistic** for the entire history.

96.3 POMDP Formulation

A POMDP extends the MDP with an observation model:

- $\mathcal{Z}$: observation space.
- $O(z \mid s', a) = \Pr[Z_{t+1} = z \mid S_{t+1} = s', A_t = a]$: observation model.

The policy now maps beliefs to actions: $\pi : \mathcal{B} \rightarrow \mathcal{A}$, where $\mathcal{B}$ is the space of all probability distributions over $\mathcal{S}$. The Bellman equation becomes:

POMDP Bellman Equation

$$V_T(b) = \gamma \max_a \left[r(b, a) + \int p(b' \mid b, a) V_{T-1}(b') db' \right],$$

where $r(b, a) = \sum_s b(s) r(s, a)$ is the expected reward under belief b .

96.4 Exact Solution: α -Vectors

For finite state/action/observation spaces and a finite horizon T , the value function $V_T(b)$ is **piecewise linear and convex** (PWLC) in the belief b :

$$V_T(b) = \max_{\alpha \in \Gamma_T} \sum_s \alpha(s) b(s),$$

where Γ_T is a finite set of $|\mathcal{S}|$ -dimensional vectors called **α -vectors**. Each α -vector corresponds to a conditional plan (a policy tree). The set Γ_T is built inductively by incorporating actions and observations into the beliefs.

Remark. The number of α -vectors can grow doubly exponentially with the horizon, making exact POMDP solvers tractable only for small problems. Approximate methods are essential for robotics.

97 Monte Carlo Tree Search (MCTS)

97.1 Core Idea

MCTS is an anytime, sample-based tree-search algorithm that focuses computational effort on promising parts of the search space. It requires only a **black-box simulator** $(s_{t+1}, o_{t+1}, r_{t+1}) \sim \mathcal{G}(s_t, a_t)$ rather than an explicit model.

Each node stores an estimated value $\bar{V}$ and a visit count n . The algorithm iterates four phases:

1. **Selection:** traverse the existing tree from the root, at each node choosing the child that maximises the **UCB1** criterion:

$$\text{UCB1}(i) = \bar{V}_i + C \sqrt{\frac{\ln N}{n_i}},$$

where N is the parent's visit count, n_i is child i 's visit count, and C balances exploration vs exploitation.

2. **Expansion:** once a node with unexplored children is reached, add one (or all) new child nodes to the tree.
3. **Simulation (rollout):** from the newly expanded node, simulate to a terminal state using a *default policy* (e.g., random action selection).
4. **Backpropagation:** propagate the terminal reward back up the traversed path, updating $\bar{V}$ and n for each ancestor.

Algorithm 21 MCTS: Selection and Expansion

```

Current  $\leftarrow S_0$ 
while Current is not a leaf do
    Current  $\leftarrow$  child maximising UCB1
end while
if  $n_{\text{Current}} = 0$  then
    Rollout(Current)
else
    Add all children of Current to the tree
    Current  $\leftarrow$  first new child
    Rollout(Current)
end if

```

Algorithm 22 MCTS: Rollout

```

while  $S_i$  is not terminal do
     $A_i \leftarrow$  random(available_actions( $S_i$ ))
     $S_i \leftarrow$  simulate( $A_i, S_i$ )
end while
Return Value( $S_i$ )

```

Key properties:

- **Anytime:** can be stopped at any point; more iterations improve the estimate.

- **Asymmetric:** deeper exploration of promising branches.
- UCB1 ensures that, given enough simulations, MCTS converges to the optimal action.
- Embarrassingly parallelisable.
- Does *not* require a closed-form dynamics model — only a simulator.

98 POMCP: Monte Carlo Planning in POMDPs

98.1 Challenges of Exact POMDP Solvers

Exact POMDP value iteration suffers from two curses:

- **Curse of dimensionality:** belief space $\mathcal{B}$ is a continuous $(|\mathcal{S}| - 1)$ -simplex.
- **Curse of history:** the number of distinct histories grows exponentially with time.

98.2 POMCP Approach

POMCP (Silver and Veness, NeurIPS 2010) solves both curses simultaneously:

- Replace the belief simplex with an **unweighted particle filter**.
- Replace full tree search with **MCTS over histories**.

98.2.1 Tree Structure

Each node corresponds to a history $h = (a_1, o_1, a_2, o_2, \dots)$ and stores:

- $N(h)$: number of times history h was visited.
- $V(h)$: average simulated return from h .
- $B(h)$: particle set approximating $b(s | h) = p(s | h)$.

Action selection within the tree uses UCB:

$$V^{\text{UCB}}(ha) = V(ha) + C \sqrt{\frac{\ln N(h)}{N(ha)}}.$$

98.2.2 Belief Update via Particle Filter

At each simulation step, a particle is sampled from $B(h_t)$, passed to the black-box simulator to obtain a candidate next state s' and observation o' . The particle is *accepted* only if $o' = o_t$ (matches the real observation). The process repeats until K particles are collected for $B(h_{t+1})$.

POMCP Belief Approximation

$$\bar{B}(s, h_t) = \frac{1}{K} \sum_{i=1}^K \mathbf{1}[s_i = s], \quad s_i \in B(h_t).$$

As $K \rightarrow \infty$, $\bar{B}(\cdot, h_t) \rightarrow p(\cdot | h_t)$. In practice, **particle degeneracy** can occur for long histories; reinitialisation or prior knowledge injection mitigates this.

98.2.3 Convergence

If the belief state is correctly maintained, POMCP converges to the optimal policy for any finite-horizon POMDP as the number of simulations grows. The policy is never explicitly represented — it is implicitly encoded in the search tree, which is rebuilt online at each decision step.

98.3 Comparison: MDP vs POMDP Planning

	MDP	POMDP / POMCP
State known?	Yes (fully observable)	No (belief maintained)
Optimal solution	Value / policy iteration	α -vectors (exact), POMCP (approx.)
Belief representation	N/A	Gaussian (KF/EKF), particles (PF)
Scalability	$O(\mathcal{S} ^2 \mathcal{A})$	POMCP scales via simulation budget
Model required	Explicit p, r	Black-box simulator sufficient

99 Summary

- **MDPs** model stochastic-action, fully-observable sequential decision problems. The optimal policy satisfies the Bellman optimality equation; it is found by value iteration ($O(|\mathcal{S}|^2|\mathcal{A}|)$ per sweep) or policy iteration (fewer iterations, more expensive per step).
- The **Bellman backup** $\hat{V}(s) \leftarrow \gamma \max_a [r(s, a) + \sum_{s'} p(s' | s, a) \hat{V}(s')]$ is the fundamental update rule; repeated application converges to V^* .
- **POMDPs** extend MDPs to partially observable settings. The belief $b(s) = p(s | z_{1:t}, u_{1:t})$ is a sufficient statistic; the policy maps beliefs to actions.
- The POMDP value function is piecewise linear and convex in b , representable by α -vectors. Exact solvers are intractable for large problems.
- **MCTS** is an anytime, simulator-driven tree search. UCB1 balances exploration and exploitation; the algorithm is asymptotically optimal given enough simulations and does not require a closed-form model.
- **POMCP** combines MCTS with an unweighted particle filter to solve large POMDPs. Histories replace belief-space points; Monte Carlo rollouts estimate values; the belief is updated by acceptance sampling against real observations.